

UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO – BCC

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

JOINVILLE

2025

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Adriano Fiorese

JOINVILLE

2025

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Adriano Fiorese

Orientador:

BANCA EXAMINADORA:

Dr. Adriano Fiorese
UDESC

Membros:

Dra. Débora Cabral Nazário
UDESC

M.e. Paulo Roberto Vieira Jr.
IFPR

Joinville, 01 de maio de 2025

RESUMO

No século XXI, sistemas digitais na Internet se consolidaram como principal meio de consumo de conteúdo em áudio e vídeo. Em decorrência disso, projetistas têm tido como desafio a realização da otimização contínua desses sistemas de modo a garantir que eles ofereçam a melhor QoE (*Quality of Experience*) ao usuário final, sobretudo em meio à restrição econômica de compatibilidade com a tecnologia já implementada. Devido ao grande volume de dados sendo trafegados, pode-se inferir que há padrões de consumo de conteúdo específicos e heterogêneos, variando de acordo com as especificações e contextos de cada sistema. Dessa forma, sabe-se que os balanceadores de carga são elementos-chave no processo de roteamento de requisições e, portanto, sua otimização é de grande valia para o processo de otimização de conteúdo. Portanto, o presente trabalho propõe a elaboração, implementação análise de um algoritmo balanceador de carga especificamente para servidores de distribuição de conteúdo com base em monitoramento de memória principal e secundária.

Palavras-chave: Balanceamento de Carga. Sistemas Distribuídos. Distribuição de Conteúdo. Monitoramento de Memória. *Streaming* de Vídeo. CDN (*Content Delivery Network*). *Cache* de Servidores. Arquitetura de Sistemas. QoE (*Quality of Experience*). Redes de Computadores. Heurísticas de Balanceamento.

ABSTRACT

In the 21st century, digital systems on the Internet have established themselves as the primary means for consuming audio and video content. Consequently, *designers* have faced the challenge of continuously optimizing these systems to ensure they offer the best Quality of Experience (QoE) to the end-user, especially amidst the economic constraint of maintaining compatibility with already implemented technology. Due to the large volume of data being transferred, it can be inferred that there are specific and heterogeneous content consumption patterns, varying according to the specifications and contexts of each system. Thus, it is known that load balancers are key elements in the request routing process, and therefore, their optimization is of great value to the content optimization process. Therefore, this work proposes the design, implementation, and subsequent analysis of a load balancing algorithm specifically for content delivery servers based on primary and secondary memory monitoring.

Keywords: Load Balancing. Distributed Systems. Memory Monitoring. Video *Streaming*. CDN (*Content Delivery Network*). Server Caching. Systems Architecture. QoE (*Quality of Experience*). Computer Networks. Balancing Heuristics.

LISTA DE ILUSTRAÇÕES

Figura 1 – Estrutura básica de uma CDN.	24
Figura 2 – Fluxo detalhado das múltiplas camadas de <i>caching</i> em CDNs.	25
Figura 3 – Arquitetura geral do sistema.	38
Figura 4 – Aplicação cliente MPEG-DASH com visualização em tempo real de métricas de QoE: gráfico de qualidade, gráfico de <i>bitrate</i> e gráfico de travamentos. . .	48
Figura 5 – Painel de monitoramento em tempo real mostrando métricas de consumo de memória, taxa de leitura de disco e requisições ativas dos servidores MPEG-DASH.	50
Figura 6 – Evolução da qualidade de vídeo no Cenário 1 (100 usuários, servidores heterogêneos).	58
Figura 7 – Evolução da qualidade de vídeo no Cenário 2 (100 usuários, servidores homogêneos).	59
Figura 8 – Evolução da qualidade de vídeo no Cenário 3 (1000 usuários, servidores heterogêneos).	59
Figura 9 – Evolução da qualidade de vídeo no Cenário 4 (1000 usuários, servidores homogêneos).	60
Figura 10 – Evolução do <i>bitrate</i> por segmento no Cenário 1 (100 usuários, servidores heterogêneos).	61
Figura 11 – Evolução do <i>bitrate</i> por segmento no Cenário 2 (100 usuários, servidores homogêneos).	61
Figura 12 – Evolução do <i>bitrate</i> por segmento no Cenário 3 (1000 usuários, servidores heterogêneos).	62
Figura 13 – Evolução do <i>bitrate</i> por segmento no Cenário 4 (1000 usuários, servidores homogêneos).	62
Figura 14 – Distribuição dos eventos de <i>stall</i> no Cenário 1 (100 usuários, servidores heterogêneos).	63
Figura 15 – Distribuição dos eventos de <i>stall</i> no Cenário 2 (100 usuários, servidores homogêneos).	64
Figura 16 – Distribuição dos eventos de <i>stall</i> no Cenário 3 (1000 usuários, servidores heterogêneos).	64
Figura 17 – Distribuição dos eventos de <i>stall</i> no Cenário 4 (1000 usuários, servidores homogêneos).	65
Figura 18 – Comparação da latência média de resposta por cenário e estratégia de balanceamento.	66

LISTA DE ABREVIATURAS E SIGLAS

AAC	<i>Advanced Audio Coding</i> (Codificação de áudio avançada)
ABR	<i>Adaptive Bitrate</i> (Taxa de Bits Adaptativa)
ALB	<i>Application Load Balancer</i>
AMQP	<i>Advanced Message Queuing Protocol</i>
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicação)
AV1	<i>AOMedia Video 1</i> (codec de vídeo)
AWS	<i>Amazon Web Services</i>
BGP	<i>Border Gateway Protocol</i> (Protocolo de Gateway de Borda)
BOLA	<i>Buffer Occupancy-based Lyapunov Algorithm</i>
CDN	<i>Content Delivery Network</i> (Rede de Entrega de Conteúdo)
CPU	<i>Central Processing Unit</i> (Unidade Central de Processamento)
DASH	<i>Dynamic Adaptive Streaming over HTTP</i>
DLQ	<i>Dead Letter Queue</i> (Fila de Mensagens Mortas)
DNS	<i>Domain Name System</i> (Sistema de Nomes de Domínio)
DSR	<i>Direct Server Return</i> (Retorno Direto do Servidor)
gRPC	<i>Google Remote Procedure Call</i>
H.264	Codec de vídeo (também conhecido como AVC)
H.265	Codec de vídeo (também conhecido como HEVC)
HEVC	<i>High Efficiency Video Coding</i> (Codificação de Vídeo de Alta Eficiência)
HTTP	<i>Hypertext Transfer Protocol</i> (Protocolo de Transferência de Hipertexto)
HTTPS	<i>Hypertext Transfer Protocol Secure</i> (Protocolo de Transferência de Hipertexto Seguro)
InP	Provedor de Infraestrutura
IoT	<i>Internet of Things</i> (Internet das Coisas)
IP	<i>Internet Protocol</i> (Protocolo de Internet)
ISP	<i>Internet Service Provider</i> (Provedor de Serviço de Internet)
JSON	<i>JavaScript Object Notation</i>
LSTM	<i>Long Short-Term Memory</i> (Memória de Longo e Curto Prazo)
LXC	<i>Linux Containers</i>

MPC	<i>Model Predictive Control</i> (Controle Preditivo Baseado em Modelo)
MPD	<i>Media Presentation Description</i> (Descrição de Apresentação de Mídia)
MPEG	<i>Moving Picture Experts Group</i>
MQTT	<i>Message Queuing Telemetry Transport</i>
NIC	<i>Network Interface Card</i> (Placa de Rede)
NS	<i>Network Simulator</i> (utilizado no contexto do NS-3)
OSI	<i>Open Systems Interconnection</i>
PoP	<i>Point of Presence</i> (Ponto de Presença)
QoE	<i>Quality of Experience</i> (Qualidade de Experiência)
QoS	<i>Quality of Service</i> (Qualidade de Serviço)
RAM	<i>Random Access Memory</i> (Memória de Acesso Aleatório)
REST	<i>Representational State Transfer</i>
RTP	<i>Real-Time Transport Protocol</i> (Protocolo de Transporte em Tempo Real)
SDN	<i>Software-Defined Network</i> (Rede Definida por Software)
SFTP	<i>Secure File Transfer Protocol</i> (Protocolo de Transferência de Arquivos Seguro)
SOAP	<i>Simple Object Access Protocol</i>
SP	Provedor de Serviço
SSL	<i>Secure Sockets Layer</i> (Camada de Soquetes Seguros)
TCP	<i>Transmission Control Protocol</i> (Protocolo de Controle de Transmissão)
UDP	<i>User Datagram Protocol</i> (Protocolo de Datagrama de Usuário)
URL	<i>Uniform Resource Locator</i> (Localizador Uniforme de Recursos)
VETH	<i>Virtual Ethernet</i> (Ethernet Virtual)
VIP	<i>Virtual IP</i> (IP Virtual)
VMM	<i>Virtual Machine Monitor</i> (Monitor de Máquina Virtual)
VMs	<i>Virtual Machines</i> (Máquinas Virtuais)
VVC	<i>Versatile Video Coding</i> (Codificação de Vídeo Versátil)
WAF	<i>Web Application Firewall</i> (Firewall de Aplicação Web)
WSL	<i>Windows Subsystem for Linux</i> (Subsistema Windows para Linux)
WSDL	<i>Web Services Description Language</i>
XML	<i>Extensible Markup Language</i> (Linguagem de Marcação Extensível)

SUMÁRIO

1	INTRODUÇÃO	11
1.1	O PROBLEMA	12
1.2	JUSTIFICATIVA	13
1.3	OBJETIVOS	13
1.4	HIPÓTESES	14
1.5	METODOLOGIA E ESTRUTURA	15
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	BALANCEADORES DE CARGA	16
2.1.1	Balanceadores de carga em nível de aplicação	17
2.1.2	Balanceadores de carga em nível de rede	18
2.2	MPEG-DASH	19
2.2.1	<i>Content Steering</i>	20
2.3	SERVIDORES DE DISTRIBUIÇÃO DE CONTEÚDO E CDNS	21
2.3.1	Funcionamento de <i>cache</i> em redes de entrega de conteúdo	23
2.4	MONITORAMENTO DE MEMÓRIA	26
2.4.1	APIs fornecidas pelos principais sistemas operacionais	26
2.4.2	Aplicações	26
2.5	INTERPRETAÇÃO DE DADOS DESATUALIZADOS	27
2.6	VIRTUALIZAÇÃO DE REDES PARA SIMULAÇÃO DE CENÁRIOS	27
2.6.1	Virtualização baseada em Containers Linux	28
2.7	TRABALHOS RELACIONADOS	29
2.8	CONSIDERAÇÕES	31
3	PROPOSTA	32
3.1	O ALGORITMO	32
3.2	OS REQUISITOS TECNOLÓGICOS	36
3.3	CONSIDERAÇÕES	38
4	IMPLEMENTAÇÃO	40
4.1	AMBIENTE DE EXPERIMENTAÇÃO	40
4.1.1	Particularidades do <i>Docker</i> e <i>cgroups</i>	40
4.1.2	Organização do conteúdo e <i>bind mounts</i>	41
4.2	DEFINIÇÃO DOS MÓDULOS	41
4.2.1	Módulos Principais	41
4.2.2	Módulos Auxiliares	42
4.3	DETALHES DE IMPLEMENTAÇÃO DOS MÓDULOS	42
4.3.1	Servidores MPEG-DASH	43

4.3.1.1	<i>Monitoramento e Publicação de Métricas</i>	43
4.3.2	Balanceador de Carga	44
4.3.2.1	<i>Arquitetura e Funcionamento</i>	44
4.3.2.2	<i>Estratégias de Balanceamento Implementadas</i>	45
4.3.3	Gerador de Carga	45
4.3.3.1	<i>Simulação de Clientes MPEG-DASH</i>	45
4.3.3.2	<i>Aplicação Cliente para Validação e Visualização</i>	46
4.3.4	Broker MQTT (NanoMQ)	47
4.3.5	Orquestração com Docker Compose	49
4.3.6	Painel de Monitoramento	49
4.4	GERAÇÃO DE CONTEÚDO	50
4.4.1	Codificação e Segmentação MPEG-DASH	50
4.4.2	Replicação e Distribuição do Conteúdo	52
5	EXPERIMENTOS E RESULTADOS	53
5.1	CENÁRIOS DE TESTE	53
5.1.1	Parâmetros Globais	53
5.1.2	Definição dos Cenários	53
5.1.3	Descrição dos Cenários	53
5.1.3.1	<i>Cenário 1: Baixa Concorrência com Servidores Heterogêneos</i>	53
5.1.3.2	<i>Cenário 2: Baixa Concorrência com Servidores Homogêneos</i>	54
5.1.3.3	<i>Cenário 3: Alta Concorrência com Servidores Heterogêneos</i>	54
5.1.3.4	<i>Cenário 4: Alta Concorrência com Servidores Homogêneos</i>	54
5.1.4	Estratégias de Balanceamento Avaliadas	55
5.2	ANÁLISE COMPARATIVA	55
5.3	VISÃO GERAL DOS RESULTADOS	56
5.4	MÉTRICAS	57
5.4.1	Qualidade	58
5.4.2	Bitrate	60
5.4.3	Número de stalls	63
5.4.4	Latência	65
5.5	ANÁLISE	66
5.5.1	Comparação entre heterogeneidade e homogeneidade	66
5.5.2	Impacto do intervalo de atualização	66
5.5.3	Análise da QoE de modo geral	67
6	CONCLUSÃO	68
6.1	SÍNTESE DOS RESULTADOS	68
6.2	VALIDAÇÃO DAS HIPÓTESES	69

6.3	ROBUSTEZ DO ALGORITMO PROPOSTO	69
6.4	FATORES DETERMINANTES DO DESEMPENHO	70
6.4.1	Heterogeneidade como Fator Crítico	70
6.4.2	Concorrência como Teste de Estresse	71
6.5	CONTRIBUIÇÕES DO TRABALHO	71
6.5.1	Algoritmo de Balanceamento Baseado em Memória	71
6.5.2	Mecanismo de Interpretação de Dados Desatualizados	71
6.5.3	Validação Experimental Abrangente	72
6.5.4	Demonstração da Heterogeneidade como Fator Crítico	72
6.5.5	Implementação de Código Aberto	72
6.5.6	Infraestrutura Experimental Reutilizável	72
6.6	LIMITAÇÕES	72
6.6.1	Ambiente Experimental Virtualizado	73
6.6.2	Escopo de Cenários de Concorrência	73
6.6.3	Algoritmo ABR Único	73
6.6.4	Métricas de Monitoramento Limitadas	73
6.6.5	Conteúdo de Vídeo Único	73
6.6.6	Disponibilidade Garantida de Conteúdo	74
6.7	TRABALHOS FUTUROS	74
6.7.1	Implantação em Ambientes Distribuídos Reais	74
6.7.2	Integração com Sistemas CDN de Produção	74
6.7.3	Aprendizado de Máquina para Predição de Carga	74
6.7.4	Otimização Multi-Métrica	75
6.7.5	Balanceamento Cross-data center	75
6.7.6	Adaptação para Edge Computing e 5G	75
6.7.7	Estudo de Interação com Algoritmos ABR	75
6.7.8	Implementação de <i>Cache</i> Inteligente Integrado	75
6.7.9	Análise de Custos Operacionais	75
6.7.10	Content Steering com Múltiplas CDNs	76
6.7.11	Direct Server Return (DSR)	76
6.7.12	<i>Software-Defined Networking</i> (SDN)	76
6.8	REPRODUTIBILIDADE DOS EXPERIMENTOS	77
6.9	CONSIDERAÇÕES FINAIS	79
	REFERÊNCIAS	81
	GLOSSÁRIO	85

1 INTRODUÇÃO

Consolidada como um produto extremamente popular, a televisão é o dispositivo central na maioria das salas de estar da população mundial. Em decorrência disso, a população se adaptou à prática de consumo de conteúdo em vídeo, criando uma base de consumidores preparada para a revolução do entretenimento digital. Com a popularização da Internet e o surgimento das plataformas de *streaming*, esse hábito de consumo foi potencializado, migrando para um modelo de consumo sob demanda. Essa mudança gerou um crescimento exponencial no volume de dados de vídeo trafegando pela rede, impondo uma pressão sem precedentes sobre a infraestrutura global e tornando o balanceamento de tráfego uma necessidade crítica para garantir a qualidade do serviço aos usuários finais (Cisco Systems, 2020).

Desse modo, pode-se constatar que há desafios no contexto da otimização contínua dos processos de distribuição de conteúdo sob demanda, haja vista que é necessário maximizar a Qualidade de Experiência (QoE) oferecida pelos serviços para cada vez mais usuários. Contudo, é crucial reconhecer que a percepção de qualidade de experiência pelo usuário final (QoE) não depende unicamente da infraestrutura e capacidade dos provedores de serviço. Ela é, na verdade, uma responsabilidade compartilhada, na qual os provedores de acesso à Internet (ISPs) desempenham um papel igualmente fundamental. Frequentemente, a infraestrutura de rede dos ISPs não acompanha a evolução dos provedores de serviço e, por isso, a qualidade percebida pelo usuário pode não ser ótima.

A distribuição de conteúdo de vídeo em larga escala geralmente ocorre por meio de uma arquitetura complexa e geograficamente dispersa, conhecida como Rede de Distribuição de Conteúdo (CDN - *Content Delivery Network*). Nessa estrutura, o conteúdo original é replicado do servidor de origem para múltiplos servidores de borda (*edge servers*) localizados em pontos estratégicos, mais próximos dos usuários finais. Quando um espectador solicita um vídeo, a requisição é direcionada ao servidor de borda geograficamente mais próximo, diminuindo a latência e o tempo de carregamento. Para otimizar esse processo e lidar com picos de audiência e a vasta quantidade de solicitações simultâneas, o balanceamento de carga torna-se indispensável. Ele distribui o tráfego de forma inteligente entre os diversos servidores de conteúdo de vídeo disponíveis, evitando que um único ponto da rede fique sobrecarregado. Essa distribuição equitativa é fundamental para prevenir gargalos, garantir a estabilidade do serviço e, consequentemente, otimizar a experiência de visualização para os consumidores.

Sendo assim, o objetivo principal deste trabalho é elaborar, implementar e analisar um algoritmo de balanceamento de carga para servidores de distribuição de conteúdo com base em monitoramento de memória, comparando-o com estratégias convencionais como *Round-Robin* e Seleção Aleatória, focando na otimização do desempenho no contexto do provedor de serviço.

1.1 O PROBLEMA

Em sistemas distribuídos modernos caracterizados pela replicação de serviços, o balanceador de carga é peça fundamental no processo de maximização do aproveitamento de recursos computacionais e entende-se que sua efetividade está na capacidade de não sobrecarregar nenhum serviço mesmo em cenários de alta demanda (Passos; Fiorese, 2019). De modo simplificado, seu funcionamento consiste em, além de centralizar o roteamento de requisições, oferecer uma abstração para os consumidores, distribuir a carga entre vários servidores de modo a evitar sobrecarga em um ou mais deles. Nesse aspecto, podem ser empregadas várias estratégias para realizar esse roteamento: Seleção aleatória, *Round-Robin*, seleção de um servidor baseada em algoritmos de eleição desenvolvidos para sistemas distribuídos, dentre outros. Enquanto são as mais utilizadas, a seleção aleatória e *Round-Robin* são heurísticas simples de serem implementadas, mas que podem eventualmente causar sobrecarga em um ou mais servidores. A abordagem que contorna esse problema é justamente a eleição do servidor mais apropriado para lidar com a requisição de acordo com parâmetros personalizados que evidenciam, usando alguma estratégia, qual é o servidor a ser escolhido. A escolha dessa estratégia é a área-chave da atuação da proposta deste trabalho.

Além disso, dispositivos de armazenamento persistente (também conhecidos como dispositivos de memória secundária) são muitas vezes pontos de gargalo em sistemas, sobretudo sistemas de distribuição de conteúdo em *streaming*, que exigem acesso aleatório à memória (Mei et al., 2013). Esses dispositivos de armazenamento persistente exibem custos arquiteturais altos (Arpaci-Dusseau et al., 1998). Nesse sentido, em *Data Centers* de larga escala, falhas de dispositivos de memória secundária são a regra e não a exceção (Wu et al., 2018), o que faz com que esses sistemas de alta escalabilidade tenham que ser tolerantes a falhas dos dispositivos de memória secundária. Dessa forma, entende-se que a métrica de taxa de uso de memória secundária é de grande valia no contexto da tomada de decisão de balanceadores de carga de requisições de distribuição de conteúdo. Em decorrência disso, para escalar sistemas na *Web* de forma sólida, uma técnica muito difundida por projetistas de sistemas é o armazenamento de dados na memória principal. A redundância passou a ser característica de sistemas escaláveis na *Web*. Alguns fatores que contribuíam para isso foram: A evolução do número de usuários finais; O consequente surgimento do *Big Data* gerando paradigmas revolucionadores no contexto de Bancos de Dados como os bancos não-relacionais (Tailor Sushant Choudhary, 2014); O uso em massa das CDNs; A popularização da técnica de virtualização como uma forma de obter confiabilidade nos sistemas; dentre outros. Todos esses fatores corroboraram para que a memória principal, sob o seu principal benefício (a velocidade de acesso), fosse utilizada como método primário de armazenamento de dados persistentes, mesmo com a sua volatilidade intrínseca.

Dessa forma, considerando que o papel dos servidores de distribuição de conteúdo é recuperar e entregar dados (não envolvendo tanto processamento quanto outros tipos de serviços dependentes de Unidade Central de Processamento (CPU)), é possível conceber que a métrica

do uso de memória (principal e secundária) é extremamente correlacionada com o desempenho desses servidores - considerando o contexto de múltiplas instâncias de servidores que mantêm seus próprios meios de armazenamento interno - cenário muito presente em CDNs com os servidores de borda (*Edge Servers* ou *Cache Servers*) (Ali; Fang; Khan, 2025).

Diante do exposto, evidencia-se uma lacuna crítica nas estratégias de balanceamento de carga convencionais. Abordagens como *Round-Robin* ou seleção aleatória, ao ignorarem o estado atual dos recursos de memória, podem inadvertidamente direcionar requisições para nós computacionais já sobrecarregados, que estejam sofrendo com alta latência de leitura de disco ou próximos da exaustão de memória principal. Essa falha de percepção não só degrada o desempenho, como também aumenta o risco de falhas em cascata. Portanto, o problema central a ser atacado neste trabalho é a otimização da escolha do servidor em balanceadores de carga para sistemas de distribuição de conteúdo, desenvolvendo uma estratégia que incorpore o monitoramento de métricas de memória (primária e secundária) como fator decisório principal. O objetivo é criar um mecanismo de balanceamento mais inteligente e preditivo, capaz de evitar gargalos de desempenho antes que eles impactem o usuário final, aumentando assim a eficiência e a resiliência do sistema.

1.2 JUSTIFICATIVA

A eficiência dos Sistemas de Distribuição de Conteúdo (CDNs) é intrinsecamente relacionada à capacidade de seus servidores em entregar dados armazenados estaticamente de forma rápida aos usuários finais. Nesse contexto, a memória - tanto a principal utilizada em *cache* quanto em disco - emerge como um recurso crítico, cujo estado reflete diretamente a aptidão de um servidor para responder a novas requisições de conteúdo.

Portanto, a proposta da elaboração e implementação de um algoritmo de balanceamento de carga justifica-se pela necessidade premente de explorar uma estratégia mais especializada e diretamente alinhada com a natureza operacional dos servidores de distribuição de conteúdo. Uma abordagem que utilize o monitoramento sistemático e exclusivo de memória dos servidores como critério para distribuição de carga visa preencher uma lacuna significativa e acredita-se que tal abordagem pode resultar em um sistema mais eficiente, capaz de otimizar a utilização de recursos de *cache*, prevenir a sobrecarga dos servidores de forma mais eficaz e, consequentemente, aprimorar a latência e a experiência do usuário final na entrega do conteúdo.

1.3 OBJETIVOS

Objetivo geral: Elaborar, implementar e analisar um algoritmo de balanceamento de carga para servidores de distribuição de conteúdo com base em monitoramento de memória de servidor de distribuição de conteúdo, comparando-o com estratégias convencionais como *Round-Robin* e Seleção Aleatória;

Como objetivos específicos pode-se listar os seguintes:

- Estudar as principais arquiteturas usadas em servidores de *cache* de CDNs;
- Estudar os principais modelos de persistência de dados em servidores de distribuição de conteúdo;
- Revisar métodos de monitoramento de memória secundária;
- Revisar métodos de monitoramento de memória principal;
- Listar requisitos topológicos da rede e da estrutura dos servidores que definirão o contexto da aplicação da metodologia;
- Levantar estratégias de transmissão contínua de dados de monitoramento de servidores de distribuição de conteúdo para o serviço de balanceador de carga;
- Estabelecer, conceitualmente, algoritmo de escolha de servidores (por parte do balanceador de carga) com base nos dados históricos fornecidos;
- Implementar uma aplicação simples de balanceador de carga, e junto com ela o algoritmo criado;
- Avaliar os resultados obtidos por meio de testes de carga, tendo como principais métricas a latência de resposta, número e duração de travamentos de vídeo, e qualidade de reprodução;

1.4 HIPÓTESES

Com base nos objetivos propostos e na fundamentação teórica que será apresentada, este trabalho parte das seguintes hipóteses a serem validadas experimentalmente:

Hipótese 1: O algoritmo de balanceamento baseado em monitoramento de memória apresentará desempenho superior aos métodos *Round-Robin* e Aleatório em cenários heterogêneos, onde os servidores possuem capacidades distintas de memória e processamento.

Hipótese 2: O intervalo de atualização de 6 segundos das métricas de monitoramento fornecerá melhores resultados que o intervalo de 30 segundos, permitindo que o balanceador responda mais rapidamente às mudanças de estado dos servidores.

Hipótese 3: O algoritmo proposto será robusto mesmo com informações moderadamente desatualizadas, operando eficientemente com intervalos de atualização maiores (30 segundos), demonstrando sua viabilidade em sistemas de larga escala onde atualizações muito frequentes podem ser impraticáveis.

Essas hipóteses serão testadas por meio de experimentos controlados que simularão diferentes cenários de carga e heterogeneidade de servidores. Tanto os experimentos quanto os conceitos relacionados (intervalo de atualização, monitoramento de memória, desatualização de informações, etc.) serão descritos nos capítulos subsequentes.

1.5 METODOLOGIA E ESTRUTURA

Este trabalho consiste na realização das três principais tarefas a seguir: Apresentação e contextualização do problema; Realização de uma revisão de literatura com os objetivos de fundamentar teoricamente o tema e fornecer uma compreensão sólida; Proposição de um modelo de implementação com todos os detalhes técnicos e conceituais necessários. O restante do trabalho é organizado da seguinte maneira: O capítulo 2 apresenta contextualização e fundamentação teórica necessária, o capítulo 3 define a proposta deste trabalho e o capítulo 4 descreve o experimento a ser realizado cujos resultados serão apresentados no capítulo 5. Por fim, o capítulo 6 fornece uma conclusão acerca do experimento e dos resultados.

2 FUNDAMENTAÇÃO TEÓRICA

Para embasar a proposta central deste trabalho, este capítulo se dedica a revisar os conceitos fundamentais. A exploração detalhada destes pilares teóricos é essencial para contextualizar as decisões metodológicas, o planejamento e a implementação subsequentes e a análise dos resultados, conferindo uma base sólida e coesa ao desenvolvimento da pesquisa.

Serão abordados, inicialmente, os balanceadores de carga, detalhando seu funcionamento e classificações, incluindo abordagens em nível de aplicação e em nível de rede, tanto em contextos tradicionais quanto no âmbito das Redes Definidas por *Software* (SDNs). Em seguida, o foco se voltará aos servidores de distribuição de conteúdo e às Redes de Distribuição de Conteúdo (CDNs), explorando as principais arquiteturas empregadas em seus servidores de *cache* e os modelos de persistência de dados utilizados. Isso incluirá uma análise do funcionamento básico do *cache* nesses sistemas, um componente crucial para a eficiência da entrega de conteúdo.

Posteriormente, serão investigadas as estratégias e ferramentas para o monitoramento de memória, tanto principal (*cache*) quanto secundária (disco), em servidores de distribuição de conteúdo. Este estudo se aprofundará nas técnicas para a transmissão contínua dos dados de monitoramento de memória. A compreensão desses mecanismos é vital, pois o uso de memória é um indicador-chave do desempenho em servidores cujo papel principal é a recuperação e entrega de dados.

A consolidação desses conhecimentos permitirá o estabelecimento conceitual de um algoritmo de escolha de servidores pelo balanceador de carga, fundamentado nos dados de monitoramento de memória, alinhando-se ao objetivo geral de elaborar, implementar e analisar tal algoritmo.

2.1 BALANCEADORES DE CARGA

Os balanceadores de carga são componentes cruciais na arquitetura de sistemas distribuídos contemporâneos, incluindo ambientes de computação em nuvem e, por extensão, os Sistemas de Distribuição de Conteúdo (CDNs) que são o foco deste trabalho. A função primordial de um balanceador de carga é distribuir de maneira eficiente e inteligente as cargas de trabalho – como requisições de usuários, tráfego de rede ou tarefas de processamento – entre múltiplos recursos computacionais, tipicamente servidores.

A escalabilidade horizontal, caracterizada pela adição de mais computadores a um sistema, tem-se consolidado como uma estratégia preferencial frente à escalabilidade vertical (o aumento de recursos em um computador existente), por sua maior flexibilidade e otimização de custos (Jane, 2025). Como consequência direta dessa abordagem distribuída, os balanceadores de carga assumem um papel crítico, sendo responsáveis por gerenciar a redundância de servidores e direcionar as requisições de forma inteligente, assegurando assim uma utilização eficaz dos recursos.

Esta distribuição estratégica é vital para otimizar a utilização dos recursos disponíveis,

maximizar a vazão (*throughput*) de requisições atendidas, minimizar o tempo de resposta para o usuário final e, fundamentalmente, evitar a sobrecarga em qualquer servidor individual. Ao prevenir que um único servidor se torne um ponto de estrangulamento, o balanceamento de carga não apenas aprimora o desempenho geral e a capacidade de escalonamento do sistema, mas também eleva significativamente sua confiabilidade e disponibilidade. Isso ocorre, por exemplo, pela capacidade de detectar falhas em servidores e redirecionar o tráfego para os servidores operantes, garantindo a continuidade do serviço.

A eficácia de um sistema distribuído moderno, portanto, está intrinsecamente ligada à sofisticação e adequação de suas estratégias de balanceamento de carga. Tendo em vista que os balanceadores de carga podem ser concebidos em duas principais categorias - em nível de aplicação e em nível de rede (Pandian; Fernando; Haoxiang, 2022) - nas subseções seguintes serão explorados esses conceitos, diferentes tipos e abordagens de balanceadores de carga, fornecendo a base necessária para a discussão de sua otimização no contexto específico da distribuição de conteúdo baseada em monitoramento de memória.

2.1.1 Balanceadores de carga em nível de aplicação

Os balanceadores de carga em nível de aplicação, também conhecidos como balanceadores de Camada 7 (L7) do modelo OSI ou camada de Aplicação no modelo TCP/IP, operam inspecionando os dados da própria aplicação. Isso significa que eles tomam decisões de roteamento com base em informações contidas no tráfego da aplicação, como cabeçalhos HTTP/HTTPS, *cookies*, tipo de conteúdo (imagens, vídeo, texto) ou qualquer outro dado específico da mensagem.

Diferentemente dos balanceadores de nível de rede, os balanceadores L7 analisam o conteúdo do tráfego, o que permite uma distribuição de carga muito mais granular e inteligente. Por exemplo, um balanceador de carga L7 pode direcionar requisições para servidores diferentes com base na URL solicitada (roteamento baseado em caminho), no tipo de navegador do usuário, ou na linguagem preferida indicada no cabeçalho da requisição. Uma requisição para uma URL de um serviço de arquivos pode ser enviada para um *pool* de servidores otimizado para servir imagens, enquanto requisições para uma *API* podem ser encaminhadas dinamicamente para um *pool* de servidores de aplicação.

Quanto às suas vantagens, os balanceadores L7 permitem decisões de roteamento mais inteligentes e flexíveis, o que resulta em maior eficiência na distribuição de carga, especialmente para aplicações *web* complexas. Adicionalmente, eles oferecem a capacidade de descarregar tarefas computacionalmente intensivas, como a terminação SSL, dos servidores, otimizando o uso de recursos destes.

Por outro lado, os balanceadores de nível de aplicação geralmente consomem mais recursos computacionais (CPU e memória) em comparação com os balanceadores de nível de rede, devido à necessidade de inspecionar e processar o conteúdo detalhado dos pacotes. Este processamento adicional também pode introduzir uma latência ligeiramente maior no encaminhamento das requisições.

Alguns exemplos de balanceadores de carga que atuam na camada de aplicação são:

1. **NGINX:** Amplamente utilizado como servidor *web*, *proxy* reverso e balanceador de carga L7, capaz de rotear tráfego HTTP/HTTPS com base em URLs, cabeçalhos, etc.
2. **HAProxy:** Outra solução popular de código aberto que pode operar tanto em L4 (camada de transporte) quanto em L7, oferecendo funcionalidades avançadas de balanceamento HTTP.
3. **AWS Application Load Balancer (ALB):** Um serviço gerenciado da *Amazon Web Services* que opera na camada 7, permitindo roteamento avançado de requisições HTTP/HTTPS.
4. **Azure Application Gateway:** O balanceador de carga L7 da Microsoft Azure, oferecendo funcionalidades como afinidade de sessão baseada em *cookies*, terminação SSL e *firewall* de aplicação *web* (WAF).

2.1.2 Balanceadores de carga em nível de rede

A introdução das Redes Definidas por *Software* (SDNs) trouxe uma nova perspectiva no contexto de balanceamento de carga, permitindo sua realização em nível de rede sem depender de uma aplicação. A arquitetura SDN ao desacoplar o plano de controle do plano de dados, trouxe uma **gestão centralizada e programável do encaminhamento de tráfego**, o que é particularmente vantajoso para as estratégias de balanceamento de carga (Belgaum et al., 2020). O controlador SDN, possuindo uma visão global e em tempo real do estado da rede – incluindo topologia, carga dos enlaces e até mesmo estado dos servidores – pode realizar a maior parte das funções que uma aplicação gerenciadora de balanceamento de carga normalmente pode realizar (Hamdan et al., 2021). Dessa forma, pode-se conceber que balanceadores de carga de nível de aplicação possuem como principal vantagem a flexibilidade em detrimento do desempenho, enquanto balanceadores de carga de nível de rede possuem como principal vantagem o desempenho mas trazendo como desvantagem uma complexidade e configurabilidade elevadas.

Neste paradigma, a lógica do balanceador de carga pode ser implementada como uma aplicação que reside no topo do controlador SDN. Esta aplicação analisa as informações da rede coletadas pelo controlador e, com base em algoritmos de balanceamento predefinidos ou adaptativos, instrui os dispositivos de encaminhamento (como switches *OpenFlow*) sobre como distribuir os fluxos de tráfego entre os servidores disponíveis. Por exemplo, o controlador pode instalar regras nos *switches* para que o tráfego destinado a um determinado endereço IP virtual (VIP) seja distribuído entre um conjunto de endereços IP reais de servidores, utilizando métricas como a menor carga do servidor, menor latência ou maior largura de banda disponível. Esta capacidade de **programação dinâmica das tabelas de fluxo** nos *switches* permite a implementação de políticas de balanceamento de carga granulares e a rápida adaptação a mudanças na rede ou na carga dos servidores, sem a necessidade de reconfiguração manual de dispositivos individuais.

Uma das principais vantagens do balanceamento de carga em SDNs é a otimização do uso dos recursos da rede e dos servidores. Com a visibilidade global, o controlador SDN pode evitar gargalos e subutilização de recursos de forma mais eficaz (Blial; Mamoun; Benaini, 2016). Além disso, a centralização da inteligência de balanceamento no controlador pode simplificar a infraestrutura de rede, potencialmente reduzindo a necessidade de dispositivos de balanceamento de carga dedicados e caros. A programabilidade também facilita a implementação de algoritmos de balanceamento mais sofisticados que podem considerar múltiplos fatores simultaneamente, realizando a gestão inteligente dos fluxos em nível de rede com base em metadados e estatísticas da rede. A capacidade de resposta a eventos da rede, como falhas de servidores ou congestão de enlaces, é também aprimorada, permitindo um redirecionamento de tráfego mais ágil e eficiente para manter a disponibilidade do serviço.

2.2 MPEG-DASH

O padrão MPEG-DASH é apresentado aqui por ser a tecnologia utilizada na implementação experimental deste trabalho, na qual os servidores fornecem segmentos de vídeo DASH e os clientes são *players* compatíveis com este padrão.

O *streaming* via HTTP se consolidou como prática tecnológica mais aplicada para entrega de conteúdo multimídia graças à popularização do consumo de conteúdo na *web*, à consequente abundância de plataformas *web* e às conexões de banda larga (Begen; Akgul; Baugher, 2011). Seguindo essa tendência, um novo padrão - *Dynamic Adaptive Streaming over HTTP* (ou simplesmente DASH) - foi desenvolvido de modo a fornecer dinamismo e flexibilidade em diversas situações distintas de conexões de rede. O padrão contrasta com protocolos mais antigos como o RTP (*Real-Time Transport Protocol*), que frequentemente enfrentam problemas com *firewalls* e exigem que os servidores mantenham um estado de sessão para cada cliente. O HTTP oferece uma abordagem sem estado (*stateless*) mais escalável e compatível com a infraestrutura existente (Sodagar, 2011). Aproveitando essa base, foi desenvolvido o padrão *Dynamic Adaptive Streaming over HTTP* (MPEG-DASH), com o objetivo de criar uma solução interoperável para o *streaming* adaptativo, unificando um mercado anteriormente fragmentado por soluções proprietárias (Sodagar, 2011).

A arquitetura do MPEG-DASH é fundamentalmente orientada ao cliente. O servidor de conteúdo tem a função de preparar e disponibilizar o material, mas toda a inteligência da sessão de *streaming* reside no cliente (Stockhammer, 2011). Isso significa que o servidor não precisa gerenciar o estado da conexão, permitindo que servidores HTTP e redes de distribuição de conteúdo (CDNs) convencionais sejam utilizados para a entrega em larga escala de forma eficiente (Stockhammer, 2011). A operação do padrão se baseia em dois componentes principais:

1. **Media Presentation Description (MPD):** É um arquivo de manifesto, geralmente em formato XML, que descreve a estrutura e as diferentes versões do conteúdo multimídia disponíveis (Sodagar, 2011). O MPD organiza o conteúdo hierarquicamente em Períodos

(*Periods*), que representam intervalos de tempo; Conjuntos de Adaptação (*Adaptation Sets*), que agrupam alternativas de um mesmo componente de mídia (como diferentes idiomas de áudio ou ângulos de câmera); e Representações (*Representations*), que são as diferentes versões de um mesmo fluxo, codificadas com distintas taxas de *bits*, resoluções ou qualidades (Sodagar, 2011). O MPD contém os metadados essenciais, incluindo as URLs para acessar cada parte do conteúdo.

2. **Segmentos:** O conteúdo de mídia em si é dividido em pequenos blocos sequenciais chamados segmentos (Sodagar, 2011). Cada representação possui sua própria sequência de segmentos, que são requisitados individualmente pelo cliente através de requisições HTTP GET padrão (Stockhammer, 2011). Por serem arquivos independentes acessados via HTTP, os segmentos são facilmente armazenáveis em *cache* por *proxies* e CDNs, o que é um dos pilares da escalabilidade do DASH.

O fluxo de trabalho típico de uma sessão DASH começa com o cliente requisitando e analisando o arquivo MPD (Sodagar, 2011). Com base nas informações do manifesto, nas condições atuais da rede (como a largura de banda estimada), nas capacidades do dispositivo (como resolução de tela) e nas preferências do usuário, a lógica de controle do cliente (conhecida como *decision engine* ou *control heuristics*) seleciona a representação mais apropriada para iniciar a reprodução (Begen; Akgul; Baugher, 2011). É importante notar que essa lógica de decisão e adaptação não é padronizada pelo MPEG-DASH, o que confere flexibilidade aos desenvolvedores de clientes (Sodagar, 2011). Após o início da reprodução, o cliente continua a requisitar os segmentos sequencialmente, monitorando o desempenho da rede e, se necessário, trocando para uma representação de maior ou menor qualidade para garantir uma reprodução contínua e com a melhor Qualidade de Experiência, do inglês *Quality of Experience*, (QoE) para o usuário possível.

2.2.1 *Content Steering*

O *Content Steering* (direcionamento de conteúdo) é um mecanismo que estende a capacidade de adaptação do cliente para além da simples seleção da qualidade do fluxo de mídia. Em vez de apenas decidir o que buscar (qual representação), o *Content Steering* permite ao cliente decidir de onde buscar o conteúdo, ou seja, de qual servidor ou CDN obter os segmentos (Sodagar, 2011).

O padrão MPEG-DASH facilita essa funcionalidade diretamente no *Media Presentation Description* (MPD) através da especificação de múltiplas URLs base (*BaseURL*). O MPD pode listar vários endereços para o mesmo conjunto de segmentos, onde cada URL pode apontar para um servidor espelho, um *data center* geograficamente distinto ou até mesmo uma CDN completamente diferente (Sodagar, 2011).

A responsabilidade de escolher qual URL base utilizar recai inteiramente sobre a lógica de controle do cliente, pois o padrão apenas fornece o mecanismo, mas não dita a política de seleção (Sodagar, 2011). Um cliente pode implementar estratégias de direcionamento para:

- **Otimizar o desempenho:** O cliente pode medir a latência ou a vazão para cada uma das URLs base disponíveis e selecionar aquela que oferece o melhor desempenho em um dado momento.
- **Aumentar a confiabilidade:** Caso a requisição para um segmento a partir de uma URL base falhe ou se torne excessivamente lenta, o cliente pode dinamicamente tentar buscar o mesmo segmento (ou os segmentos subsequentes) a partir de uma URL base alternativa, criando um mecanismo de *failover* no nível da aplicação.
- **Balancear a carga:** Em cenários mais avançados, o cliente poderia distribuir suas requisições entre múltiplas fontes de conteúdo para maximizar a largura de banda disponível e evitar sobrecarregar um único servidor ou rota de rede (Sodagar, 2011).

Dessa forma, o *Content Steering* transforma o cliente DASH em um participante ativo no balanceamento de carga e na otimização da entrega, tornando o padrão altamente resiliente e adaptável a implantações de grande escala que envolvem múltiplas CDNs e infraestruturas de servidores complexas.

2.3 SERVIDORES DE DISTRIBUIÇÃO DE CONTEÚDO E CDNS

Esta seção estabelece o contexto de aplicação do algoritmo proposto. Como os servidores de distribuição de conteúdo têm como função principal recuperar e entregar dados, o uso de memória é um indicador-chave de desempenho, justificando sua escolha como métrica de balanceamento.

As Redes de Entrega de Conteúdo (popularmente conhecidas como *Content Delivery Networks* ou CDNs) são uma parte crítica da infraestrutura da Internet, criada para distribuir conteúdo de forma rápida, confiável e eficiente. Elas atingem esse objetivo por meio da distribuição redundante de conteúdo em vários servidores geograficamente dispersos. Quando um usuário realiza uma requisição para um conteúdo por meio de um serviço *web*, a requisição é encaminhada dinamicamente para o servidor mais próximo do usuário em vez de ser enviada para o servidor de origem do conteúdo. Essa é uma prática simples que reduz a latência, minimiza a perda de pacotes (por diminuir o número de saltos) e remove a sobrecarga de servidores que possuem o conteúdo original, resultando em uma QoE mais elevada.

É interessante notar também que as Redes de Entrega de Conteúdo, apesar de serem conjuntos de dispositivos computacionais interconectados assim como as redes de computadores tradicionais, não são necessariamente um subconjunto dessas redes no contexto conceitual. Em vez disso, são sistemas distribuídos de computadores, geralmente administrados em nível de

aplicação por uma organização privada. Em outras palavras, são grupos de computadores que se comunicam de forma padronizada para cumprir com o objetivo de entregar conteúdo para os usuários finais, mas que não estão no mesmo nível de abstração conceitual das redes de computadores. Dessa forma, as CDNs são, na verdade, arquiteturas de aplicação altamente complexas e flexíveis e que, por extensão, estão localizadas no topo da pilha de abstração de camadas de rede - a camada de aplicação.

Nesse sentido, considerando que o princípio central das CDNs é aumentar a taxa de transmissão e diminuir a latência por meio da aproximação física do conteúdo aos usuários, os seus mecanismos-chave são:

1. **Arquitetura:** Predominantemente, as CDNs se organizam em duas camadas principais de servidores: **Os servidores de origem** que detém o conteúdo original e os servidores de borda que possuem cópias redundantes dos conteúdos e são posicionados estrategicamente mais próximos dos usuários finais. Esses servidores de borda são organizados em *Points of Presence* (PoPs), *Data Centers* organizados com o intuito de dispor servidores de borda para as CDNs.
2. **Cache:** As CDNs possuem vários servidores de borda localizados em regiões geográficas distribuídas. Quando um usuário realiza uma requisição para o conteúdo em questão, a CDN responde com o conteúdo do servidor localizado no PoP mais próximo, significativamente reduzindo a distância física sobre qual a requisição precisa trafegar. Esse funcionamento básico reduz a latência e aumenta a taxa de transmissão de dados, ainda sendo efetivo para a Internet sob o ponto de vista de ecossistema, uma vez que há menos requisições geograficamente dispersas evitando sobrecarregar a rede global.
3. **Roteamento:** As CDNs se beneficiam de várias técnicas para determinar o melhor PoP para lidar com uma requisição do usuário. Uma das técnicas para realizar esse processo é a utilização de roteamento baseado em DNS, no qual o sistema DNS da CDN direciona o usuário para o endereço de IP do servidor mais apropriado baseado na localização geográfica.
4. **Balanceamento de carga:** As redes de entrega de conteúdo distribuem requisições dos usuários para evitar que quaisquer servidores de um PoP se sobrecarreguem. Conforme mencionado, o principal fator para distribuição de requisições entre servidores é a localização geográfica. No entanto, outras técnicas mais avançadas também são empregadas, como *anycast* (Preston, 2021), na qual os servidores de borda anunciam possuir o mesmo endereço IP através de um roteador BGP e, naturalmente, o servidor geograficamente mais próximo da requisição efetivamente recebe e a processa.

Para os servidores de borda das CDNs, que estão dispostos em larga escala, se configuram apenas com as tarefas de entrega de conteúdo estático pré-processado (como é o caso do

MPEG-DASH), é correto dizer que eles teriam menos tarefas intensivas de processo específicas de conteúdo no momento da entrega em comparação com, por exemplo, a transcodificação de vídeo instantânea ou a geração de conteúdo dinâmico durante o tratamento de cada requisição. Nesse sentido, faz sentido conceber que os recursos computacionais mais utilizados nesse caso são os recursos de armazenamento - tanto em memória principal quanto em memória secundária.

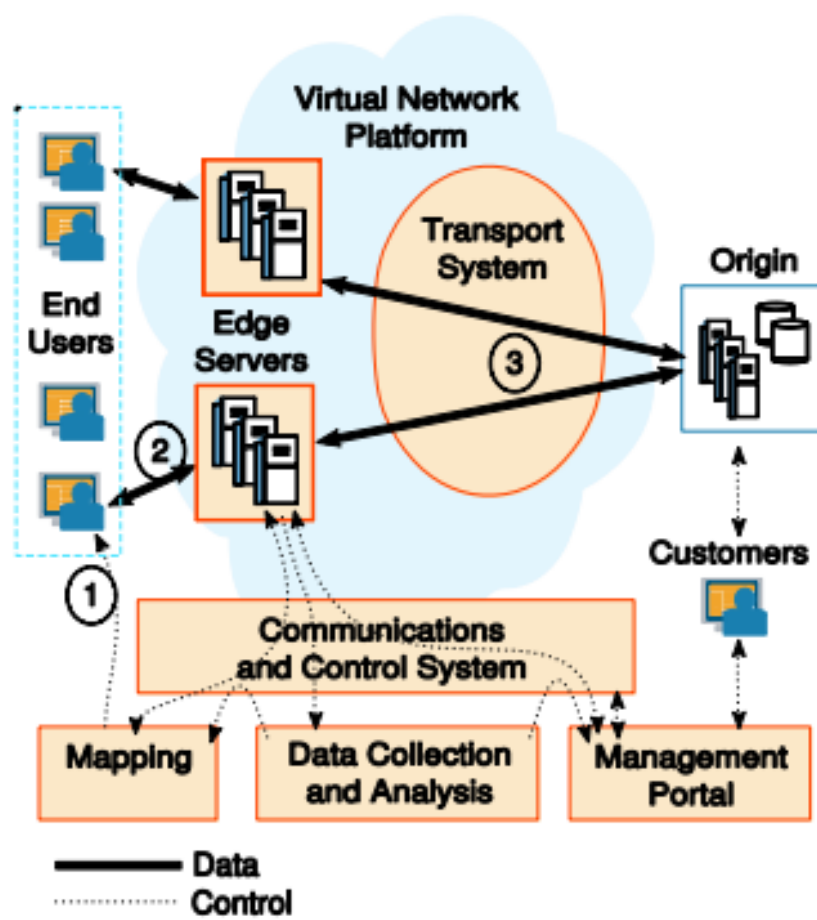
2.3.1 Funcionamento de *cache* em redes de entrega de conteúdo

Na complexidade intrínseca às CDNs, se encontram múltiplas camadas de *caching* de modo a cumprir com o propósito de evitar tráfego em amplas distâncias. Conforme a Figura 2, é possível conceber pelo menos três níveis nas arquiteturas tradicionais de CDNs (Vakali; Pallis, 2003):

1. **Acesso direto ao *Proxy*:** Na primeira iteração de uma requisição que passa por uma CDN, é realizada uma verificação nos servidores de *proxy* dos provedores de rede (ISPs). Os servidores *proxy* são basicamente computadores (físicos ou virtuais) dispostos diretamente nas infraestruturas dos provedores de serviço, mantidos para armazenar conteúdo frequentemente acessado pelos usuários e até mesmo evitar sobrecarga nos servidores de borda das CDNs.
2. **Acesso iterativo aos servidores de borda da CDN:** Em segundo plano, são realizadas tentativas sucessivas de acesso ao conteúdo requisitado através dos servidores de borda da CDN, um por vez. Ao encontrar o conteúdo em questão, ele pode ser armazenado em *cache* nos servidores nos quais o conteúdo não foi encontrado.
3. ***Peering* de CDNs:** Em algumas CDNs específicas, maiores e mais complexas, é possível que seja realizada uma configuração de colaboração (*peering*) que faz com que CDNs possam buscar por conteúdo em outras CDNs, aumentando ainda mais a eficiência.

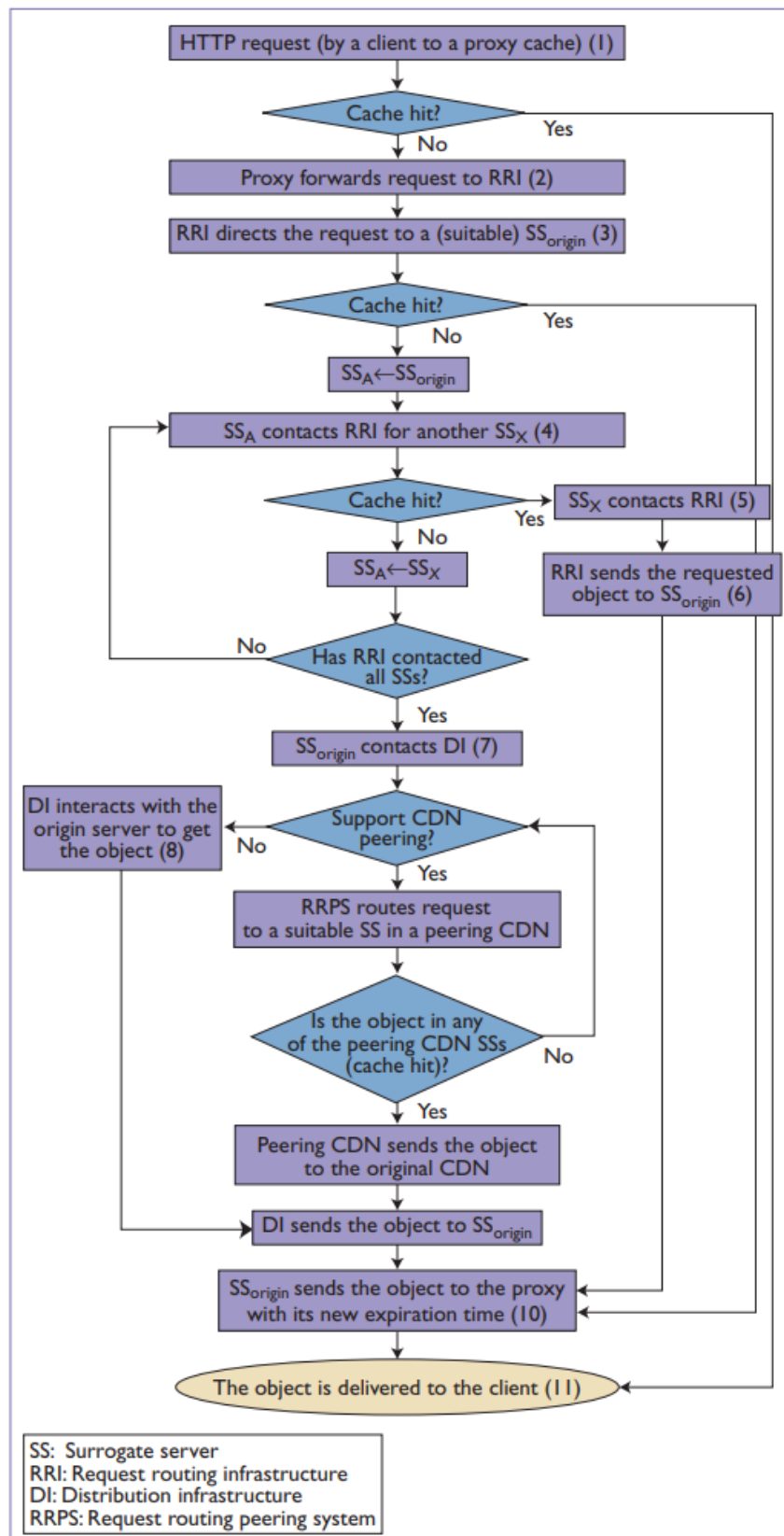
É válido conceber que o funcionamento geral de uma CDN consiste na disposição e organização de servidores (físicos e virtuais) de modo a minimizar a distância percorrida por pacotes de requisições na Internet. Além disso, sabe-se que a ferramenta tecnológica mais utilizada para escolher dinamicamente o servidor conforme o conteúdo requisitado e distâncias geográficas é o roteamento dinâmico de DNS, considerada a técnica mais popular no contexto das CDNs (Wang; Huang; Rose, 2018).

Figura 1 – Estrutura básica de uma CDN.



Fonte: Adaptado de (Nygren; Sitaraman; Sun, 2010)

Figura 2 – Fluxo detalhado das múltiplas camadas de *caching* em CDNs.



Fonte: Adaptado de (Vakali; Pallis, 2003)

2.4 MONITORAMENTO DE MEMÓRIA

Esta seção é central para o trabalho, pois o algoritmo proposto utiliza exclusivamente métricas de memória principal e secundária como parâmetros de decisão para o balanceamento de carga.

Com a emergente necessidade de modernizar e escalar sistemas nos mais diversos setores da indústria, por consequência, emergiu também, no contexto tecnológico, a necessidade da disponibilização de recursos computacionais sob demanda de modo mais flexível possível. Esse fenômeno resultou no paradigma de disponibilização de recursos computacionais de forma mais abstrata e encapsulada, conhecido como Computação em Nuvem (*Cloud Computing*) (Boss et al., 2007). Junto com a aplicação do novo paradigma, também se fez necessária a evolução sistemática dos processos de monitoramento de recursos computacionais e de rede, como vazão de dados, latência, processamento da CPU, uso de memória principal e secundária, dentre outros.

2.4.1 APIs fornecidas pelos principais sistemas operacionais

A principal e mais utilizada maneira de se obter a métrica do consumo de memória é por meio de APIs disponibilizadas pelos Sistemas Operacionais. A maioria dos *Kernels*, durante o gerenciamento de memória e de disco relacionados aos processos gerenciados, possui instruções para armazenar e recuperar informações, atualizadas constantemente, a respeito do consumo de memória e de disco. Embora a disposição dessas informações seja diferente de sistema para sistema, a maior parte deles disponibiliza uma API para que aplicações no espaço do usuário (*user-space*) consigam obter essa informação.

Para o sistema operacional Linux, por exemplo, a interface primária para monitoramento é o diretório virtual `/proc/meminfo`. Através deste diretório, o SO fornece uma análise detalhada do uso da RAM (*Random Access Memory* - ou Memória Principal) (The Linux Kernel Developers, 2020). Além disso, o SO fornece informações detalhadas sobre estatísticas de leitura para cada bloco no disco (memória secundária) por meio do diretório `/proc/diskstats` (The Linux Kernel Developers, 2011). Outros sistemas operacionais possuem seus próprios métodos e APIs para disponibilizar essas informações. Enquanto o *macOS* utiliza subsistemas (Apple Inc., 2013) (Apple Inc., 2012), o *Windows* possui uma API bem definida baseada em requisições denominada *Performance Counters*, que sistematicamente disponibiliza essas informações (Microsoft Corporation, 2023b) (Microsoft Corporation, 2023a).

2.4.2 Aplicações

Com base no princípio anterior, existem várias soluções em nível de aplicação que referenciam o sistema operacional de forma dinâmica para dispor, transportar e processar essas informações de monitoramento de forma sistemática. Alguns exemplos são: o sistema *Tivoli® Monitoring* da IBM (Boss et al., 2007), o clássico *Task Manager* do *Windows*, as soluções de

linha de comando `top`, `htop` e `free` do Linux, dentre outras aplicações que podem funcionar tanto de forma isolada quanto distribuída.

2.5 INTERPRETAÇÃO DE DADOS DESATUALIZADOS

Esta seção é crítica, pois o algoritmo proposto depende de informações de memória coletadas periodicamente, criando um intervalo entre a coleta e o uso durante o qual o estado dos servidores pode mudar. O trabalho "*Interpreting Stale Load Information*" (Dahlin, 2000) fornece a base teórica para o algoritmo implementado, através da adaptação do método *Basic LI*.

No contexto de balanceamento de carga, é fato que a estratégia de direcionamento da carga para a máquina com menos sobrecarga pode ter desempenho ruim se a informação de carga dos servidores estiver relativamente desatualizada (Eager; Lazowska; Zahorjan, 1986; Mirchandaney; Towsley; Stankovic, 1989; Mitzenmacher, 2000; Shivaratri; Krueger; Singhal, 1993). Nesse sentido, sistemas de balanceamento de carga que possuem como estratégia a escolha de servidores pouco sobrecarregados que não tratam devidamente a desatualização da informação de carga dos servidores podem produzir o efeito contrário ao desejado, efetivamente sobrecarregando as máquinas que processam requisições.

A falta de tratamento devido a dados desatualizados pode causar um fenômeno no qual máquinas que aparentam estar subutilizadas rapidamente se tornam sobrecarregadas, uma vez que, durante a defasagem da propagação da informação, todos os clientes enviam suas requisições para as mesmas máquinas, que por sua vez ficam sobrecarregadas até que uma nova informação de carga seja propagada pelo sistema. Para combater esse problema, algumas abordagens comuns incluem o uso de algoritmos de seleção aleatória ou *Round-Robin*, que ignoram completamente os dados de carga, ou o uso de um subconjunto aleatório de servidores para a tomada de decisão.

Por outro lado, se for imprescindível que os dados de sobrecarga de um ou mais recursos computacionais dos servidores gerenciados sejam considerados como variáveis utilizadas pela estratégia de balanceamento de carga, existem algumas estratégias para resolver o problema da desatualização dos dados. O trabalho "*Interpreting Stale Load Information*" (Dahlin, 2000) desenvolve estratégias que interpretam a informação de carga com base em sua idade em vez de arriscar um desempenho extremamente baixo ou ignorar a oportunidade de usar essa informação para otimizar o sistema.

2.6 VIRTUALIZAÇÃO DE REDES PARA SIMULAÇÃO DE CENÁRIOS

A virtualização de redes é apresentada aqui por ser a tecnologia utilizada na implementação experimental, onde toda a infraestrutura foi implementada em contêineres Docker, permitindo a simulação de múltiplos servidores em uma única máquina física.

A virtualização de redes surgiu como uma abordagem para flexibilização da adição de novas tecnologias na arquitetura da Internet, extremamente necessária em um cenário no qual a introdução de novas tecnologias radicalmente diferentes se tornou praticamente impossível

devido à necessidade de consenso entre múltiplos *stakeholders* com objetivos conflitantes. Esse conceito propõe permitir a coexistência de múltiplas arquiteturas de rede heterogêneas sobre uma infraestrutura física compartilhada, promovendo assim a inovação e o desenvolvimento de aplicações diversificadas (Chowdhury; Boutaba, 2010).

Uma rede virtual é essencialmente uma coleção de nós e links virtuais, representando um subconjunto dos recursos da rede física subjacente. A ideia fundamental é separar as responsabilidades dos tradicionais Provedores de Serviço de Internet (ISPs) em duas novas entidades: os Provedores de Infraestrutura (InPs), que gerenciam os recursos físicos, e os Provedores de Serviço (SPs), que criam redes virtuais ao agregar recursos de múltiplos InPs para oferecer serviços de ponta a ponta (Chowdhury; Boutaba, 2010). Essa separação cria um ambiente dinâmico que facilita a implantação de arquiteturas heterogêneas coexistentes, superando as limitações da Internet atual. A virtualização visa atingir objetivos de *design* como flexibilidade, heterogeneidade, gerenciabilidade, isolamento e programabilidade.

2.6.1 Virtualização baseada em Containers Linux

Enquanto a virtualização tradicional, por meio de Máquinas Virtuais (VMs), emula uma plataforma de *hardware* completa, exigindo um sistema operacional convidado (*GuestOS*) completo e um *software* de monitoramento (VMM ou *hypervisor*), a virtualização em nível de sistema operacional, ou baseada em contêineres, adota uma abordagem mais leve. Nesta técnica, o *kernel* do sistema operacional hospedeiro é modificado para suportar múltiplas instâncias isoladas do espaço de usuário (os contêineres). Como os programas dentro dos contêineres utilizam diretamente a API do sistema hospedeiro, o *overhead* de virtualização é mínimo, uma vez que não há necessidade de emulação de *hardware*. Essa eficiência torna os contêineres, como o LXC (Linux contêineres), uma tecnologia chave para plataformas de emulação, permitindo que múltiplos "*netboxes*" (redes virtuais encapsuladas) executem de forma concorrente e autônoma em um único PC, com recursos computacionais privados e isolados.

Para construir redes virtuais, a abstração das interfaces de rede é fundamental. Em um ambiente de contêineres, isso é alcançado pela criação de interfaces de rede virtuais (como pares *Virtual Ethernet* ou VETH) que operam como túneis. O núcleo de uma "*netbox*" é um motor de simulação (como o NS-3) executando dentro de um contêiner, onde os nós da rede simulada são equipados com interfaces do tipo *Ethernet*. Essas interfaces são então ligadas às interfaces virtuais do contêiner. Os contêineres podem ser interligados por meio de *bridges* de *software* que rodam no sistema hospedeiro ou conectados diretamente a placas de rede físicas (NICs), permitindo a comunicação com entidades externas ou outras *netboxes*. Essa arquitetura permite que a cascata de múltiplas *netboxes* sintetize as diferentes seções de uma rede heterogênea complexa, com cada *netbox* emulando as características de uma rede específica de forma exclusiva e isolada.

2.7 TRABALHOS RELACIONADOS

O sistema NGINX é amplamente utilizado como servidor *Web*, *proxy* reverso e balanceador de carga para lidar com aplicações de alta concorrência. Seus algoritmos nativos de balanceamento, como *Weighted Round-Robin*, *Least Connections* e *IP hash*, são as estratégias mais comuns. A abordagem *IP hash*, por exemplo, foi projetada para direcionar requisições de um mesmo cliente para o mesmo servidor, mantendo a continuidade da sessão. Contudo, a função de *hash* nativa é considerada muito simples, o que leva a uma alta probabilidade de "colisões de *hash*". Essas colisões resultam em uma distribuição de carga não uniforme, fazendo com que alguns servidores fiquem sobrecarregados enquanto outros permanecem subutilizados (Ma; Chi, 2022). Essa limitação evidencia que, embora os balanceadores de carga baseados no NGINX sejam eficientes em distribuir o tráfego de modo geral, não há uma estratégia que monitore o uso de memória em tempo real para ajudar na tomada de decisões dinâmicas, sobretudo no contexto de distribuição de conteúdo, cenário no qual a recuperação de arquivos em memória é predominante. Isso pode resultar em sobrecarga nos servidores de *cache*, especialmente em cenários de alto tráfego, onde a memória é um recurso crítico. Portanto, a adição de uma camada de monitoramento de memória pode aprimorar a inteligência do balanceador de carga, permitindo uma alocação de tráfego que considere o estado real dos recursos do servidor, em vez de depender apenas de heurísticas de distribuição.

O *HAProxy* (HAProxy, 2025) é um *software* de código aberto que fornece balanceamento de carga e *proxy* reverso para aplicações baseadas em TCP/IP e HTTP. Embora seja reconhecido por sua eficiência e baixo uso de memória, não há indicações de que utilize monitoramento de memória em tempo real para otimizar a distribuição de tráfego. Integrar tal funcionalidade poderia aprimorar a alocação de recursos e prevenir sobrecargas nos servidores de *cache*.

No contexto da poderosa e moderna prática de *containerização* de servidores, o *Docker* é uma plataforma de código aberto disponibilizada especificamente para esse propósito. O *Docker Swarm*, mecanismo interno da plataforma, realiza a orquestração sistemática de *contêineres* em sistemas complexos e computacionalmente descentralizados. Embora a ferramenta possua um mecanismo interno de balanceamento de carga, o roteamento é feito utilizando técnicas simples como *Round-Robin* e distribuição aleatória para realizar o roteamento distribuído de requisições para os servidores em *contêineres*. Em 2018, foi realizado um experimento utilizando essas ferramentas de modo a introduzir a influência do consumo de memória nos servidores e, consequentemente, aumentar a eficiência do balanceador de carga (Bella; Data; Yahya, 2018). No entanto, a pesquisa não só teve foco exclusivamente voltado às especificidades da ferramenta *Docker*, mas também não envolveu nenhuma análise no contexto de servidores de distribuição de conteúdo.

Também foram elaborados vários algoritmos que tornam o balanceamento de carga dinâmico (*Dynamic Load Balancing*). Um deles (Adewojo; Bass, 2023) teve foco na determinação e uso de métricas cuidadosamente selecionadas para recursos computacionais dos servidores

de destino: Número de *threads*, tamanho do *buffer* de rede, consumo de *CPU*, consumo de memória e largura de banda. O objetivo do trabalho é propor uma nova técnica para a resolução do problema causado pelo fenômeno recorrente na *web* chamado de *Flash Crowds*, caracterizado pelo aumento abrupto de requisições de clientes em serviços *web*. O trabalho trata de aplicações *cloud* no geral, não tendo especificidade voltada a servidores de distribuição de conteúdo.

Uma revisão (Mahmood et al., 2021) foi realizada de modo a obter um retrato do panorama geral das técnicas mais utilizadas para o balanceamento de carga. Após a apresentação dos algoritmos, uma comparação sistemática foi realizada entre eles e um dos critérios de comparação foi a parametrização utilizada por cada algoritmo. Quase todos os algoritmos apresentados pela revisão são dinâmicos e utilizam mais de um parâmetro de configuração. Do total de 15 algoritmos apresentados, 11 usam o tempo de resposta da requisição como parâmetro, 9 utilizam a vazão de dados, 6 utilizam consumo de memória nos servidores, 6 utilizam o consumo de disco, 5 usam o consumo de recursos computacionais em geral na unidade do servidor, 4 usam especificamente o consumo de *CPU*, 4 usam como parâmetro a duração média das requisições (que se diferencia do tempo de resposta, visto que este é monitorado individualmente por requisições de teste que não refletem a carga verdadeira), 2 usam técnicas simples como *Round-Robin* e distribuição aleatória, 1 usa a taxa de falha de requisições e 1 usa o número de conexões simultaneamente ativas. Dessa forma, percebe-se que, na revisão, não há nenhum algoritmo que possivelmente fundamentaria a criação de uma arquitetura de balanceamento de carga especificamente voltada para servidores de distribuição de conteúdo, tendo como base exclusivamente o monitoramento de memória em servidores.

No trabalho "*Dynamic Load Balancing for Efficient Video Streaming Service*" (Kim; Won, 2015), é abordado o desequilíbrio de carga em servidores de *streaming* causado pela concentração de requisições em conteúdos populares. A estratégia proposta é um sistema de gerenciamento de conteúdo dinâmico, denominado iSCON, que monitora os servidores em tempo real e redistribui a carga por meio da replicação de conteúdos populares em servidores com menor tráfego. A abordagem dos autores se concentra exclusivamente na largura de banda de transmissão como o principal indicador de carga; o algoritmo de balanceamento é acionado quando a largura de banda de um servidor excede um limiar predefinido. Embora as especificações dos servidores, incluindo a memória *RAM*, sejam detalhadas, o monitoramento do uso de memória não é considerado um parâmetro para as decisões de balanceamento de carga. O foco permanece na equalização do tráfego de rede, deixando uma lacuna na otimização de cenários onde a pressão sobre a memória, e não apenas a saturação da largura de banda, pode se tornar o gargalo para o desempenho do sistema de distribuição de conteúdo.

No trabalho "*Data Allocation and Dynamic Load Balancing for Distributed Video Storage Server*", (Tsao et al., 1999) propõem uma solução de duas fases para aumentar a disponibilidade em um *cluster* de servidores de vídeo distribuídos. A primeira fase consiste em um esquema de alocação inicial de dados que distribui réplicas de vídeo pelos servidores com base em probabilidades de acesso esperadas, a fim de alcançar um balanceamento de carga estático. A segunda fase

é uma estratégia de balanceamento de carga dinâmica chamada "*load shifting*" (deslocamento de carga), na qual uma requisição bloqueada em um servidor sobrecarregado pode ser admitida ao se migrar uma ou mais requisições já em andamento daquele servidor para outros nós no *cluster*, potencialmente em uma reação em cadeia. O acionamento deste mecanismo dinâmico é baseado em limiares predefinidos que comparam a carga de um servidor, medida pelo número de *streams* ativas ($P_{i,j}$), com a carga de seus vizinhos ou com sua capacidade máxima de serviço (A_{max}). O conceito de "carga" é, portanto, uma função do número de *streams* ativas em relação à capacidade máxima do servidor, sem considerar o estado de recursos de mais baixo nível, como o uso de memória ou *CPU*. Esta abstração é eficaz para balancear o número de requisições, mas não endereça potenciais gargalos de desempenho que podem surgir do esgotamento da memória do sistema, especialmente em servidores que realizam *cache* intensivo de dados de vídeo, um problema que o presente estudo visa investigar.

2.8 CONSIDERAÇÕES

Este capítulo apresentou os fundamentos teóricos essenciais para o desenvolvimento da proposta deste trabalho. A exploração dos balanceadores de carga, tanto em nível de aplicação quanto em nível de rede, estabeleceu a base conceitual necessária para compreender as diferentes abordagens arquiteturais disponíveis e justificar a escolha pela implementação em nível de aplicação.

O estudo do padrão MPEG-DASH e do funcionamento das CDNs forneceu o contexto de aplicação específico, evidenciando que servidores de distribuição de conteúdo dependem intensivamente de operações de memória, tanto principal quanto secundária. Esta característica operacional fundamenta a escolha do monitoramento de memória como métrica central para o algoritmo de balanceamento proposto.

A revisão das técnicas de monitoramento de memória e das estratégias para interpretação de dados desatualizados forneceu os mecanismos técnicos e teóricos necessários para a implementação prática. Especificamente, o trabalho "*Interpreting Stale Load Information*" oferece a base algorítmica para lidar com a defasagem temporal inerente ao processo de monitoramento distribuído.

A análise dos trabalhos relacionados demonstrou que, embora existam diversas abordagens de balanceamento de carga dinâmica, não há trabalhos que se concentrem exclusivamente no monitoramento de memória para balanceamento em servidores de distribuição de conteúdo, estabelecendo assim a originalidade e relevância da proposta apresentada.

Por fim, a exploração da virtualização de redes, especialmente baseada em contêineres, justificou as escolhas arquiteturais para a implementação experimental, permitindo a realização de experimentos controlados em um ambiente isolado.

3 PROPOSTA

Esta seção aborda pormenorizadamente os caminhos escolhidos para a implementação do modelo de balanceamento de carga, bem como os métodos que serão utilizados para realizar a avaliação dos resultados finais.

3.1 O ALGORITMO

O primeiro passo, crucial para a determinação do modelo de otimização de balanceamento de carga, é a definição do algoritmo de balanceamento. Basicamente, a tarefa principal de um algoritmo de balanceamento de carga é o modo como a escolha do servidor será feita a cada requisição que chega ao sistema balanceador de carga.

Em particular, o algoritmo escolhido será **probabilístico**. Isso significa que, para cada servidor que poderá ser escolhido no processo, haverá uma probabilidade intrínseca baseada em parâmetros tanto individuais quanto gerais, e a estratégia de determinação dessa probabilidade é o que definirá o algoritmo em si. De acordo com o objetivo principal do trabalho, as duas principais (mas não únicas) métricas que exercerão influência sobre a determinação da probabilidade de escolha de cada servidor são o **consumo de memória RAM** e a **Taxa de Leitura de Disco**. Essas duas métricas são essenciais dentro do contexto de servidores que distribuem conteúdo, uma vez que sua única tarefa é recuperar estes arquivos do disco e efetivamente entregá-los ao solicitante.

Em paralelo, os servidores enviarão, periodicamente, os dados de monitoramento referentes ao consumo de memória principal e à taxa de leitura de disco. A ideia é que o balanceador mantenha uma estrutura dinâmica que será atualizada de forma contínua, atualizando os parâmetros necessários e efetivamente promovendo eficiência na distribuição das requisições. O método que será usado para realizar a implementação deste conceito, bem como a definição das taxas de atualização dos dados dos servidores para o balanceador de carga, serão discutidos posteriormente.

A Equação 1 utilizada no algoritmo corresponde a uma versão adaptada do *Basic LI* (Dahlin, 2000) de modo que, dada uma requisição, a determinação da probabilidade de escolha de um servidor P_i será determinada de acordo com seus parâmetros, onde:

n : Número total de servidores.

L_i : Carga reportada no servidor i (cálculo determinado na Equação 3).

L_{tot} : Soma de todas as cargas reportadas ($\sum L_i$).

$arrive_T$: Informação de carga total esperada para o tempo de vida médio das informações de carga dos servidores (cálculo determinado na Equação 2).

P_i : Probabilidade de que uma requisição que chega seja enviada para o servidor i .

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

Nota-se que $arrive_T$ é um parâmetro central deste processo. Ele cresce proporcionalmente ao tempo médio de desatualização das informações e, de acordo com a equação anterior (1), um valor grande de $arrive_T$ fará com que a distribuição de probabilidade seja uniforme, efetivamente diminuindo a informação de carga. Esse comportamento é esperado e otimizado porque, quanto mais antigas as informações de carga são, menos o algoritmo as considera no balanceamento, que apresentará comportamento similar à seleção aleatória (sem pesos). Todos os parâmetros envolvidos são mantidos e atualizados iterativamente no estado interno do balanceador de carga e são atualizados mediante à chegada de novas informações de carga. O cálculo de $arrive_T$ se dá da seguinte forma:

$arrive_T$: Informação de carga total esperada para o tempo de vida médio das informações de carga dos servidores;

R_{total} : Número total de requisições recebidas desde o início do processo da aplicação do balanceador de carga;

$T_{elapsed}$: Tempo, em segundos, desde o início do processo da aplicação do balanceador de carga;

T_{stale} : Tempo médio de desatualização de dados de carga de todos os servidores;

L_{tot} : Soma de todas as cargas reportadas;

R_{active} : Número total de requisições ativas em todos os servidores, calculado como $R_{active} = \sum_{i=1}^n AR_i$, onde AR_i representa o número de requisições ativas no servidor i no momento da coleta da métrica;

C : Coeficiente de suavização para relativização de desatualização alta e baixa (definido heurísticamente mediante experimentação);

$$arrive_T = \left(\frac{R_{total}}{T_{elapsed}} \right) \cdot T_{stale} \cdot \left(\frac{L_{tot}}{R_{active}} \right) \cdot C \quad (2)$$

Note-se que no contexto do trabalho (Dahlin, 2000), "carga" é um termo que remete ao tamanho da fila de requisições em estado de processamento pendente. Para fins de implementação e aderência com o problema proposto por este trabalho, o parâmetro de carga (L_i e consequentemente L_{tot}), aplicado na Equação 1, será calculado previamente através da coleta periódica das métricas de consumo de memória RAM e taxa de leitura de disco (cujo processo será descrito posteriormente), da normalização desses parâmetros em pontos percentuais e, finalmente, da mesclagem desses parâmetros para que possam ser inseridos na fórmula. Nesse sentido, a carga de um servidor (L_i) será determinada considerando:

n : Número do total de servidores.

LM_i : Consumo de memória principal no servidor i , representando a quantidade de memória usada para transferir o conteúdo no tempo T , normalizado em pontos percentuais com relação à capacidade total de memória do servidor i .

LD_i : Carga do disco no servidor i , representando a velocidade do fluxo de busca e leitura das informações do disco no tempo T , normalizada em pontos percentuais com relação a capacidade total de leitura do disco no servidor i .

TM : Taxa de uso geral de memória, determinado por $\sum_{i=0}^n \frac{LM_i}{n}$.

TD : Taxa de consumo geral de leitura de disco, determinado por $\sum_{i=0}^n \frac{LD_i}{n}$.

Sendo assim, a Equação 3 define a carga, no contexto deste trabalho, de um servidor de conteúdo.

$$L_i = CM * LM_i + CD * LD_i \quad (3)$$

Nesse contexto, CM e CD são coeficientes que multiplicam as suas respectivas cargas normalizadas. Eles são definidos de acordo com as Equações 4 e 5, respectivamente, e representam a fração normalizada de memória e disco totais, respectivamente, em relação ao total conjugado de memória e disco.

$$CM = \frac{TM}{TM + TD} \quad (4)$$

$$CD = \frac{TD}{TM + TD} \quad (5)$$

Os coeficientes CM e CD são valores entre 0 e 1 de modo que $CM + CD = 1$. Eles são determinados dessa forma para que o recurso computacional que mais esteja sendo utilizado de modo geral em todos os servidores efetivamente contribua mais para o valor da carga de cada servidor L_i , de forma diretamente proporcional à taxa de consumo de cada um dos parâmetros. Finalmente, para que o sistema efetivamente esteja apto para calcular o valor de cada servidor (L_i), basta que ele obtenha os valores de LM_i e LD_i de todos os servidores. Com eles, será possível calcular primeiramente TM e TD , e, com esses valores, calcular CM e CD dinamicamente.

O funcionamento do algoritmo ocorre em dois momentos distintos e independentes. O primeiro momento corresponde à **atualização da distribuição de probabilidade**, que é acionada sempre que uma nova informação de carga chega ao balanceador via MQTT. Neste processo,

o balanceador atualiza as métricas do servidor que enviou a informação (LM_i, LD_i, T_i, AR_i) e, em seguida, recalcula todos os parâmetros globais necessários: as médias de uso de memória (TM) e disco (TD), o tempo médio de desatualização (T), os coeficientes de ponderação (CM e CD), a carga de cada servidor (L_i), a carga total (L_{tot}), o parâmetro de chegadas esperadas ($arrive_T$) e, finalmente, a probabilidade de seleção de cada servidor (P_i). É importante notar que, mesmo quando apenas um servidor envia novas métricas, todos os parâmetros são recalculados considerando todos os servidores conhecidos, garantindo que a distribuição de probabilidade seja sempre consistente e atualizada.

O segundo momento corresponde à **seleção do servidor**, que ocorre a cada requisição que chega ao balanceador. Neste processo, o balanceador simplesmente utiliza a distribuição de probabilidade mais recente (calculada no primeiro momento) para realizar uma seleção aleatória com pesos. Especificamente, o algoritmo incrementa o contador total de requisições (R), obtém um snapshot das probabilidades P_i já calculadas e, então, realiza uma seleção aleatória ponderada, onde cada servidor tem uma chance de ser escolhido proporcional à sua probabilidade P_i . Caso nenhuma distribuição de probabilidade tenha sido calculada ainda (situação inicial antes da chegada da primeira métrica), o algoritmo utiliza uma distribuição uniforme, onde todos os servidores têm igual probabilidade de serem escolhidos.

Algoritmo 1 Atualização da distribuição de probabilidade

Gatilho: Chegada de uma nova informação de carga via MQTT.

Requer:

Para cada servidor S_i , suas métricas: LM_i, LD_i, T_i, AR_i .

Garante:

Distribuição de probabilidade P_i atualizada para todos os servidores.

- 1: **Procedimento** ATUALIZARDISTRIBUICAO(LM_i, LD_i, T_i, AR_i)
 - 2: Atualizar métricas do servidor S_i
 - 3: Calcular agregados globais:
 - 4: $TM \leftarrow \frac{1}{n} \sum_{i=1}^n LM_i, TD \leftarrow \frac{1}{n} \sum_{i=1}^n LD_i$
 - 5: $T \leftarrow \frac{1}{n} \sum_{i=1}^n (T_{atual} - T_i)$ // Tempo médio de desatualização
 - 6: Calcular coeficientes CM e CD conforme Equações 4 e 5
 - 7: Calcular carga L_i para cada servidor conforme Equação 3
 - 8: $L_{tot} \leftarrow \sum_{i=1}^n L_i$
 - 9: Calcular $arrive_T$ conforme Equação 2
 - 10: Calcular probabilidade P_i para cada servidor conforme Equação 1
 - 11: **if** nenhum servidor disponível **ou** $arrive_T \leq 0$ **then**
 - 12: $P_i \leftarrow 1/n$ para todos os servidores // Distribuição uniforme
 - 13: **Fim if**
 - 14: **Fim Procedimento**
-

Para o funcionamento do algoritmo, o balanceador de carga mantém em memória um conjunto de estruturas de dados que armazenam o estado atual do sistema. Essas estruturas incluem: as métricas individuais de cada servidor (LM_i, LD_i, T_i, AR_i), as cargas calculadas (L_i), as probabilidades de seleção (P_i), além de variáveis agregadas como L_{tot} e n . Todas essas

informações são dinamicamente atualizadas sempre que uma nova métrica chega via MQTT, e o número de servidores (n) é automaticamente ajustado quando novos servidores são detectados através da identificação única presente nas mensagens MQTT. A detecção de novos servidores ocorre de forma automática: quando uma métrica é recebida de um servidor ainda não conhecido, ele é automaticamente adicionado ao conjunto de servidores monitorados.

3.2 OS REQUISITOS TECNOLÓGICOS

A infraestrutura experimental para este trabalho foi implementada em um ambiente virtualizado, contido integralmente em uma única máquina física. A escolha por esta abordagem, suportada pela tecnologia de *contêineres Docker*, é estratégica e visa mitigar a influência de variáveis de rede externas, como latência e instabilidade de conexão. Ao confinar toda a comunicação a uma rede virtual interna gerenciada pelo *Docker*, o foco da análise é direcionado exclusivamente ao impacto do monitoramento de recursos, como a memória, no desempenho e na otimização do balanceador de carga, que é o objetivo central deste trabalho. Adicionalmente, a utilização do *Docker* como plataforma padrão para todos os componentes do sistema garante a padronização do ambiente de execução, eliminando problemas de dependência de sistemas operacionais e bibliotecas, o que assegura a consistência e a reprodutibilidade dos experimentos.

Neste ecossistema controlado, quatro aplicações principais interagem para formar o sistema em estudo: As três principais são o servidor de conteúdo, que será replicado de modo redundante para que o cenário emule simplificadaamente uma CDN, o cliente e o balanceador de carga. Em conjunto, ainda haverá mais um serviço auxiliar que fará interações diretamente com os servidores e com o balanceador de carga, sendo ele o *broker* de mensagens MQTT. O servidor de conteúdo será responsável por fornecer segmentos de vídeo de acordo com o padrão MPEG-DASH. O balanceador de carga atuará como um intermediário, recebendo as requisições dos clientes e distribuindo-as de forma inteligente, baseado na avaliação de carga, entre os servidores disponíveis. A aplicação cliente iniciará várias requisições de modo paralelo, direcionando-as ao balanceador de carga (solicitando segmentos subsequentes conforme o MPEG-DASH) e, subsequentemente, consumindo o conteúdo de mídia retornado, fechando assim o ciclo de interação do sistema. Em paralelo, os próprios servidores, periodicamente, fornecerão seus dados de carga de memória principal e secundária publicando-os em tópicos específicos do *broker* por meio do modelo de comunicação indireta *publish-subscribe* (Eugster et al., 2003). O balanceador de carga, que estará inscrito como assinante desses tópicos, eventualmente consumirá esses dados de modo a realimentar os parâmetros internos utilizados para os cálculos envolvidos (ver Equações 1 e 3).

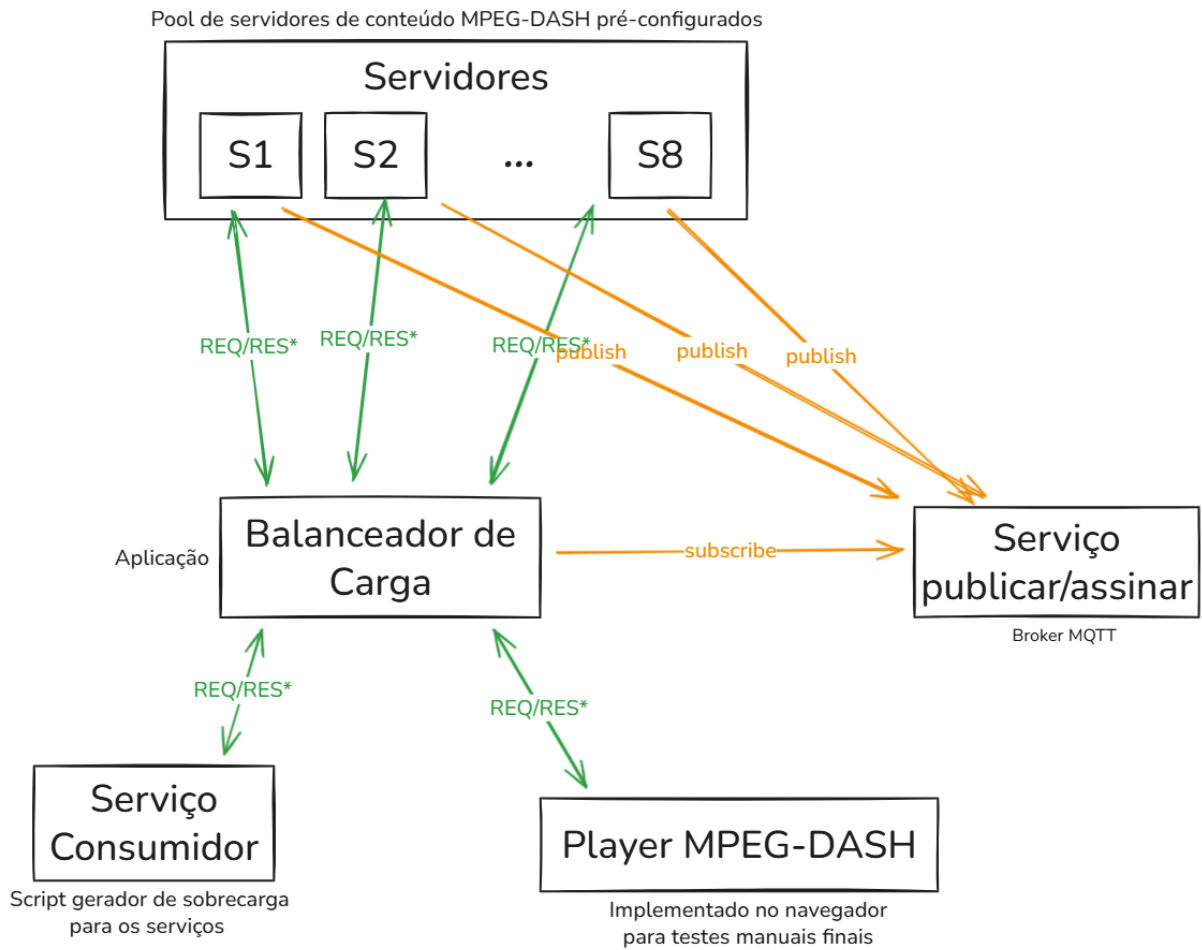
A arquitetura do sistema foi dimensionada para simular um cenário de alta demanda e é composta por uma variedade de serviços. O conjunto de recursos consiste em 8 *containers* atuando como servidores MPEG-DASH, constituindo o *pool* de nós para os quais a carga será distribuída. Há um contêiner dedicado para atuar como *broker* MQTT, com o propósito de

mediar a comunicação publicar-assinar para a atualização dos parâmetros do balanceador de carga. Também haverá um serviço de monitoramento com acesso direto tanto aos servidores quanto ao *broker* MQTT sob a finalidade de auxiliar no processo de desenvolvimento e depuração, fornecendo observabilidade. O sistema contará com um único balanceador de carga, que representa a peça central da investigação. Para gerar uma carga de trabalho realista e concorrente, será implementado um *script* cliente simples, projetado para gerar múltiplas requisições paralelas, simulando o comportamento de um grande número de usuários acessando o serviço simultaneamente por meio de um *player* MPEG-DASH. Finalmente, um simples *player* de vídeo baseado em navegador será utilizado como ferramenta de validação, cujo propósito é permitir uma avaliação qualitativa da experiência do usuário (QoE) ao reproduzir o vídeo disposto pelos servidores, mesmo sob a sobrecarga imposta pela aplicação cliente de teste.

A seleção de tecnologias para cada componente foi alinhada aos objetivos do projeto. Duas imagens *Docker* pré-existentes serão configuradas e usadas para criar os servidores MPEG-DASH e o *broker* MQTT (*Message Queuing Telemetry Transport*), respectivamente. Uma aplicação será desenvolvida e encapsulada em um contêiner na forma de um *script Python*, de modo que será responsável por simular sobrecarga nos serviços e no balanceador de carga. Por fim, o balanceador de carga, componente central desta pesquisa, será desenvolvido com *design* modular e aberto, permitindo a fácil substituição e modificação da estratégia de balanceamento, o que viabilizará futuras comparações de desempenho com algoritmos generalistas, como *Round-Robin* e Seleção Aleatória.

De modo abstrato, a Figura 3 apresenta o panorama geral da arquitetura final do sistema criado de acordo com a Figura 3.

Figura 3 – Arquitetura geral do sistema.



* Requisição e Resposta de conteúdo em áudio/vídeo

Fonte: Elaborado pelo autor (2025)

3.3 CONSIDERAÇÕES

Este capítulo apresentou a proposta metodológica para a implementação de um sistema de balanceamento de carga baseado em monitoramento de recursos computacionais, especificamente memória RAM e taxa de leitura de disco. O algoritmo proposto, fundamentado no trabalho de (Dahlin, 2000), utiliza uma abordagem probabilística que adapta dinamicamente a distribuição de requisições entre servidores conforme o estado atual de carga de cada nó.

A escolha por métricas de memória e disco justifica-se pelo contexto de aplicação: servidores de distribuição de conteúdo que recuperam arquivos do disco e os transferem para clientes, sendo esses recursos críticos para o desempenho do sistema. A implementação do algoritmo em dois momentos distintos — atualização da distribuição de probabilidade e seleção de servidor — permite que o balanceador mantenha informações atualizadas sobre o estado do sistema sem impactar significativamente a latência de processamento de requisições.

A arquitetura proposta, baseada em contêineres *Docker* e comunicação via MQTT,

oferece um ambiente controlado e reproduzível para a avaliação do algoritmo, isolando variáveis externas e permitindo que o foco da análise seja direcionado exclusivamente ao impacto do monitoramento de recursos no balanceamento de carga. A modularidade do sistema, com estratégias de balanceamento intercambiáveis, viabiliza comparações futuras com algoritmos generalistas e permite a extensão da pesquisa para outros contextos de aplicação.

Os próximos capítulos detalharão a implementação prática desta proposta, os experimentos realizados e os resultados obtidos, fornecendo evidências empíricas sobre a eficácia da abordagem proposta e seu impacto na qualidade de experiência dos usuários finais.

4 IMPLEMENTAÇÃO

Este capítulo apresenta uma descrição detalhada do ambiente experimental utilizado para validação da proposta, incluindo a configuração do ambiente de testes, a definição dos módulos principais e auxiliares do sistema, os detalhes de implementação de cada componente, o processo de geração de conteúdo multimídia e, por fim, a especificação dos cenários de teste utilizados para a análise comparativa dos algoritmos de balanceamento de carga.

4.1 AMBIENTE DE EXPERIMENTAÇÃO

O ambiente experimental foi configurado para proporcionar um cenário controlado e reproduzível, capaz de simular condições realistas de operação de um sistema de distribuição de conteúdo. A infraestrutura foi estabelecida em uma única máquina física operando o sistema operacional *Windows*, com o subsistema *Windows* para Linux (WSL2) configurado para execução de um *kernel* Linux completo. Esta escolha arquitetural permite a utilização de recursos nativos do Linux, como o sistema de grupos de controle (*cgroups*), essenciais para o monitoramento e limitação de recursos computacionais dos *contêineres*.

A plataforma de containerização *Docker* foi empregada como ambiente de execução para todos os componentes do sistema. O *Docker* oferece isolamento de processos, gerenciamento de recursos e padronização do ambiente de execução, garantindo que os experimentos possam ser reproduzidos de forma consistente. Além disso, a utilização de *contêineres* elimina problemas relacionados a dependências de bibliotecas e configurações de sistema operacional, assegurando que cada componente opere em um ambiente idêntico e controlado.

4.1.1 Particularidades do *Docker* e *cgroups*

O *Docker* utiliza o sistema de *cgroups* (grupos de controle) do *kernel Linux* para implementar limites de recursos e realizar o monitoramento de uso de *CPU*, memória e leitura de disco. Especificamente, o experimento utiliza *cgroups v2*, que oferece uma interface unificada e simplificada para o gerenciamento de recursos. Através dos arquivos de sistema disponibilizados pelo *cgroups*, é possível ler diretamente as métricas de consumo de memória e taxa de leitura de disco de cada contêiner em tempo real.

Para garantir a consistência entre as execuções dos experimentos e eliminar o efeito de *cache* do sistema operacional nos resultados, é fundamental realizar a limpeza do *cache* de páginas do *kernel Linux* antes de cada execução. Esta operação é realizada através do comando:

```
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Este comando instrui o *kernel* a liberar os *caches* de página (*pagecache*), *entries* e *inodes*, garantindo que cada execução do experimento parta de um estado limpo, sem dados previamente

armazenados em memória. A operação requer privilégios de superusuário (`sudo`) e é executada antes de cada bateria de testes.

O *Docker* também permite a configuração de limites explícitos de recursos para cada contêiner através de parâmetros na configuração do *Docker Compose*. No experimento, foram configurados limites de memória *RAM* (através do parâmetro `memory`) e limites de taxa de leitura de disco (através do parâmetro `device_read_bps`). Estes limites são fundamentais para simular servidores com capacidades heterogêneas, permitindo avaliar o comportamento do algoritmo de balanceamento em cenários com recursos desiguais.

4.1.2 Organização do conteúdo e bind mounts

Para simular o cenário realista de servidores de distribuição de conteúdo com arquivos exclusivos, foi necessário replicar o conteúdo de vídeo em múltiplos diretórios no sistema de arquivos do *host* WSL. O conteúdo multimídia foi armazenado em um diretório global localizado em `/data` no sistema de arquivos *Linux* do WSL, com subdiretórios específicos para cada servidor (`/data/server-1`, `/data/server-2`, e assim por diante até `/data/server-8`).

Cada contêiner de servidor MPEG-DASH foi configurado com um *bind mount* que mapeia o diretório específico do servidor no *host* para o diretório interno do contêiner. Esta técnica permite que cada servidor acesse apenas seu próprio conjunto de arquivos, simulando a exclusividade de conteúdo encontrada em servidores de borda de uma CDN real. O uso de *bind mounts* também evita a necessidade de copiar os arquivos para dentro das imagens *Docker*, reduzindo o tamanho das imagens e facilitando a atualização do conteúdo sem reconstruir os *contêineres*.

4.2 DEFINIÇÃO DOS MÓDULOS

O sistema experimental é composto por módulos principais e auxiliares, cada um desempenhando uma função específica na arquitetura do experimento. Esta seção descreve a organização e o propósito de cada componente.

4.2.1 Módulos Principais

Os módulos principais formam o núcleo funcional do sistema de distribuição de conteúdo e são responsáveis pelo processamento e encaminhamento das requisições dos clientes:

1. **Servidores MPEG-DASH:** Conjunto de 8 instâncias de servidores *web* responsáveis por servir o conteúdo de vídeo segmentado de acordo com o padrão MPEG-DASH. Cada servidor é executado em um contêiner *Docker* isolado e possui limites específicos de memória e taxa de leitura de disco, permitindo a simulação de servidores com capacidades heterogêneas. Os servidores também são responsáveis por coletar e publicar suas próprias métricas de consumo de recursos.

2. **Balanceador de Carga:** Componente central do experimento, responsável por receber todas as requisições HTTP dos clientes e distribuí-las entre os servidores MPEG-DASH disponíveis de acordo com a estratégia de balanceamento configurada. O balanceador opera como um *proxy* reverso HTTP e é implementado para suportar múltiplas estratégias de seleção de servidor, permitindo a comparação entre diferentes algoritmos.
3. **Gerador de Carga:** Aplicação cliente que simula múltiplos usuários acessando o sistema de forma concorrente. O gerador é parametrizável em termos de número de usuários simultâneos e duração do teste, permitindo a criação de cenários de carga variados para avaliar o comportamento do sistema sob diferentes níveis de estresse.

4.2.2 Módulos Auxiliares

Os módulos auxiliares fornecem serviços de suporte necessários para o funcionamento e monitoramento do sistema:

1. **Broker MQTT (NanoMQ):** Serviço de mensageria (*broker*) que implementa o protocolo MQTT (*Message Queuing Telemetry Transport*), um protocolo leve de comunicação *publish-subscribe*. O *broker* atua como intermediário, recebendo as métricas publicadas pelos servidores MPEG-DASH e disponibilizando-as para o balanceador de carga através de assinaturas em tópicos específicos. O uso do MQTT permite a comunicação assíncrona e desacoplada entre os componentes.
2. **Painel de Monitoramento em Tempo Real:** Interface *web* que oferece visualização gráfica e em tempo real das métricas de cada servidor e do balanceador de carga. O painel permite acompanhar o comportamento do sistema durante os experimentos, facilitando a detecção de anomalias e a compreensão do comportamento dinâmico do algoritmo de balanceamento.
3. **Coletor de Métricas (API):** Serviço Python que expõe uma API REST para consulta das métricas dos *contêineres* através do acesso direto aos arquivos do *cgroups*. Este componente complementa o monitoramento realizado internamente pelos servidores, permitindo a validação cruzada das métricas e fornecendo dados para o painel de monitoramento.

4.3 DETALHES DE IMPLEMENTAÇÃO DOS MÓDULOS

Esta seção apresenta os detalhes técnicos de implementação de cada módulo do sistema, incluindo as tecnologias empregadas, as principais funcionalidades e os trechos de código mais relevantes.

4.3.1 Servidores MPEG-DASH

Os servidores MPEG-DASH foram implementados utilizando *ASP.NET Core* 8.0, uma plataforma de desenvolvimento *web* moderna e de alto desempenho da *Microsoft*. A escolha desta tecnologia foi motivada por sua capacidade nativa de servir arquivos estáticos de forma eficiente, sua arquitetura assíncrona que permite alto nível de concorrência e a disponibilidade de bibliotecas para comunicação MQTT.

Cada servidor é configurado como uma aplicação *web* mínima que serve arquivos estáticos do diretório `wwwroot/Static`, onde estão armazenados os segmentos de vídeo MPEG-DASH. A aplicação não realiza processamento de vídeo em tempo real; ela apenas entrega os arquivos pré-processados aos clientes que os solicitam.

4.3.1.1 Monitoramento e Publicação de Métricas

O componente mais crítico da implementação dos servidores é o sistema de monitoramento de recursos e publicação de métricas. Esta funcionalidade é implementada através da classe `MetricsPublisher`, um serviço de segundo plano (*background service*) que executa continuamente enquanto o servidor está ativo.

O `MetricsPublisher` realiza periodicamente as seguintes operações:

1. Leitura do consumo atual de memória através do arquivo `/sys/fs/cgroup/memory.current` do *cgroups v2*;
2. Obtenção das estatísticas de leitura de memória secundária através do arquivo `/sys/fs/cgroup/io.stat` extraindo o total de *bytes* lidos (`rbytes`);
3. Cálculo da taxa de leitura de disco através da diferença entre leituras consecutivas dividida pelo intervalo de tempo decorrido;
4. Normalização das métricas em relação aos limites configurados para o contêiner, gerando valores entre 0 e 1;
5. Publicação das métricas normalizadas em um tópico MQTT para consumo pelo balanceador de carga.

O intervalo de publicação é configurável através da variável de ambiente `METRICS_PUBLISH_INTERVAL` permitindo avaliar o impacto da frequência de atualização das métricas no desempenho do algoritmo de balanceamento.

A estrutura de dados publicada contém os seguintes campos:

- `memory_current`: Consumo normalizado de memória (0 a 1);
- `disk_read`: Taxa normalizada de leitura de disco (0 a 1);

- `active_request_count`: Número de requisições HTTP ativas sendo processadas no momento;
- `timestamp_unix`: *Timestamp* da coleta.

O contador de requisições ativas é implementado através da classe `RequestCounter`, que utiliza operações atômicas para incrementar e decrementar o contador de forma *thread-safe*, garantindo a consistência dos dados mesmo sob alta concorrência.

4.3.2 Balanceador de Carga

O balanceador de carga foi implementado em *Rust*, uma linguagem de programação de sistemas que oferece garantias de segurança de memória em tempo de compilação e desempenho comparável a linguagens como *C* e *C++*. A escolha do *Rust* foi motivada pela necessidade de alto desempenho e baixa latência no processamento das requisições, características essenciais para um componente crítico como o balanceador de carga.

4.3.2.1 Arquitetura e Funcionamento

O balanceador opera como um *proxy* reverso HTTP, recebendo requisições dos clientes na porta 8080 e encaminhando-as para um dos servidores MPEG-DASH disponíveis. Um aspecto fundamental da implementação é que o balanceamento ocorre ao nível de cada requisição HTTP individual. Isso significa que, durante uma única sessão de reprodução MPEG-DASH, diferentes requisições (como a solicitação do manifesto MPD, dos segmentos de inicialização e dos segmentos de mídia) podem ser atendidas por servidores diferentes, dependendo das decisões tomadas pelo algoritmo de balanceamento a cada momento.

A arquitetura do balanceador é baseada no padrão de projeto *Strategy*, que permite a fácil substituição do algoritmo de seleção de servidor sem modificar o código principal do balanceador. O *trait* (estrutura da linguagem *Rust* que define comportamento que deve ser implementado em algum tipo - similar à interfaces em linguagens tradicionais orientadas a objeto) `ServerSelectionStrategy` define a interface que deve ser implementada por qualquer estratégia de balanceamento:

```
pub trait ServerSelectionStrategy: Send + Sync {
    fn pick_server(&self, servers: &[String]) -> Option<String>;
}
```

Esta abstração permite que o experimento compare diferentes algoritmos de balanceamento utilizando exatamente a mesma infraestrutura, garantindo que as diferenças observadas nos resultados sejam devidas exclusivamente ao algoritmo e não a variações na implementação.

4.3.2.2 Estratégias de Balanceamento Implementadas

Três estratégias de balanceamento foram implementadas para fins de comparação:

1. **Round-Robin:** Estratégia determinística que seleciona os servidores de forma cíclica, garantindo uma distribuição uniforme das requisições ao longo do tempo. A implementação utiliza um contador atômico que é incrementado a cada seleção.
2. **Seleção Aleatória:** Estratégia não-determinística que seleciona um servidor de forma uniformemente aleatória entre os disponíveis. Esta estratégia serve como linha de base para avaliar o ganho proporcionado por estratégias mais sofisticadas.
3. **Monitoramento de Memória:** Estratégia proposta neste trabalho, que utiliza as métricas de consumo de memória e disco publicadas pelos servidores para calcular probabilidades de seleção baseadas na carga atual e na interpretação de dados desatualizados, conforme descrito no Capítulo 3.

A estratégia de monitoramento de memória mantém uma estrutura de dados interna com as métricas mais recentes de cada servidor, que é atualizada através de uma assinatura MQTT no tópico `loadbalancer/metrics`. Quando uma nova requisição chega, o algoritmo calcula as probabilidades de seleção de acordo com as Equações 1 e 3, e realiza uma seleção aleatória ponderada baseada nessas probabilidades.

4.3.3 Gerador de Carga

O gerador de carga é uma aplicação *Python* que utiliza a biblioteca `aiohttp` para realizar requisições HTTP assíncronas de alto desempenho. A implementação é baseada em um modelo de concorrência com múltiplas *coroutines* assíncronas (*workers*), onde cada *worker* simula o comportamento de um cliente MPEG-DASH real.

4.3.3.1 Simulação de Clientes MPEG-DASH

Cada *worker* do gerador de carga executa o seguinte fluxo de operações, que emula o comportamento de um *player* de vídeo MPEG-DASH:

1. **Seleção de Conteúdo:** O *worker* seleciona aleatoriamente um dos diretórios de vídeo disponíveis (de `video-1` até `video-4`, de acordo com a configuração).
2. **Requisição do Manifesto MPD:** A primeira requisição HTTP é feita para obter o arquivo MPD (*Media Presentation Description*), que é um documento XML contendo metadados sobre o vídeo, incluindo as resoluções disponíveis, os *bitrates*, a duração dos segmentos e os *templates* de URL para acessar cada segmento.

3. **Parsing do Manifesto:** O arquivo MPD é analisado utilizando a biblioteca padrão `xml.etree.Element` extraíndo as informações sobre os segmentos de inicialização e os segmentos de mídia de todas as representações (resoluções) disponíveis.
4. **Requisição dos Segmentos de Inicialização:** Para cada representação (resolução) do vídeo, o *worker* solicita o segmento de inicialização correspondente, que contém informações de *codec* e configuração necessárias para decodificar os segmentos subsequentes.
5. **Requisição Sequencial dos Segmentos de Mídia:** O *worker* então solicita sequencialmente os segmentos de mídia, aguardando a conclusão de cada requisição HTTP antes de iniciar a próxima. Este comportamento, sem *buffering* antecipado, simula um cenário onde o cliente consome o conteúdo em tempo real, sem realizar *download* prévio de múltiplos segmentos. Esta é uma simulação simplificada que não representa exatamente o comportamento de *players* MPEG-DASH modernos, que geralmente implementam estratégias de *buffering* e seleção adaptativa de qualidade, mas é adequada para gerar carga no sistema e avaliar o desempenho do balanceador.

O número de *workers* concorrentes é parametrizável através do argumento de linha de comando `-concurrent`, permitindo simular desde cenários de baixa carga (dezenas de usuários) até cenários de alta carga (milhares de usuários simultâneos). A duração do teste também é configurável através do parâmetro `-duration`, permitindo controlar o tempo total de execução do experimento.

4.3.3.2 Aplicação Cliente para Validação e Visualização

Além do gerador de carga automatizado, foi desenvolvida uma aplicação cliente baseada em *HTML* (`test-dash.html`) que utiliza a biblioteca `dash.js` para reprodução real de vídeos MPEG-DASH com visualização em tempo real das métricas de QoE. Esta aplicação serve como ferramenta de validação e depuração, permitindo observar o comportamento do sistema durante a reprodução de vídeo e coletar dados detalhados sobre a qualidade da experiência do usuário.

A aplicação implementa um *player* de vídeo completo com as seguintes funcionalidades:

1. **Gerenciamento de Sessão de Vídeo:** A aplicação inicializa o *player* `dash.js` com configurações específicas de *buffer* (`stableBufferTime`, `initialBufferLevel`) e registra *handlers* para eventos de reprodução (`PLAYBACK_STARTED`, `PLAYBACK_WAITING`, `PLAYBACK_PLAYING`, `PLAYBACK_ENDED`). Esses eventos permitem rastrear o ciclo de vida completo da sessão de reprodução, incluindo o início do *streaming*, momentos de travamento (*stalls*), retomada da reprodução e finalização do vídeo.
2. **Visualização em Tempo Real:** Três gráficos interativos são atualizados dinamicamente durante a reprodução do vídeo, utilizando a biblioteca `Chart.js`:

- **Gráfico de Qualidade (*Quality Timeline*):** Exibe a evolução da qualidade (resolução e *bitrate*) selecionada pelo algoritmo de adaptação do *dash.js* ao longo dos segmentos de vídeo reproduzidos. O eixo Y representa as diferentes resoluções disponíveis (144P a 1080P), e o eixo X representa a sequência de segmentos, permitindo visualizar as mudanças de qualidade durante a sessão.
- **Gráfico de *Bitrate* (*Bitrate Timeline*):** Mostra a taxa de transferência média (*throughput*) medida pelo *dash.js* para cada segmento baixado, em *kilobits* por segundo (kbps). Este gráfico utiliza uma escala logarítmica no eixo Y para melhor visualização da variação de *bitrate* ao longo do tempo.
- **Gráfico de Travamentos (*Stall Timeline*):** Apresenta um gráfico de dispersão onde cada ponto representa um evento de travamento (*stall*), com a posição no eixo X indicando o segmento onde ocorreu o travamento e o eixo Y mostrando a duração do travamento em segundos. Este gráfico permite identificar visualmente a frequência e a severidade dos travamentos durante a sessão.

3. **Coleta de Dados para Análise:** Ao final de cada sessão de reprodução, a aplicação exporta automaticamente para o console do navegador *arrays* estruturados contendo todos os dados coletados:

- Sequência de qualidades reproduzidas (IDs de representação e *labels* descritivos);
- Sequência de *bitrates* medidos para cada segmento;
- Lista detalhada de travamentos com *timestamp*, duração e índice do segmento;
- Estatísticas agregadas: número total de travamentos e tempo total de travamento acumulado.

Estes dados exportados podem ser facilmente copiados do console e processados posteriormente para análise estatística e geração de gráficos comparativos entre diferentes cenários de teste.

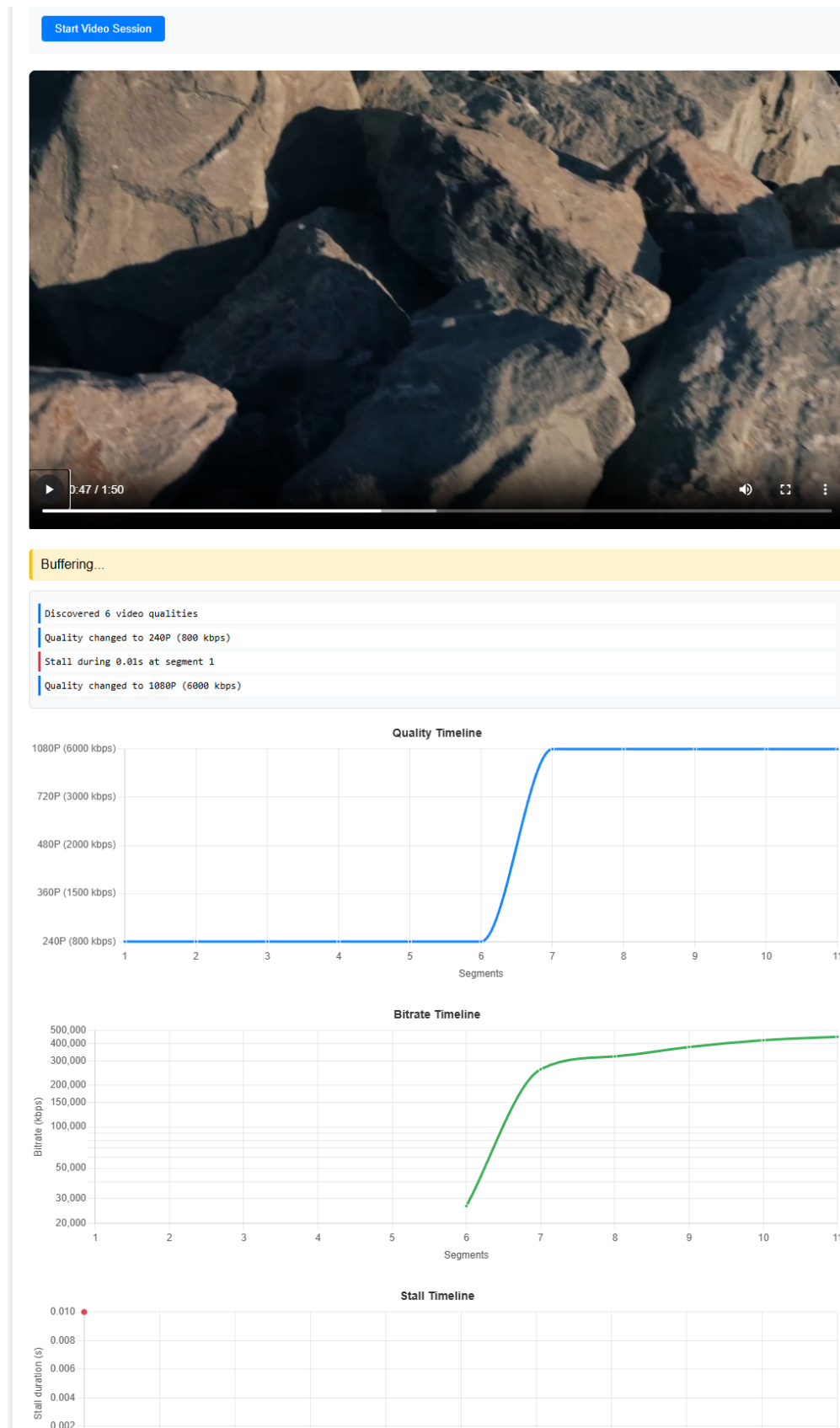
A Figura 4 apresenta a interface da aplicação cliente durante a reprodução de um vídeo, mostrando o *player* de vídeo e os três gráficos de monitoramento em tempo real, que permitem a validação visual do comportamento do sistema e a identificação imediata de problemas de QoE.

4.3.4 *Broker* MQTT (NanoMQ)

O *broker* MQTT utilizado no experimento é o *NanoMQ*, uma implementação leve e de alto desempenho do protocolo MQTT desenvolvida pela *EMQ Technologies*. O *NanoMQ* foi escolhido por sua simplicidade de configuração, baixo consumo de recursos e conformidade completa com o padrão MQTT 5.0.

O *broker* é executado em um contêiner *Docker* dedicado e expõe a porta 1883 para comunicação MQTT não criptografada. A configuração é realizada através de um arquivo

Figura 4 – Aplicação cliente MPEG-DASH com visualização em tempo real de métricas de QoE: gráfico de qualidade, gráfico de *bitrate* e gráfico de travamentos.



Fonte: Elaborado pelo autor (2025)

`nanomq.conf` montado como volume no contêiner, permitindo ajustes finos de parâmetros como o número máximo de conexões simultâneas e o tamanho dos *buffers* de mensagens.

No contexto do experimento, o *broker* atua como canal de comunicação entre os servidores MPEG-DASH (publicadores) e o balanceador de carga (assinante), permitindo que as métricas sejam transmitidas de forma assíncrona e desacoplada, sem que os servidores precisem conhecer o endereço do balanceador.

4.3.5 Orquestração com *Docker Compose*

A orquestração de todos os componentes do sistema é realizada através do *Docker Compose*, uma ferramenta para definir e executar aplicações *Docker* multi-contêiner. O arquivo `docker-compose.yml` descreve de forma declarativa todos os serviços, suas configurações, limites de recursos, volumes e dependências.

Cada servidor MPEG-DASH é definido como um serviço separado com suas próprias configurações de recursos. Por exemplo, o servidor 1 é configurado com 4096 MB de memória e taxa máxima de leitura de disco de 512 MB/s, enquanto o servidor 8 possui apenas 512 MB de memória e 4 MB/s de leitura de disco. Esta heterogeneidade é fundamental para avaliar a capacidade do algoritmo de balanceamento em distribuir a carga de forma proporcional à capacidade de cada servidor.

Os comandos principais para gerenciar o ambiente experimental são:

```
# Limpar \textit{cache} e iniciar todos os serviços
echo 3 | sudo tee /proc/sys/vm/drop_caches && \
docker compose down && \
docker compose up -d --build

# Visualizar logs do balanceador
docker compose logs -f load-balancer

# Encerrar todos os serviços
docker compose down
```

4.3.6 Painel de Monitoramento

O painel de monitoramento foi implementado como uma página *HTML* estática que utiliza a biblioteca *Chart.js* para renderizar gráficos em tempo real. A página é servida através de um contêiner *nginx* e se comunica com a API de métricas através de requisições HTTP periódicas utilizando *JavaScript*.

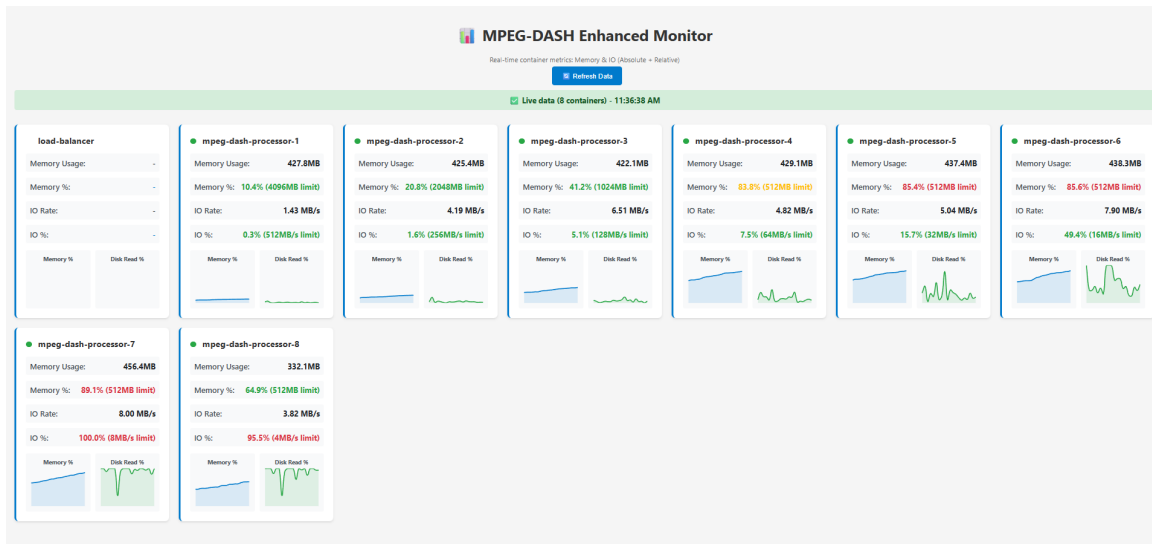
O painel exibe, para cada servidor e para o balanceador de carga:

- Consumo de memória (absoluto e percentual);

- Taxa de leitura de disco (MB/s e percentual);
- Número de requisições ativas;
- Gráficos de séries temporais mostrando a evolução das métricas.

O painel é acessível através de um navegador *web* na porta 3001 do *host* e foi utilizado durante o desenvolvimento e depuração do sistema para validar o comportamento dos componentes e identificar problemas de configuração. A Figura 5 apresenta a interface do painel de monitoramento, mostrando as métricas de múltiplos servidores MPEG-DASH em tempo real.

Figura 5 – Painel de monitoramento em tempo real mostrando métricas de consumo de memória, taxa de leitura de disco e requisições ativas dos servidores MPEG-DASH.



Fonte: Elaborado pelo autor (2025)

4.4 GERAÇÃO DE CONTEÚDO

O conteúdo multimídia utilizado nos experimentos foi gerado utilizando o *ffmpeg*, uma ferramenta de código aberto amplamente utilizada para processamento de áudio e vídeo. O *ffmpeg* é capaz de codificar, decodificar, transcodificar, multiplexar e aplicar filtros em arquivos multimídia, sendo a ferramenta padrão da indústria para preparação de conteúdo para *streaming*.

4.4.1 Codificação e Segmentação MPEG-DASH

O processo de geração do conteúdo MPEG-DASH envolveu a conversão de um arquivo de vídeo de entrada para o formato MPEG-DASH com múltiplas representações (resoluções e *bitrates*). O comando utilizado foi:

```
ffmpeg -y -i video.mp4 \
-filter_complex "[0:v]format=yuv420p,setsar=1,split=6[v0][v1][v2][v3][v4][v5]; \
```

```

[v0]scale=1920:1080:flags=lanczos[v1080]; \
[v1]scale=1280:720:flags=lanczos[v720]; \
[v2]scale=854:480:flags=lanczos[v480]; \
[v3]scale=640:360:flags=lanczos[v360]; \
[v4]scale=426:240:flags=lanczos[v240]; \
[v5]scale=256:144:flags=lanczos[v144]" \
-map "[v1080]" -map "[v720]" -map "[v480]" -map "[v360]" \
-map "[v240]" -map "[v144]" -map 0:a:0 \
-c:v:0 libx264 -b:v:0 6000k -maxrate:v:0 6600k -bufsize:v:0 12000k \
-c:v:1 libx264 -b:v:1 3000k -maxrate:v:1 3300k -bufsize:v:1 6000k \
-c:v:2 libx264 -b:v:2 2000k -maxrate:v:2 2200k -bufsize:v:2 4000k \
-c:v:3 libx264 -b:v:3 1500k -maxrate:v:3 1650k -bufsize:v:3 3000k \
-c:v:4 libx264 -b:v:4 800k -maxrate:v:4 880k -bufsize:v:4 1600k \
-c:v:5 libx264 -b:v:5 300k -maxrate:v:5 330k -bufsize:v:5 600k \
-profile:v:0 high -profile:v:1 main -profile:v:2 main \
-profile:v:3 main -profile:v:4 main -profile:v:5 main \
-g 120 -keyint_min 120 -sc_threshold 0 -pix_fmt yuv420p \
-preset veryfast \
-c:a aac -b:a 128k -ar 48000 -ac 2 \
-use_template 1 -use_timeline 1 -seg_duration 4 \
-init_seg_name 'init-stream$RepresentationID$.m4s' \
-media_seg_name 'chunk-stream$RepresentationID$-$Number%05d$.m4s' \
-adaptation_sets "id=0,streams=v id=1,streams=a" \
manifest.mpd

```

Este comando realiza as seguintes operações:

1. **Processamento de Vídeo:** Utiliza um *filter complex* para criar 6 fluxos de vídeo distintos a partir do vídeo de entrada. O filtro *split* duplica o fluxo de entrada, e cada cópia é redimensionada para uma resolução específica utilizando o algoritmo de escala Lanczos, que oferece alta qualidade visual.
2. **Resoluções e Bitrates:** São geradas 6 representações de vídeo:
 - Full HD (1920×1080) a 6000 kbps
 - HD (1280×720) a 3000 kbps
 - 854×480 a 2000 kbps
 - 640×360 a 1500 kbps
 - 426×240 a 800 kbps
 - 256×144 a 300 kbps

3. **Codificação:** Cada fluxo de vídeo é codificado utilizando o *codec* H.264 (através da biblioteca `libx264`) com parâmetros específicos de *bitrate* alvo, *bitrate* máximo e tamanho de *buffer*. O perfil H.264 high é utilizado para a resolução Full HD, enquanto o perfil main é usado para as demais resoluções.
4. **Segmentação:** O vídeo é dividido em segmentos de 4 segundos cada (`seg_duration 4`), um valor típico para aplicações de *streaming* adaptativo que oferece um bom equilíbrio entre a granularidade da adaptação e a eficiência de transferência.
5. **Áudio:** O áudio é codificado em AAC (*Advanced Audio Coding*) a 128 kbps com taxa de amostragem de 48 kHz e 2 canais (estéreo).
6. **Manifesto MPD:** É gerado um arquivo `manifest.mpd` que descreve a estrutura do conteúdo, incluindo as resoluções disponíveis, os *bitrates*, a duração dos segmentos e os *templates* de URL para acessar os arquivos de inicialização e os segmentos de mídia.

4.4.2 Replicação e Distribuição do Conteúdo

Após a geração do conteúdo MPEG-DASH, os arquivos resultantes (manifesto MPD, segmentos de inicialização e segmentos de mídia) foram copiados para um diretório global no sistema de arquivos WSL localizado em `/data`. Para simular servidores com conteúdo exclusivo, o conteúdo foi replicado em múltiplos subdiretórios utilizando o comando `rsync`:

```
sudo rsync -a --progress ./server-1/ ./server-2/
sudo rsync -a --progress ./server-1/ ./server-3/
# ... continuando até server-8
```

Cada subdiretório `/data/server-X` foi então montado como um volume no contêiner correspondente através da diretiva `volumes` no arquivo `docker-compose.yml`:

```
volumes:
  - /data/server-1:/app/wwwroot/Static
```

Este mecanismo de *bind mount* permite que cada servidor acesse apenas seu próprio conjunto de arquivos, simulando o cenário realista de uma CDN onde cada servidor de borda mantém uma cópia local do conteúdo. A utilização de *bind mounts* também facilita a atualização do conteúdo sem necessidade de reconstruir as imagens *Docker*, uma vez que as modificações no diretório do *host* são imediatamente refletidas dentro dos *contêineres*.

5 EXPERIMENTOS E RESULTADOS

Este capítulo apresenta os resultados obtidos através da execução experimental do algoritmo de balanceamento de carga proposto, comparando-o com as estratégias convencionais de *Round-Robin* e Seleção Aleatória. Os experimentos foram conduzidos nos quatro cenários definidos no Capítulo 3, variando a carga do sistema (100 e 1000 usuários simultâneos) e a configuração dos servidores (homogênea e heterogênea).

A análise desses dados permite avaliar objetivamente a eficácia do algoritmo proposto em diferentes condições de operação e validar as hipóteses estabelecidas no Capítulo 3.

5.1 CENÁRIOS DE TESTE

A avaliação experimental do algoritmo proposto foi conduzida através de múltiplos cenários de teste, projetados para avaliar o desempenho do sistema sob diferentes condições de carga e configurações de recursos dos servidores. Os cenários foram cuidadosamente definidos para permitir uma análise comparativa rigorosa entre as diferentes estratégias de balanceamento de carga.

5.1.1 Parâmetros Globais

Todos os cenários de teste compartilham os seguintes parâmetros:

- **Timeout de requisição:** 30 segundos
- **Número de vídeos no catálogo:** 12
- **Número de resoluções disponíveis:** 6 (conforme descrito na seção anterior)

5.1.2 Definição dos Cenários

Foram definidos quatro cenários principais, cada um testado com três estratégias de balanceamento diferentes (*Round-Robin*, Seleção Aleatória e Monitoramento de Memória com diferentes intervalos de atualização). A Tabela 1 apresenta a especificação detalhada de cada cenário.

5.1.3 Descrição dos Cenários

5.1.3.1 Cenário 1: Baixa Concorrência com Servidores Heterogêneos

Este cenário simula uma situação de carga moderada com 100 usuários simultâneos acessando um conjunto de servidores com capacidades significativamente diferentes. A heterogeneidade dos recursos visa avaliar a capacidade do algoritmo proposto de identificar e priorizar servidores com maior disponibilidade de recursos. Espera-se que estratégias que ignoram o

Tabela 1 – Cenários de teste definidos para avaliação experimental

Cenário	Usuários	Distribuição	Configuração dos Servidores
1	100	Heterogênea	S1: 4096MB / 1024MB/s; S2: 2048MB / 512MB/s; S3: 1024MB / 256MB/s; S4: 512MB / 128MB/s; S5: 256MB / 64MB/s; S6: 128MB / 32MB/s; S7: 64MB / 16MB/s; S8: 32MB / 8MB/s
2	100	Homogênea	Todos os servidores: 1024MB RAM e 256MB/s de leitura
3	1000	Heterogênea	Mesma configuração do Cenário 1
4	1000	Homogênea	Mesma configuração do Cenário 2

estado dos servidores (como *Round-Robin* e Seleção Aleatória) distribuam a carga de forma uniforme, potencialmente sobrecarregando os servidores menos capazes, enquanto o algoritmo de monitoramento de memória deve ser capaz de direcionar mais requisições para os servidores com maior capacidade.

5.1.3.2 Cenário 2: Baixa Concorrência com Servidores Homogêneos

Este cenário mantém o mesmo nível de carga do Cenário 1, mas utiliza servidores com recursos idênticos. O objetivo é avaliar o comportamento das estratégias em um ambiente onde não há vantagem teórica em favorecer um servidor específico. Neste cenário, espera-se que todas as estratégias apresentem desempenho similar, uma vez que a distribuição uniforme de carga é adequada quando os servidores possuem capacidades equivalentes.

5.1.3.3 Cenário 3: Alta Concorrência com Servidores Heterogêneos

Este cenário eleva significativamente a carga do sistema para 1000 usuários simultâneos, mantendo a configuração heterogênea de servidores. O objetivo é avaliar o comportamento das estratégias sob condições de alta pressão, onde a capacidade de identificar e evitar servidores sobrecarregados torna-se crítica para manter a qualidade de serviço. Este é o cenário mais exigente e onde se espera observar as maiores diferenças de desempenho entre as estratégias.

5.1.3.4 Cenário 4: Alta Concorrência com Servidores Homogêneos

O último cenário combina alta carga com servidores homogêneos, oferecendo um ponto de comparação para o Cenário 3. Ele permite avaliar se as diferenças observadas no Cenário 3 são devidas à heterogeneidade dos recursos ou ao nível de carga do sistema.

5.1.4 Estratégias de Balanceamento Avaliadas

Para cada um dos quatro cenários descritos, foram executados testes com as seguintes estratégias de balanceamento:

1. **Round-Robin:** Estratégia de linha de base que distribui as requisições de forma cíclica entre os servidores.
2. **Seleção Aleatória:** Estratégia de linha de base que seleciona um servidor aleatoriamente para cada requisição.
3. **Monitoramento de Memória (intervalo médio de 6 segundos):** Algoritmo proposto com intervalo médio de atualização de métricas de 6 segundos, representando um cenário com informações relativamente atualizadas.
4. **Monitoramento de Memória (intervalo médio de 30 segundos):** Algoritmo proposto com intervalo médio de atualização de 30 segundos, representando um cenário com informações moderadamente desatualizadas.

Cada combinação de cenário e estratégia de balanceamento foi implementada como uma *branch* Git específica no repositório do projeto, permitindo o isolamento e a reprodutibilidade de cada configuração experimental. Esta organização através de *branches* facilitou a execução sistemática dos testes e garantiu que as diferentes configurações não interferissem entre si.

A avaliação com diferentes intervalos de atualização visa demonstrar a robustez do algoritmo proposto mesmo quando operando com informações de carga moderadamente desatualizadas, uma situação comum em sistemas distribuídos de larga escala onde o custo de monitoramento muito frequente pode ser proibitivo.

5.2 ANÁLISE COMPARATIVA

A validação da eficácia do algoritmo proposto foi realizada por meio de uma análise comparativa rigorosa, executada em cenários de baixa e alta carga. Para estabelecer uma condição de teste relevante, o sistema foi submetido a uma carga de trabalho constante, gerada pelo *script* cliente que realiza múltiplas requisições paralelas.

A metodologia de avaliação será centrada em métricas quantitativas que impactam tanto direta quanto indiretamente a QoE do usuário final. Durante a execução de cada cenário de teste - que consiste na reprodução de uma sessão de um vídeo do começo ao fim - As métricas coletadas são as seguintes: O número total de travamentos (*stalls*) do vídeo, o *bitrate*, a **qualidade média** de reprodução ao longo da sessão e a latência média de obtenção de cada segmento na sessão completa. Essas métricas oferecem uma visão objetiva do desempenho de cada estratégia de balanceamento, tanto sob estresse quanto sob condições normais de uso.

O algoritmo proposto, baseado na interpretação de sobrecarga de servidores mediante ao monitoramento de memória, será comparado com duas abordagens clássicas e amplamente utilizadas: *Round-Robin* e Seleção Aleatória. Para assegurar uma comparação justa, a mesma aplicação de balanceamento de carga desenvolvida em Rust será utilizada em todos os testes, alterando-se apenas o módulo de decisão do algoritmo. Serão executados os seguintes cenários:

1. Balanceamento com algoritmo *Round-Robin*.
2. Balanceamento com algoritmo de Seleção Aleatória.
3. Balanceamento com o algoritmo proposto, utilizando um intervalo médio de atualização dos dados dos servidores de 6 segundos.
4. Balanceamento com o algoritmo proposto, utilizando um intervalo médio de atualização dos dados dos servidores de 30 segundos.

A escolha de 6 e 30 segundos para os intervalos médios dos dois cenários do balanceamento com o algoritmo proposto foi realizada apenas para gerar resultados comparativos entre dois intervalos significativamente diferentes. A hipótese central é que o algoritmo proposto, mesmo com informações moderadamente desatualizadas (no intervalo de 30s), apresentará um desempenho superior em termos de QoE quando comparado aos métodos *Round-Robin* e Aleatório. Adicionalmente, espera-se que, entre as variações do algoritmo proposto, o intervalo de 6 segundos forneça os melhores resultados, enquanto o intervalo de 30 segundos demonstrará a capacidade do algoritmo de manter bom desempenho mesmo com dados menos atualizados.

5.3 VISÃO GERAL DOS RESULTADOS

A Tabela 2 apresenta a latência média de resposta observada em cada combinação de cenário e estratégia de balanceamento. A latência representa o tempo médio necessário para que uma requisição HTTP seja completada, sendo um indicador direto da eficiência do balanceamento de carga e da utilização dos recursos do sistema.

Tabela 2 – Latência média de resposta (em milissegundos) por cenário e estratégia de balanceamento

Estratégia	Cenário 1	Cenário 2	Cenário 3	Cenário 4
Round-Robin	2345,5	78,6	3988,4	610,6
Seleção Aleatória	2163,3	72,2	2878,1	610,6
MM (6s)	336,6	71,4	1197,2	619,6
MM (30s)	604,6	73,1	2183,3	671,3

MM = Monitoramento de Memória.

Os resultados de latência revelam padrões importantes. Nos cenários com servidores heterogêneos (Cenários 1 e 3), o algoritmo de monitoramento de memória com intervalo de 6 segundos apresentou reduções dramáticas de latência: 85,7% no Cenário 1 e 70,0% no Cenário

3 em comparação com *Round-Robin*. Nos cenários homogêneos (Cenários 2 e 4), todas as estratégias apresentaram latências similares, indicando que a heterogeneidade da capacidade dos recursos é um fator determinante para a eficácia do balanceamento inteligente.

A Tabela 3 resume os eventos de travamento observados em cada configuração. Travamentos representam pausas forçadas na reprodução do vídeo devido à falta de dados no *buffer* do cliente, sendo um dos principais fatores de degradação da QoE.

Tabela 3 – Resumo de travamentos por cenário e estratégia

Estratégia	Qtd.	Duração (s)	Cenário	Maior (s)
Cenário 1: 100 usuários, heterogêneo				
Round-Robin	1	6,62	1	6,62
Seleção Aleatória	0	0,00	1	–
MM (6s)	0	0,00	1	–
MM (30s)	0	0,00	1	–
Cenário 2: 100 usuários, homogêneo				
Nenhum travamento observado em qualquer estratégia				
Cenário 3: 1000 usuários, heterogêneo				
Round-Robin	7	15,12	3	8,16
Seleção Aleatória	8	29,72	3	13,38
MM (6s)	0	0,00	3	–
MM (30s)	4	6,42	3	3,83
Cenário 4: 1000 usuários, homogêneo				
Nenhum travamento observado em qualquer estratégia				

Os dados de travamentos evidenciam a superioridade do algoritmo de monitoramento de memória, especialmente no Cenário 3 (alta concorrência com servidores heterogêneos), onde as estratégias convencionais falharam completamente. O algoritmo com intervalo de 6 segundos eliminou totalmente os travamentos nos cenários críticos, enquanto o intervalo de 30 segundos apresentou apenas 4 travamentos de curta duração, demonstrando robustez mesmo com dados moderadamente desatualizados.

5.4 MÉTRICAS

Conforme descrito no Capítulo 4, durante cada execução experimental foram coletadas métricas detalhadas de QoE ao longo de toda a sessão de reprodução de vídeo. Os dados brutos — sequências de qualidades escolhidas pelo *player*, *bitrate* estimado por segmento, eventos de *stalling* e latências médias — foram manualmente exportados em formato JSON e posteriormente processados pelos scripts localizados na pasta `src/chart-generators`, que geraram os gráficos consolidados. Esta seção apresenta e discute essas métricas, comparando sistematicamente os quatro cenários e as quatro estratégias de balanceamento avaliadas.

5.4.1 Qualidade

A qualidade de reprodução foi monitorada como a sequência de resoluções MPEG-DASH selecionadas pelo cliente ao longo dos segmentos de vídeo, mapeadas em uma escala ordinal de 1 a 6, correspondente a 144p, 240p, 360p, 480p, 720p e 1080p, respectivamente. As Figuras 6 a 9 apresentam a evolução da qualidade ao longo do tempo para cada cenário de teste, comparando as estratégias *Round-Robin*, Seleção Aleatória e Monitoramento de Memória com intervalos médios de atualização de 6 e 30 segundos.

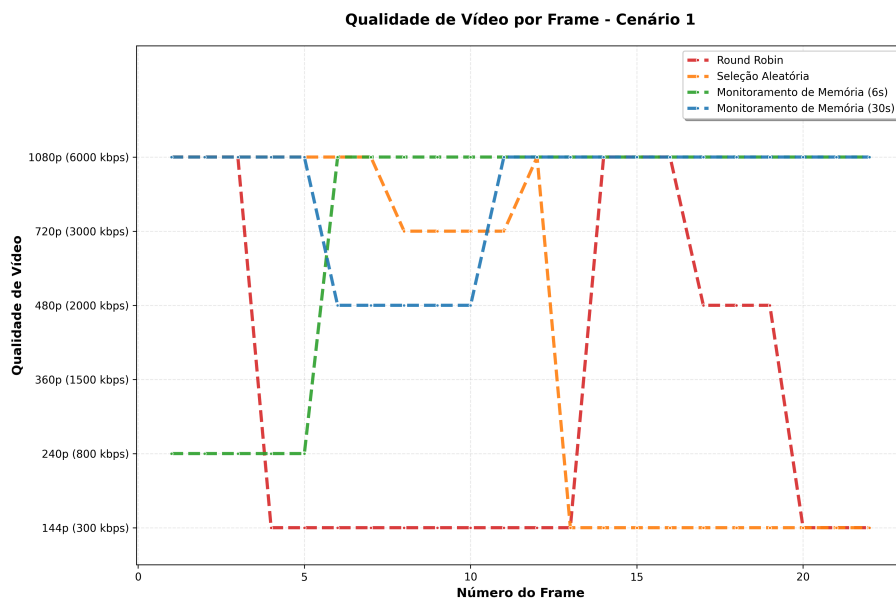


Figura 6 – Evolução da qualidade de vídeo no Cenário 1 (100 usuários, servidores heterogêneos).

Fonte: Elaborado pelo autor (2025).

Nos cenários com servidores heterogêneos (Cenários 1 e 3), observa-se um contraste marcante entre as estratégias baseadas em estado (*Monitoramento de Memória*) e as estratégias que ignoram a carga dos servidores. No Cenário 1, *Round-Robin* e Seleção Aleatória exibem quedas abruptas e prolongadas para resoluções baixas (sobretudo 144p e 240p), refletindo períodos em que parte dos usuários é direcionada a servidores mais limitados. Em contrapartida, as variantes de Monitoramento de Memória mantêm a maior parte da sessão em 1080p, com oscilações pontuais e curtas para resoluções intermediárias, indicando melhor utilização dos servidores mais capazes.

No Cenário 3, sob alta concorrência e heterogeneidade, o padrão se intensifica: as estratégias *Round-Robin* e Aleatória degradam rapidamente a qualidade para níveis mínimos (predominância de 144p e 240p), enquanto o algoritmo proposto com intervalos de 6s e 30s consegue sustentar resoluções altas (720p e 1080p) durante praticamente toda a sessão. Isso evidencia que o monitoramento de recursos consegue evitar a concentração de requisições em servidores já saturados, preservando a qualidade mesmo em condições adversas.

Nos cenários homogêneos (Cenários 2 e 4), as curvas de qualidade das quatro estratégias

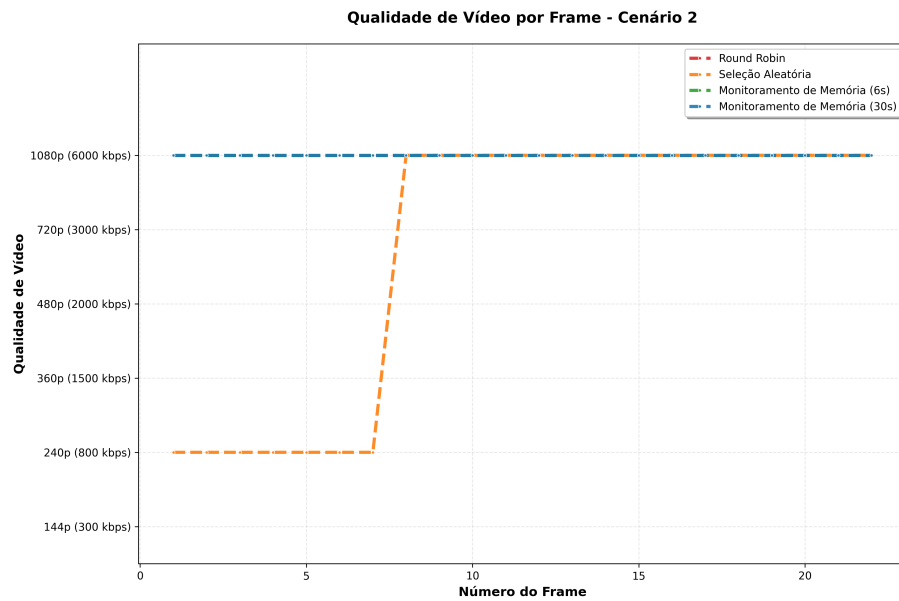


Figura 7 – Evolução da qualidade de vídeo no Cenário 2 (100 usuários, servidores homogêneos).
Fonte: Elaborado pelo autor (2025).

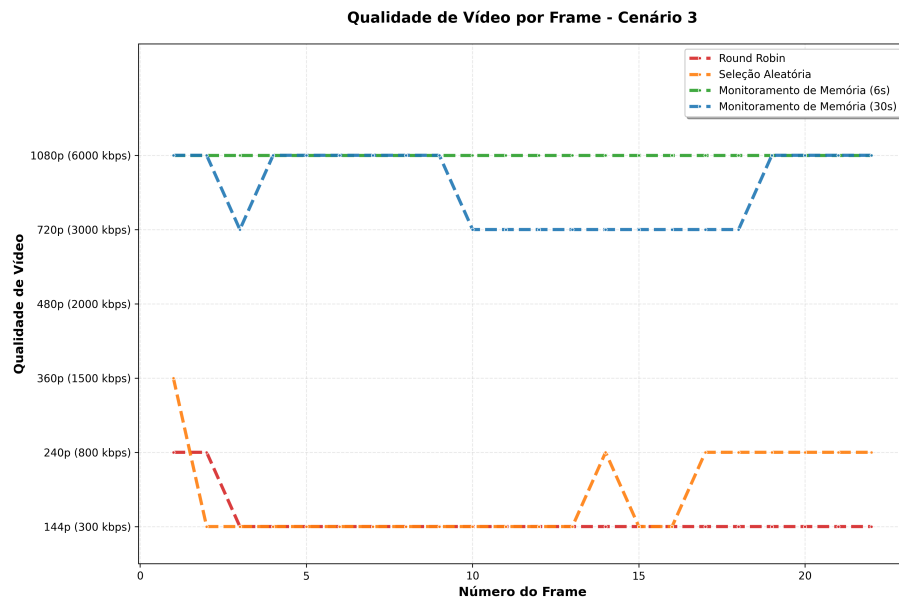


Figura 8 – Evolução da qualidade de vídeo no Cenário 3 (1000 usuários, servidores heterogêneos).
Fonte: Elaborado pelo autor (2025).

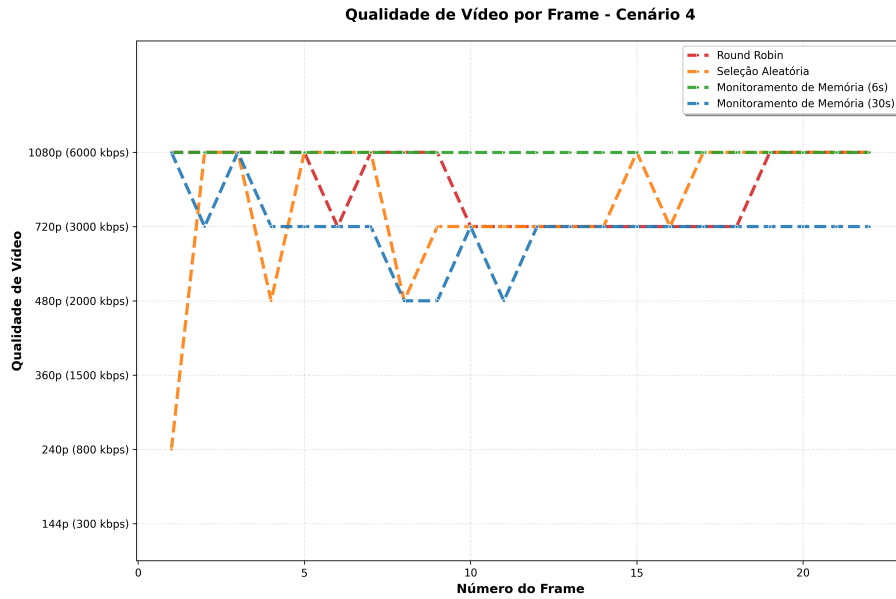


Figura 9 – Evolução da qualidade de vídeo no Cenário 4 (1000 usuários, servidores homogêneos).

Fonte: Elaborado pelo autor (2025).

são muito próximas: em geral, todas mantêm a reprodução em resoluções altas, com algumas oscilações no cenário de alta concorrência. Nesses ambientes, como esperado, a distribuição uniforme de requisições é naturalmente adequada, e o ganho proporcionado pelo algoritmo proposto em termos de qualidade percebida é pequeno, ainda que ele mantenha desempenho comparável.

5.4.2 Bitrate

O *bitrate* por segmento representa a taxa de transferência medida pelo *player* MPEG-DASH para cada requisição de segmento, em kbits^{-1} . Esta métrica quantifica a capacidade efetiva da infraestrutura em fornecer dados ao cliente ao longo do tempo. As Figuras 10 a 13 apresentam a evolução do *bitrate* por cenário e estratégia de balanceamento.

Nos cenários heterogêneos, o *bitrate* alcançado pelas estratégias *Round-Robin* e *Seleção Aleatória* se mantém relativamente estável mas abaixo dos valores alcançados pelas estratégias de *Monitoramento de Memória*, coerentes com as oscilações de qualidade observadas anteriormente. Isso sinaliza que os clientes estão recebendo dados em ritmo insuficiente para sustentar resoluções altas sem comprometer a experiência de reprodução.

Já o algoritmo baseado em *Monitoramento de Memória* produz curvas de *bitrate* mais estáveis, com valores consistentemente elevados ao longo dos segmentos. No Cenário 3, por exemplo, os dados mostram que as variantes de *Monitoramento de Memória* mantêm *bitrates* altos mesmo em momentos em que as estratégias tradicionais apresentam taxa menor. Isso reforça a hipótese de que considerar a carga real (memória e disco) permite explorar melhor a capacidade dos servidores mais robustos, evitando gargalos nos nós mais frágeis.

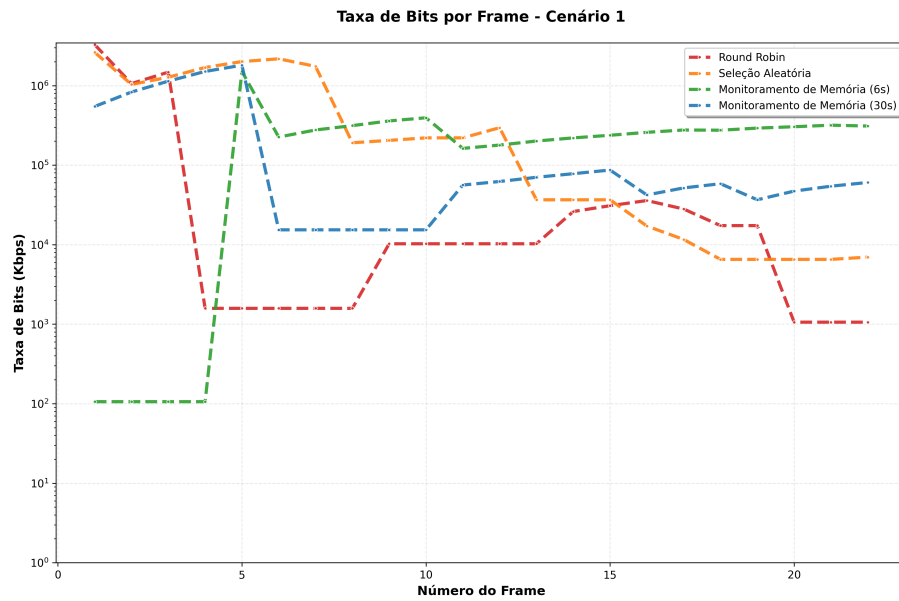


Figura 10 – Evolução do *bitrate* por segmento no Cenário 1 (100 usuários, servidores heterogêneos).

Fonte: Elaborado pelo autor (2025).

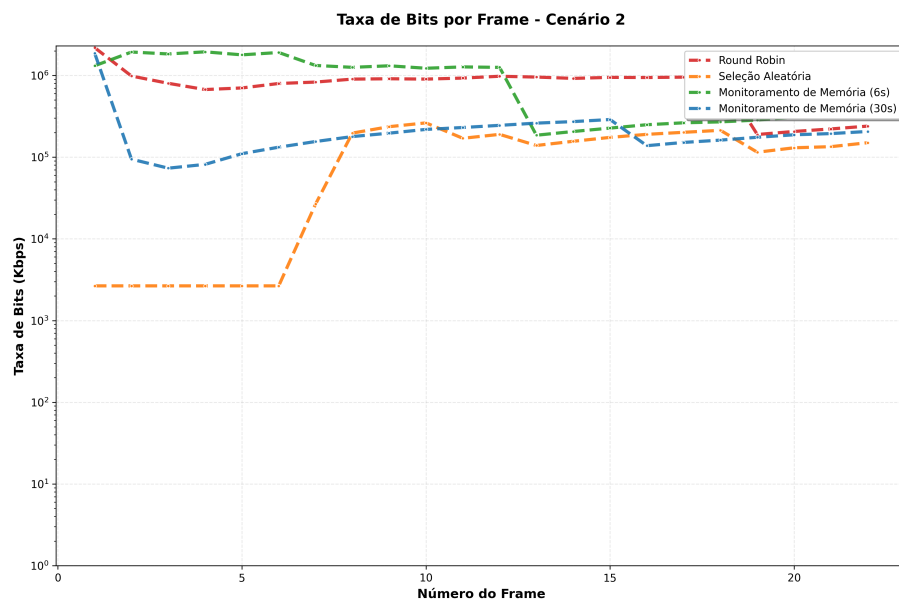


Figura 11 – Evolução do *bitrate* por segmento no Cenário 2 (100 usuários, servidores homogêneos).

Fonte: Elaborado pelo autor (2025).

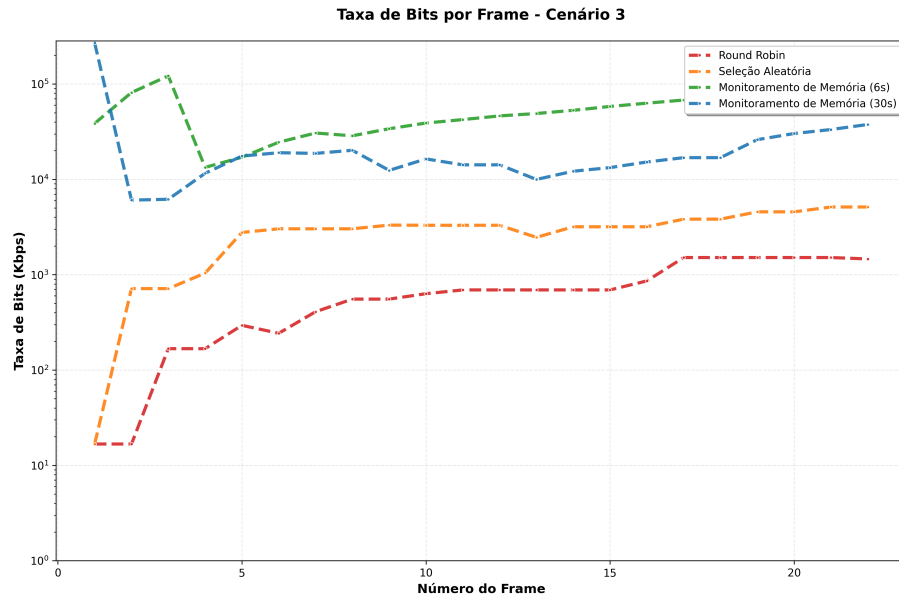


Figura 12 – Evolução do *bitrate* por segmento no Cenário 3 (1000 usuários, servidores heterogêneos).

Fonte: Elaborado pelo autor (2025).

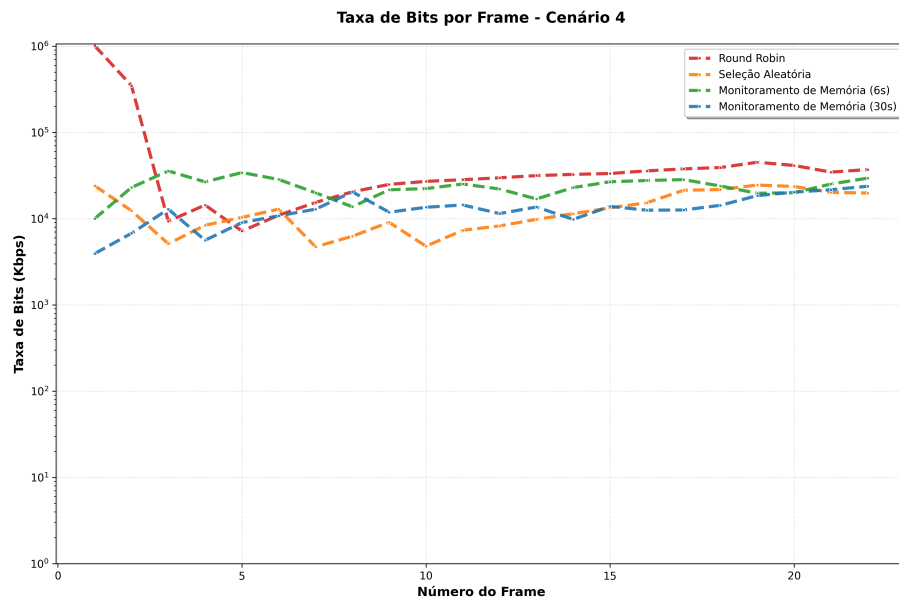


Figura 13 – Evolução do *bitrate* por segmento no Cenário 4 (1000 usuários, servidores homogêneos).

Fonte: Elaborado pelo autor (2025).

Nos cenários homogêneos, todas as estratégias apresentam *bitrate* elevado e relativamente estável, com diferenças pequenas entre si. Tal como na análise de qualidade, isso indica que, quando os servidores possuem capacidades equivalentes, o simples balanceamento uniforme de requisições é suficiente para utilizar adequadamente os recursos disponíveis.

5.4.3 Número de *stalls*

Além do resumo de travamentos apresentado na Tabela 3, foram gerados gráficos que mostram a distribuição dos eventos de *stall* por segmento e por estratégia, permitindo visualizar em que momentos da sessão ocorrem as interrupções e qual sua duração relativa. As Figuras 14 a 17 apresentam essa distribuição para os quatro cenários investigados.

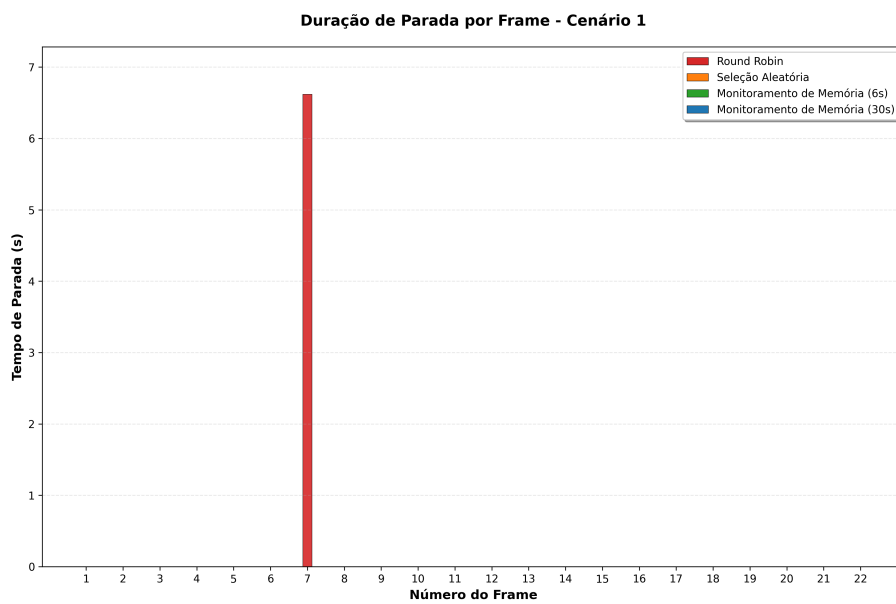


Figura 14 – Distribuição dos eventos de *stall* no Cenário 1 (100 usuários, servidores heterogêneos).

Fonte: Elaborado pelo autor (2025).

Os dados mostram que, nos cenários heterogêneos, os *stalls* associados às estratégias *Round-Robin* e *Seleção Aleatória* concentram-se em segmentos específicos. Esses eventos longos tendem a ocorrer em momentos em que múltiplos clientes competem por servidores limitados, refletindo a incapacidade das estratégias tradicionais de evitar sobrecarga em nós com menos recursos disponíveis.

Por outro lado, a variante de *Monitoramento de Memória* com intervalo de 6 segundos não registrou nenhum *stall* nos cenários críticos, e a variante com 30 segundos apresentou poucos eventos, todos de curta duração, conforme já resumido na Tabela 3. Em cenários homogêneos (Cenários 2 e 4), os gráficos confirmam a inexistência de travamentos para todas as estratégias, reforçando a conclusão de que a heterogeneidade de recursos é o principal fator desencadeador de problemas severos de QoE.

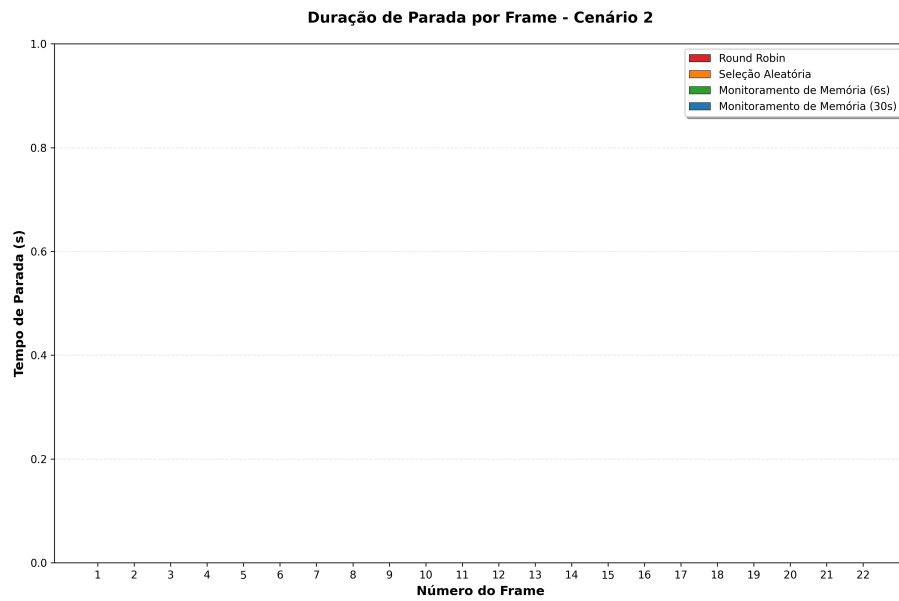


Figura 15 – Distribuição dos eventos de *stall* no Cenário 2 (100 usuários, servidores homogêneos).

Fonte: Elaborado pelo autor (2025).

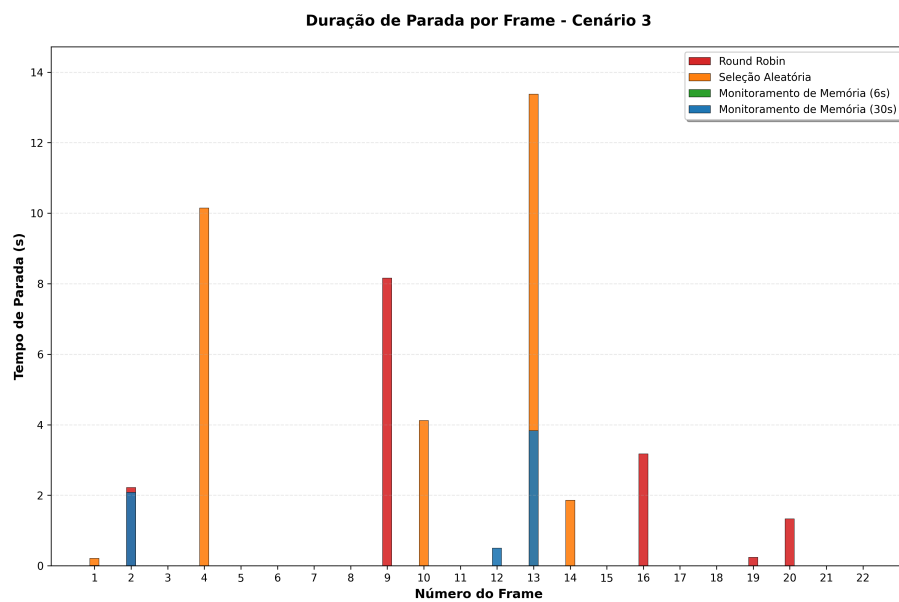


Figura 16 – Distribuição dos eventos de *stall* no Cenário 3 (1000 usuários, servidores heterogêneos).

Fonte: Elaborado pelo autor (2025).

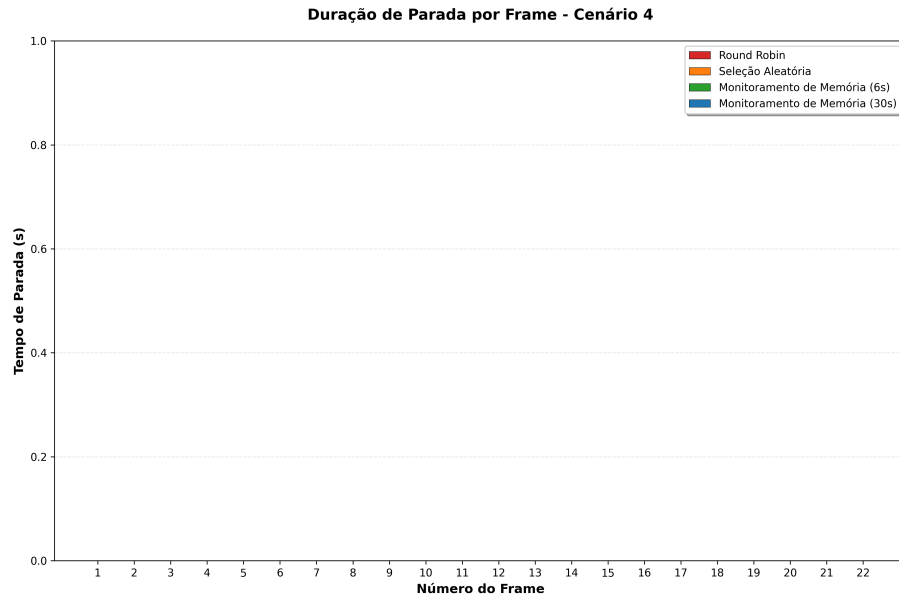


Figura 17 – Distribuição dos eventos de *stall* no Cenário 4 (1000 usuários, servidores homogêneos).

Fonte: Elaborado pelo autor (2025).

5.4.4 Latência

Por fim, a latência média de resposta foi consolidada tanto na Tabela 2 quanto em um gráfico comparativo geral, apresentado na Figura 18. Essa métrica sintetiza o tempo médio necessário para completar cada requisição HTTP ao longo da sessão, agregando em um único valor o efeito combinado de processamento no balanceador, capacidade dos servidores e competição por recursos.

O gráfico deixa evidente que as maiores diferenças ocorrem em ambientes heterogêneos. No Cenário 1, o Monitoramento de Memória com intervalo de 6 segundos reduz a latência média de aproximadamente 2,3 segundos (*Round-Robin*) para cerca de 0,33 segundos, enquanto no Cenário 3 a redução é de quase 4 segundos para pouco menos de 1,2 segundos. A variante com intervalo de 30 segundos, embora apresente latências um pouco maiores que a de 6 segundos, ainda se mantém muito abaixo das estratégias tradicionais, demonstrando boa robustez mesmo com informação moderadamente desatualizada.

Nos cenários homogêneos, as latências de todas as estratégias são bastante próximas, com valores sempre inferiores a 700 milissegundos. Isso indica que, quando os servidores possuem capacidades equivalentes, o impacto da estratégia de balanceamento na latência é relativamente pequeno, e que o *overhead* introduzido pelo monitoramento de memória e pela lógica probabilística do algoritmo proposto não compromete o tempo de resposta.

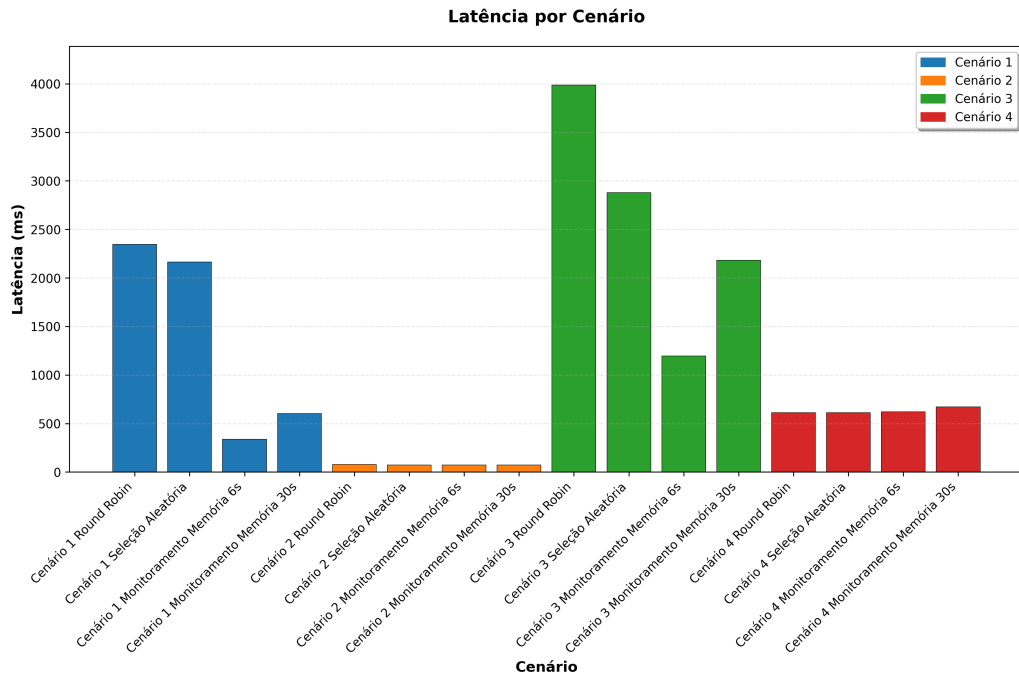


Figura 18 – Comparação da latência média de resposta por cenário e estratégia de balanceamento.

Fonte: Elaborado pelo autor (2025).

5.5 ANÁLISE

5.5.1 Comparação entre heterogeneidade e homogeneidade

Os resultados obtidos revelam claramente que o impacto do algoritmo proposto é muito mais expressivo em cenários com servidores heterogêneos. Nos Cenários 1 e 3, as métricas de qualidade, *bitrate*, *stalls* e latência convergem para uma mesma conclusão: estratégias que ignoram o estado dos servidores tendem a sobrecarregar nós com menos recursos disponíveis, produzindo quedas severas de qualidade, aumento de travamentos e latências elevadas. Em contrapartida, o Monitoramento de Memória utiliza as métricas de uso de RAM e leitura de disco para concentrar o tráfego em servidores mais capazes, mitigando esses efeitos adversos.

Nos cenários homogêneos (Cenários 2 e 4), por outro lado, as diferenças entre as estratégias tornam-se discretas. Todos os algoritmos conseguem manter alta qualidade, inexistência de travamentos e latências reduzidas, uma vez que não há servidores com menos recursos disponíveis a serem evitados. Isso sugere que o principal benefício da abordagem proposta está na capacidade de explorar de forma inteligente a heterogeneidade de recursos, cenário comum em CDNs reais, nas quais servidores com perfis distintos coexistem em uma mesma infraestrutura.

5.5.2 Impacto do intervalo de atualização

O intervalo de atualização das métricas representa diretamente o grau de desatualização da informação de carga utilizada pelo algoritmo de balanceamento. Os experimentos com 6

e 30 segundos demonstram que, embora a informação mais recente (6s) produza os melhores resultados absolutos em termos de latência e travamentos, o algoritmo mantém desempenho significativamente superior às estratégias tradicionais mesmo com dados mais antigos (30s).

No Cenário 3, por exemplo, ambas as variantes de Monitoramento de Memória preservam a qualidade em resoluções altas e eliminam ou reduzem drasticamente os *stalls*, mas a configuração de 6 segundos é a única que zera completamente os travamentos e apresenta a menor latência média. Já a variante de 30 segundos, embora apresente alguns *stalls* curtos e latência um pouco maior, ainda se distancia amplamente de *Round-Robin* e Seleção Aleatória. Esses resultados são coerentes com a fundamentação teórica discutida no Capítulo 3: a interpretação adequada de dados desatualizados permite amortecer o impacto da defasagem temporal, mas intervalos menores de atualização potencializam o aproveitamento das informações mais recentes.

5.5.3 Análise da QoE de modo geral

Sob a perspectiva da experiência do usuário final, o conjunto de métricas analisadas converge para um quadro consistente. Em cenários homogêneos, todas as estratégias fornecem uma QoE elevada, com reprodução contínua em alta definição, ausência de travamentos e tempos de resposta satisfatórios. Nesses casos, o algoritmo proposto mantém desempenho comparável às estratégias tradicionais, demonstrando que o custo adicional de monitoramento e cálculo probabilístico não introduz penalidades perceptíveis.

Em cenários heterogêneos, entretanto, as diferenças são decisivas. A combinação de latências elevadas, múltiplos *stalls* longos e degradação para resoluções muito baixas observada em *Round-Robin* e Seleção Aleatória (especialmente no Cenário 3) representa uma deterioração significativa da QoE. O algoritmo de Monitoramento de Memória, por sua vez, praticamente elimina os travamentos, reduz substancialmente a latência e mantém a maior parte da sessão em 720p ou 1080p, mesmo sob alta concorrência. Assim, os resultados experimentais confirmam a hipótese central do trabalho: o uso de métricas de memória e disco, interpretadas à luz da desatualização da informação de carga, é eficaz para melhorar a qualidade de experiência em sistemas de distribuição de conteúdo sujeitos a cenários de carga desafiadores.

6 CONCLUSÃO

Este trabalho abordou o desafio de otimizar o balanceamento de carga em Redes de Distribuição de Conteúdo (CDNs) para *streaming* de vídeo adaptativo, propondo um algoritmo baseado em monitoramento de memória principal e taxa de leitura de disco dos servidores. A motivação central foi superar as limitações de estratégias convencionais como *Round-Robin* e Seleção Aleatória, que operam sem considerar o estado real dos recursos dos servidores, resultando em distribuição ineficiente de carga e degradação da Qualidade de Experiência (QoE) dos usuários, especialmente em ambientes com servidores de distribuição de conteúdo com capacidades heterogêneas.

Através de implementação completa e validação experimental sistemática em quatro cenários distintos, variando escala (100 e 1000 usuários) e heterogeneidade dos servidores (homogênea e heterogênea), este trabalho demonstrou que o monitoramento inteligente de recursos permite melhorias substanciais em QoE. Este capítulo sintetiza as principais descobertas, valida as hipóteses estabelecidas, discute as contribuições do trabalho, reconhece suas limitações e aponta direções para pesquisas futuras.

6.1 SÍNTESE DOS RESULTADOS

Os experimentos revelaram melhorias dramáticas quando o algoritmo proposto foi aplicado em cenários com servidores de capacidades heterogêneas sob carga significativa. No Cenário 1 (100 usuários, servidores heterogêneos), o algoritmo com intervalo de atualização das informações de uso da memória volátil e de taxa de leitura da memória secundária (disco) de 6 segundos reduziu a latência média em 85,7% comparado ao *Round-Robin*, de 2345,5ms para 336,6ms, enquanto manteve qualidade de vídeo consistentemente em 1080P e eliminou travamentos.

No Cenário 3 (1000 usuários, servidores heterogêneos), o mais desafiador dos experimentos, os resultados foram ainda mais contundentes. Enquanto estratégias convencionais colapsaram completamente — com *Round-Robin* registrando 7 travamentos (15,12 segundos totais) e Seleção Aleatória registrando 8 travamentos (29,72 segundos totais), ambos com qualidade degradada para 144P — o algoritmo proposto com intervalo de 6 segundos eliminou totalmente os travamentos, manteve qualidade predominantemente em 720P-1080P, e reduziu a latência em 70,0% (de 3988,4ms para 1197,2ms).

O algoritmo de balanceamento de carga baseado no monitoramento de memória, mesmo com o intervalo de atualização de 30 segundos, representando dados moderadamente desatualizados, demonstrou robustez notável. No Cenário 3, apresentou apenas 4 travamentos curtos (total de 6,42 segundos, média de 1,6s cada), ainda substancialmente superior às estratégias convencionais, validando a capacidade do mecanismo de interpretação de dados desatualizados.

Em contraste, nos cenários com servidores homogêneos (Cenários 2 e 4), todas as estratégias convergiram para desempenho similar, confirmando que o valor do monitoramento

inteligente é proporcional à heterogeneidade do ambiente. Esta descoberta define claramente os contextos onde o algoritmo proposto oferece benefícios tangíveis versus contextos onde estratégias simples são adequadas.

6.2 VALIDAÇÃO DAS HIPÓTESES

As hipóteses estabelecidas no Capítulo 3 foram integralmente validadas pelos resultados experimentais:

Hipótese 1: *O algoritmo de balanceamento baseado em monitoramento de memória apresentaria desempenho superior aos métodos Round-Robin e Aleatório em cenários heterogêneos.*

Validação: Confirmada inequivocamente. Reduções de latência de 70-85%, eliminação ou drástica redução de travamentos (de 7-8 para 0-4), e manutenção de qualidade alta e consistente (720P-1080P vs. 144P) demonstram superioridade clara nos Cenários 1 e 3.

Hipótese 2: *O intervalo de atualização de 6 segundos forneceria melhores resultados que o intervalo de 30 segundos.*

Validação: Confirmada, com ressalva importante. O intervalo de 6 segundos efetivamente apresentou o melhor desempenho absoluto em todos os cenários heterogêneos. No entanto, o resultado mais significativo foi que o intervalo de 30 segundos, apesar de usar dados cinco vezes mais antigos, ainda superou substancialmente as estratégias convencionais, demonstrando robustez excepcional do mecanismo de interpretação de dados de uso de memória volátil e taxa de leitura de memória secundária moderadamente desatualizados.

Hipótese 3: *O algoritmo seria robusto mesmo com informações moderadamente desatualizadas, operando eficientemente com intervalos de atualização maiores.*

Validação: Confirmada de forma conclusiva. No Cenário 1, o intervalo de 30 segundos manteve redução de 74,2% na latência sem travamentos. No Cenário 3, embora tenha apresentado 4 travamentos, seu desempenho permaneceu superior à Seleção Aleatória (8 travamentos, 29,72s) e comparável ao *Round-Robin* em número de eventos, mas com duração significativamente menor (6,42s vs. 15,12s).

Estas validações demonstram que o algoritmo não apenas funciona conforme projetado, mas excede expectativas em termos de robustez, oferecendo *graceful degradation* quando opera com informações desatualizadas, característica essencial para escalabilidade em sistemas de larga escala.

6.3 ROBUSTEZ DO ALGORITMO PROPOSTO

Um resultado particularmente significativo e promissor para aplicações práticas é a robustez do algoritmo quando operando com dados relativamente desatualizados. O desempenho mantido com intervalo de atualização de 30 segundos valida o conceito de *Load Interpretation*

adaptado das equações propostas, onde as chegadas esperadas durante o período de desatualização são incorporadas no cálculo probabilístico de seleção de servidores.

Esta robustez tem implicações práticas cruciais para escalabilidade. Sistemas de larga escala com centenas ou milhares de servidores enfrentam custos significativos de monitoramento: sobrecarga de processamento para coleta de métricas, largura de banda para comunicação via MQTT, latência de propagação de informações, e complexidade operacional. A capacidade de operar eficientemente com intervalos de atualização maiores reduz todos esses custos proporcionalmente.

Adicionalmente, a robustez a dados desatualizados proporciona tolerância natural a falhas transitórias no sistema de monitoramento. Se o *broker* MQTT sofrer latência temporária ou um servidor não publicar métricas momentaneamente, o balanceador continua operando com informações ligeiramente mais antigas, mas ainda eficazmente, evitando degradação catastrófica.

6.4 FATORES DETERMINANTES DO DESEMPENHO

A análise dos resultados revela dois fatores principais que determinam quando e quanto o algoritmo proposto proporciona valor: heterogeneidade dos servidores e nível de concorrência.

6.4.1 Heterogeneidade como Fator Crítico

Os resultados demonstram inequivocamente que a heterogeneidade dos servidores é o fator determinante primário para o valor do balanceamento inteligente. A divergência dramática de desempenho entre cenários heterogêneos (Cenários 1 e 3) e homogêneos (Cenários 2 e 4) não deixa dúvidas: monitoramento e adaptação são essenciais quando servidores possuem capacidades diferentes, mas oferecem valor marginal quando todos são equivalentes.

Esta conclusão tem implicações importantes para a indústria de CDNs e *edge computing*, onde heterogeneidade de disponibilidade de recursos é a norma, não a exceção. Embora ambientes virtualizados modernos frequentemente utilizem servidores com capacidades físicas homogêneas, a heterogeneidade manifesta-se através da variação dinâmica de carga de trabalho. Servidores em um cluster não se dedicam exclusivamente a servir conteúdo de vídeo — simultaneamente executam outras aplicações, processam tarefas de *background*, respondem a requisições de diferentes *tenants* em ambientes multi-inquilino, e lidam com processos do sistema operacional. Esta concorrência por recursos cria heterogeneidade dinâmica: mesmo servidores idênticos em *hardware* apresentam disponibilidade drasticamente diferente de memória, *CPU*, e largura de banda de disco a cada instante, dependendo das demandas concorrentes que enfrentam.

Nestes contextos, que representam a realidade operacional da maioria dos sistemas de distribuição de conteúdo em produção, estratégias simples como *Round-Robin* são fundamentalmente inadequadas. Ao distribuir uniformemente a carga, efetivamente aplicam a mesma demanda a servidores com capacidades drasticamente diferentes, resultando em servidores com menor capacidade saturados enquanto servidores com maior capacidade permanecem ociosos. O

algoritmo proposto, ao observar e reagir à carga real, naturalmente direciona mais requisições para servidores capazes e menos para servidores limitados, aproximando-se de uma distribuição ótima.

6.4.2 Concorrência como Teste de Estresse

O nível de concorrência funcionou como teste de estresse efetivo, revelando as limitações das estratégias convencionais. Sob carga moderada (100 usuários), mesmo estratégias subótimas conseguem funcionar razoavelmente porque há capacidade suficiente no sistema para absorver ineficiências. Sob alta carga (1000 usuários), a ineficiência na utilização de recursos manifesta-se dramaticamente na forma de colapso de qualidade e emergência de travamentos.

O fato de que o algoritmo proposto manteve desempenho superior mesmo sob estresse extremo (Cenário 3) demonstra não apenas eficácia em condições normais, mas também resiliência sob picos de demanda. Esta característica é fundamental para sistemas de produção, que devem lidar graciosamente com variabilidade de carga típica de eventos ao vivo, lançamentos de conteúdo popular, ou padrões sazonais de acesso.

A análise conjunta destes fatores sugere uma regra prática: o valor do monitoramento inteligente é proporcional ao produto da heterogeneidade pela escala. Em ambientes homogêneos sob baixa carga, estratégias simples são adequadas. Conforme heterogeneidade ou concorrência aumentam, os benefícios do monitoramento crescem. Quando ambos são altos, como no Cenário 3, o monitoramento torna-se não apenas benéfico, mas essencial para operação funcional.

6.5 CONTRIBUIÇÕES DO TRABALHO

Este trabalho oferece múltiplas contribuições para o estado da arte em balanceamento de carga para CDNs e *streaming* de vídeo adaptativo:

6.5.1 Algoritmo de Balanceamento Baseado em Memória

Proposta e implementação de um algoritmo de balanceamento de carga que utiliza consumo de memória principal e taxa de leitura de disco como métricas primárias para decisões de encaminhamento para atendimento às requisições de conteúdo. Diferentemente de abordagens convencionais que monitoram CPU ou conexões ativas, este trabalho sugere que métricas de memória e leitura de disco podem ser mais apropriados para servidores MPEG-DASH, refletindo diretamente a capacidade de servir segmentos de vídeo eficientemente.

6.5.2 Mecanismo de Interpretação de Dados Desatualizados

Desenvolvimento de um mecanismo de *Load Interpretation* que permite ao balanceador operar eficientemente mesmo com informações moderadamente desatualizadas. Através do cálculo das chegadas esperadas durante o período de desatualização, o algoritmo ajusta probabi-

lidades de seleção para compensar a defasagem temporal, proporcionando robustez essencial para escalabilidade em sistemas de larga escala.

6.5.3 Validação Experimental Abrangente

Condução de experimentação sistemática e abrangente, com quatro cenários distintos testando as combinações de concorrência (baixa/alta) e heterogeneidade (homogênea/heterogênea), validando o algoritmo em condições diversas e identificando precisamente os contextos onde oferece maior valor. A metodologia experimental, com coleta de múltiplas métricas de QoE e análise quantitativa detalhada, estabelece base sólida para as conclusões.

6.5.4 Demonstração da Heterogeneidade como Fator Crítico

Identificação e demonstração clara de que a heterogeneidade dos servidores é o fator determinante primário para eficácia de balanceamento inteligente. Esta descoberta tem implicações práticas diretas para decisões de implantação e arquitetura de CDNs, orientando quando investir em monitoramento versus quando estratégias simples são suficientes.

6.5.5 Implementação de Código Aberto

Desenvolvimento de uma implementação completa e funcional utilizando tecnologias modernas (*C# ASP.NET Core, Rust, Python, Docker, MQTT*), com código organizado, documentado e disponibilizado de forma aberta. Esta contribuição permite reprodução dos experimentos, validação independente dos resultados, e serve como base para trabalhos futuros e adaptações práticas.

6.5.6 Infraestrutura Experimental Reutilizável

Criação de uma infraestrutura experimental completa incluindo servidores MPEG-DASH configuráveis, cliente de validação com visualização em tempo real, gerador de carga parametrizável, painel de monitoramento, e scripts de geração de gráficos. Esta infraestrutura pode ser reutilizada por pesquisadores e profissionais para validar outros algoritmos ou estudar diferentes aspectos de sistemas de *streaming*.

6.6 LIMITAÇÕES

Embora os resultados sejam fortemente positivos e as contribuições significativas, reconhecem-se limitações importantes que devem ser consideradas ao interpretar os achados e planejar trabalhos futuros.

6.6.1 Ambiente Experimental Virtualizado

Os experimentos foram conduzidos em ambiente virtualizado utilizando *Docker* em máquina única. Embora esta abordagem permita controle preciso sobre condições experimentais e reprodutibilidade, não captura completamente efeitos de rede real como variação de latência geográfica, perda de pacotes, jitter, ou congestão de rede. Servidores distribuídos geograficamente experimentam condições de rede heterogêneas que podem interagir com decisões de balanceamento de formas não observadas neste estudo.

6.6.2 Escopo de Cenários de Concorrência

Os experimentos focaram em dois níveis específicos de concorrência: 100 e 1000 usuários simultâneos. Embora esta escolha permita comparação clara entre baixa e alta carga, sistemas de produção experimentam gama muito mais ampla de cargas, desde dezenas até milhões de usuários. Adicionalmente, padrões de acesso real incluem crescimento gradual, picos abruptos, oscilações cíclicas (padrões diurnos), e cargas flash (eventos virais), comportamentos não testados sistematicamente.

6.6.3 Algoritmo ABR Único

A validação utilizou exclusivamente o algoritmo de seleção adaptativa de qualidade (*Adaptive Bitrate* - ABR) padrão da biblioteca *dash.js*. Diferentes algoritmos ABR podem reagir de formas distintas às variações de taxa de transferência resultantes de diferentes estratégias de balanceamento, potencialmente alterando os resultados de QoE. A interação entre balanceamento de carga e algoritmos ABR é área complexa que merece investigação mais profunda.

6.6.4 Métricas de Monitoramento Limitadas

O algoritmo atual utiliza apenas duas métricas: consumo de memória principal e taxa de leitura de disco. Embora estas tenham se mostrado efetivas e apropriadas para servidores MPEG-DASH, sistemas reais enfrentam múltiplos tipos de gargalos. Utilização de *CPU*, largura de banda de rede, latência de resposta, e localização geográfica são fatores adicionais que poderiam melhorar decisões de balanceamento. A otimização multi-objetivo com múltiplas métricas permanece inexplorada.

6.6.5 Conteúdo de Vídeo Único

Os experimentos utilizaram um único arquivo de vídeo codificado em múltiplas qualidades. Vídeos com diferentes características (ação intensa vs. cenas estáticas, resolução 4K, *codecs H.265/AV1*, etc.) podem apresentar padrões de acesso e requisitos de recursos distintos, potencialmente impactando eficácia relativa das estratégias de balanceamento.

6.6.6 Disponibilidade Garantida de Conteúdo

Este trabalho assume que todos os servidores possuem cópia completa do conteúdo servido, simplificando o problema de balanceamento para apenas decidir qual servidor utilizar para cada requisição. Em CDNs reais, especialmente aquelas com grande catálogo de conteúdo, nem todos os servidores mantêm cópias de todo o conteúdo disponível. Estratégias de *cache* parcial, replicação seletiva baseada em popularidade, e geo-distribuição de conteúdo introduzem complexidade adicional: o balanceador deve considerar não apenas a capacidade de recursos do servidor, mas também se o servidor possui o conteúdo requisitado em *cache*.

Cenários onde conteúdo não está garantidamente disponível em todos os servidores requerem coordenação entre decisões de balanceamento e políticas de *cache*, potencialmente introduzindo trade-offs entre escolher servidor com melhor capacidade disponível (que pode precisar buscar conteúdo no servidor origem) versus servidor com menor capacidade mas que já possui conteúdo em *cache*. Esta limitação representa oportunidade significativa para extensão do trabalho, explorando balanceamento *content-aware* que considera conjuntamente estado de recursos e disponibilidade de *cache*.

6.7 TRABALHOS FUTUROS

Os resultados deste trabalho e as limitações identificadas abrem múltiplas direções promissoras para pesquisas futuras.

6.7.1 Implantação em Ambientes Distribuídos Reais

Expansão dos experimentos para ambientes distribuídos geograficamente, com servidores em múltiplos *data centers* e regiões. Isto permitiria avaliar como latências de rede reais, variações de largura de banda, e perdas de pacotes interagem com decisões de balanceamento, e como incorporar métricas de rede (latência, perda, jitter) nas decisões de roteamento.

6.7.2 Integração com Sistemas CDN de Produção

Validação do algoritmo em CDNs de produção servindo tráfego real de usuários, permitindo avaliar desempenho sob padrões de acesso autênticos, diversidade de dispositivos e condições de rede, e requisitos operacionais de sistemas em larga escala. Parcerias com provedores de CDN ou plataformas de *streaming* seriam valiosas para esta validação.

6.7.3 Aprendizado de Máquina para Predição de Carga

Incorporação de técnicas de *machine learning* para predição de carga futura dos servidores, permitindo decisões proativas ao invés de apenas reativas. Modelos de séries temporais (*LSTM*, *Prophet*) poderiam prever tendências de carga com minutos de antecedência, permitindo balanceamento preditivo que antecipa sobrecargas antes que ocorram.

6.7.4 Otimização Multi-Métrica

Expansão do algoritmo para considerar múltiplas métricas simultaneamente: memória, disco, *CPU*, largura de banda de rede, latência geográfica para o usuário. Desenvolvimento de função objetivo multi-critério que pondere estes fatores apropriadamente, potencialmente utilizando técnicas de otimização multi-objetivo (*Pareto*) ou aprendizado por reforço para aprender pesos ótimos dinamicamente.

6.7.5 Balanceamento Cross-data center

Extensão do algoritmo para balanceamento não apenas entre servidores de um único *data center*, mas entre múltiplos *data centers* geograficamente distribuídos. Isto introduz trade-offs complexos entre capacidade de recursos (servidor menos carregado pode estar longe) e proximidade geográfica (servidor próximo pode estar mais carregado), requerendo modelagem sofisticada de custos multi-dimensionais.

6.7.6 Adaptação para Edge Computing e 5G

Exploração de como o algoritmo pode ser adaptado para ambientes de *edge computing* e redes *5G*, onde recursos computacionais estão distribuídos em torres de celular e pontos de presença de borda. Nestes contextos, mobilidade dos usuários, *handoffs* entre células, e latências ultra-baixas introduzem desafios adicionais.

6.7.7 Estudo de Interação com Algoritmos ABR

Investigação sistemática de como diferentes algoritmos *ABR* (*BOLA*, *MPC*, *Pensieve*) interagem com estratégias de balanceamento de carga. Exploração de *co-design* onde balanceador e *ABR* cooperam ativamente, com balanceador informando capacidade esperada e *ABR* ajustando decisões de qualidade proativamente.

6.7.8 Implementação de Cache Inteligente Integrado

Integração de estratégias de *cache* inteligente que considerem não apenas popularidade de conteúdo mas também capacidade de recursos dos servidores. Servidores mais capazes poderiam *cache*ar conteúdo de maior demanda, enquanto servidores limitados focariam em conteúdo de nicho, otimizando conjuntamente *cache* e balanceamento.

6.7.9 Análise de Custos Operacionais

Estudo econômico comparando custos operacionais (energia, largura de banda, depreciação de *hardware*) sob diferentes estratégias de balanceamento. Balanceamento ótimo pode não ser aquele que maximiza QoE absoluta, mas que maximiza QoE por unidade de custo, considerando realidades econômicas de operação de CDN.

6.7.10 Content Steering com Múltiplas CDNs

Extensão do algoritmo de monitoramento de memória para cenários de *Content Steering* em MPEG-DASH, onde clientes podem alternar dinamicamente entre múltiplas CDNs durante uma sessão de *streaming*. Este trabalho focou em balanceamento intra-CDN, operando dentro de um único Ponto de Presença (PoP). Uma direção promissora é aplicar princípios similares de monitoramento de recursos para decisões inter-CDN, onde cada CDN possui seu próprio balanceador de carga interno, mas um sistema de *steering* de nível superior decide qual CDN utilizar com base em métricas agregadas de capacidade.

Content Steering, padronizado recentemente no DASH-IF, permite que clientes recebam instruções sobre quais CDNs priorizar através de documentos de steering atualizados dinamicamente. Integrar monitoramento de memória e carga neste contexto permitiria *steering content-aware* e *resource-aware*, direcionando tráfego não apenas baseado em proximidade geográfica ou custo, mas também em disponibilidade real de recursos em cada CDN. Isto é especialmente relevante para provedores de *streaming* que utilizam múltiplas CDNs simultaneamente para redundância e otimização de custos.

6.7.11 Direct Server Return (DSR)

Investigação de arquiteturas de balanceamento baseadas em *Direct Server Return*, onde o balanceador de carga processa apenas requisições de entrada (caminho de ida), mas respostas são enviadas diretamente dos servidores aos clientes, evitando o balanceador no caminho de retorno. Em *streaming* de vídeo, onde respostas (segmentos de vídeo) são substancialmente maiores que requisições, DSR pode reduzir dramaticamente carga no balanceador e latência percebida.

Aplicar monitoramento de memória em arquitetura DSR introduz desafios interessantes: o balanceador não observa o tráfego de resposta, não podendo medir diretamente taxa de transferência ou sucesso de entregas. Seria necessário confiar exclusivamente em métricas do lado do servidor (memória, disco, *CPU*) e potencialmente informações agregadas reportadas periodicamente. Avaliar trade-offs entre benefícios de DSR (menor latência, maior vazão) e complexidade de monitoramento é direção valiosa, especialmente para CDNs de ultra-larga escala onde o próprio balanceador pode tornar-se gargalo.

6.7.12 Software-Defined Networking (SDN)

Exploração de como princípios de *Software-Defined Networking* podem potencializar balanceamento dinâmico baseado em recursos. Em arquiteturas SDN, *switches* são programáveis através de controladores centralizados, permitindo reconfiguração de fluxos de tráfego em tempo real sem intervenção manual. Este paradigma alinha-se naturalmente com balanceamento adaptativo baseado em monitoramento.

Uma direção promissora é instrumentar *switches* SDN com informações de carga dos servidores, permitindo decisões de encaminhamento no próprio switch baseadas em métricas

atuais. O controlador SDN poderia receber métricas de memória e disco dos servidores (via MQTT ou outro protocolo), calcular pesos probabilísticos usando o algoritmo proposto, e programar tabelas de fluxo nos *switches* para distribuir tráfego proporcionalmente. Isto move decisões de balanceamento para mais próximo da borda da rede, potencialmente reduzindo latência de decisão e eliminando saltos adicionais através de balanceadores dedicados.

Adicionalmente, SDN permite experimentação com esquemas de balanceamento sofisticados como *flow-based routing* (diferentes fluxos para diferentes servidores) versus *packet-based routing* (cada pacote pode ir para servidor diferente, requerendo estados complexos), e *elephant vs. mice flow handling* (grandes sessões de *streaming* tratadas diferentemente de pequenas requisições). A integração de monitoramento de recursos com SDN representa fronteira empolgante para balanceamento de carga de próxima geração.

6.8 REPRODUTIBILIDADE DOS EXPERIMENTOS

Todo o código-fonte, scripts de configuração, infraestrutura experimental, e dados utilizados estão disponibilizados publicamente em repositório Git no endereço:

<https://github.com/peedrofernandes/memory-oriented-load-balancer.git>

O repositório está organizado de forma a facilitar a reprodução exata de cada cenário experimental.

- Servidor MPEG-DASH implementado em *C#* com *ASP.NET Core*
- Cliente de validação com visualização em tempo real
- Gerador de carga parametrizável em *Python*
- Sistema de monitoramento de recursos em *Rust*
- Infraestrutura de comunicação via MQTT
- Arquivo de vídeo de teste codificado em múltiplas qualidades
- Scripts de orquestração com *Docker Compose*
- Ferramentas de análise e geração de gráficos

Para reproduzir os experimentos específicos, foram criadas *branches* dedicadas para cada combinação de cenário e estratégia de balanceamento, totalizando 16 configurações experimentais:

- **Cenário 1** (100 usuários, servidores heterogêneos):
 - `scenario-1-memory-monitoring-6s`

- scenario-1-memory-monitoring-30s
- scenario-1-round-robin
- scenario-1-random-selection
- **Cenário 2** (100 usuários, servidores homogêneos):
 - scenario-2-memory-monitoring-6s
 - scenario-2-memory-monitoring-30s
 - scenario-2-round-robin
 - scenario-2-random-selection
- **Cenário 3** (1000 usuários, servidores heterogêneos):
 - scenario-3-memory-monitoring-6s
 - scenario-3-memory-monitoring-30s
 - scenario-3-round-robin
 - scenario-3-random-selection
- **Cenário 4** (1000 usuários, servidores homogêneos):
 - scenario-4-memory-monitoring-6s
 - scenario-4-memory-monitoring-30s
 - scenario-4-round-robin
 - scenario-4-random-selection

Cada *branch* contém configurações pré-ajustadas (arquivos `docker-compose.yml`, parâmetros de gerador de carga, configurações de servidores) correspondentes exatamente às condições experimentais descritas no Capítulo 5. Para reproduzir um cenário específico, basta fazer *checkout* da *branch* correspondente e executar os scripts de orquestração fornecidos.

Esta organização permite não apenas validação independente dos resultados apresentados, mas também facilita experimentação com variações dos cenários (diferentes números de usuários, outras configurações de heterogeneidade, intervalos de atualização alternativos) ou adaptação do sistema para outros contextos de pesquisa. A documentação completa, incluindo instruções detalhadas de execução e requisitos de sistema, está disponível no arquivo `README.md` do repositório.

6.9 CONSIDERAÇÕES FINAIS

Este trabalho demonstrou de forma conclusiva que o monitoramento inteligente de recursos, especificamente memória principal e taxa de leitura de disco, possibilita melhorias substanciais em Qualidade de Experiência para *streaming* de vídeo adaptativo em Redes de Distribuição de Conteúdo com servidores heterogêneos. As reduções de latência de até 85,7%, eliminação de travamentos em cenários críticos, e manutenção de qualidade de vídeo alta e consistente validam a abordagem proposta e demonstram sua superioridade sobre estratégias convencionais em contextos apropriados.

A identificação da heterogeneidade como fator crítico determina claramente quando investir em balanceamento inteligente: essencial em CDNs reais com servidores de múltiplas gerações e capacidades, mas de valor marginal em ambientes homogêneos padronizados. A robustez demonstrada com dados moderadamente desatualizados resolve uma preocupação fundamental sobre escalabilidade, mostrando que o algoritmo pode operar eficientemente mesmo com intervalos de atualização maiores, reduzindo custos de monitoramento proporcionalmente.

Do ponto de vista prático, este trabalho oferece não apenas validação acadêmica de conceitos, mas uma solução implementável e testada que pode ser adaptada para sistemas de produção. A implementação de código aberto, infraestrutura experimental reutilizável, e análise detalhada de diferentes cenários fornecem base sólida para profissionais da indústria avaliarem aplicabilidade em seus contextos específicos.

Para a indústria de *streaming* de vídeo, que movimenta bilhões de dólares globalmente e cujo crescimento é impulsionado por demanda crescente por conteúdo em alta qualidade, qualquer melhoria em QoE traduz-se diretamente em maior satisfação de usuários, redução de abandono (*churn*), e vantagem competitiva. Os resultados deste trabalho sugerem que provedores de CDN operando em ambientes heterogêneos — a grande maioria — podem beneficiar significativamente de balanceamento baseado em monitoramento de recursos.

Olhando para o futuro, o *streaming* de vídeo continua evoluindo: resolução 4K/8K, realidade virtual, transmissão ao vivo interativa, e novos *codecs* (AVI, VVC) aumentam complexidade e requisitos de recursos. Simultaneamente, *edge computing* e redes 5G distribuem conteúdo ainda mais próximo aos usuários, criando ambientes inerentemente heterogêneos com milhares de pontos de presença de capacidades variadas. Neste contexto, balanceamento inteligente e adaptativo não é luxo opcional, mas necessidade fundamental.

Este trabalho estabelece fundação sólida, mas há muito a explorar. As direções de trabalho futuro delineadas — desde implantação em produção até aprendizado de máquina preditivo — representam apenas o início de uma jornada mais longa rumo a sistemas de distribuição de conteúdo verdadeiramente inteligentes, que aprendem, adaptam-se, e otimizam-se continuamente para proporcionar a melhor experiência possível aos usuários, independentemente de onde estejam ou quais recursos estejam disponíveis.

A mensagem central permanece clara: em um mundo onde conteúdo de vídeo domina o

tráfego da Internet e expectativas de usuários por qualidade aumentam continuamente, decisões informadas por dados sobre onde e como distribuir esse conteúdo não são apenas técnicas — são estratégicas. O balanceamento de carga baseado em monitoramento de recursos é um passo importante nessa direção, demonstrando que sistemas que observam, interpretam e respondem ao estado real de seus componentes superam aqueles que operam cegamente, especialmente quando confrontados com a heterogeneidade e variabilidade inerentes à infraestrutura moderna de Internet.

REFERÊNCIAS

- ADEWOJO, Adekunbi; BASS, Julian. A novel weight-assignment load balancing algorithm for cloud applications. **SN Computer Science**, v. 4, 03 2023. Citado na página 29.
- ALI, Waris; FANG, Chao; KHAN, Akmal. A survey on the state-of-the-art cdn architectures and future directions. **Journal of Network and Computer Applications**, v. 236, 2025. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804525000037>. Citado na página 13.
- Apple Inc. **About the Virtual Memory System**. 2012. Documentação de arquivo que descreve os conceitos fundamentais do sistema de memória virtual do macOS. Disponível em: <https://developer.apple.com/library/archive/documentation/Performance/Conceptual/ManagingMemory/Articles/AboutMemory.html>. Citado na página 26.
- Apple Inc. **IOKit Fundamentals: Introduction**. 2013. Documentação de arquivo que apresenta o framework I/O Kit, fundamental para o monitoramento de dispositivos, incluindo I/O de disco. Disponível em: <https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/Introduction/Introduction.html>. Citado na página 26.
- ARPACI-DUSSEAU, R.H. et al. The architectural costs of streaming i/o: A comparison of workstations, clusters, and smps. In: **Proceedings 1998 Fourth International Symposium on High-Performance Computer Architecture**. [S.l.: s.n.], 1998. Citado na página 12.
- BEGEN, Ali; AKGUL, Tankut; BAUGHER, Mark. Watching video over the web: Part 1: Streaming protocols. **IEEE Internet Computing**, v. 15, n. 2, 2011. Citado 2 vezes nas páginas 19 e 20.
- BELGAUM, Mohammad Riyaz et al. A systematic review of load balancing techniques in software-defined networking. **IEEE Access**, v. 8, 2020. Citado na página 18.
- BELLA, Mochamad Rexa Mei; DATA, Mahendra; YAHYA, Widhi. Web server load balancing based on memory utilization using docker swarm. In: **2018 International Conference on Sustainable Information Engineering and Technology (SIET)**. [S.l.: s.n.], 2018. Citado na página 29.
- BLIAL, Othmane; MAMOUN, Mouad Ben; BENAINI, Redouane. An overview on sdn architectures with multiple controllers. **Journal of Computer Networks and Communications**, v. 2016, n. 1, 2016. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/9396525>. Citado na página 19.
- BOSS, Greg et al. Cloud computing. **IBM white paper**, [http://download.boulder.ibm.com/ibmdl/pub/software/dw/wes/hipods/Cloud ...](http://download.boulder.ibm.com/ibmdl/pub/software/dw/wes/hipods/Cloud...), v. 321, 2007. Citado na página 26.
- CHOWDHURY, N.M. Mosharaf Kabir; BOUTABA, Raouf. A survey of network virtualization. **Computer Networks**, v. 54, n. 5, 2010. ISSN 1389-1286. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1389128609003387>. Citado na página 28.
- Cisco Systems. **Cisco Annual Internet Report (2018–2023) White Paper**. 2020. Cisco White Paper. Accessed: 2025-01-10. Disponível em: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>. Citado na página 11.

DAHLIN, M. Interpreting stale load information. **IEEE Transactions on Parallel and Distributed Systems**, v. 11, n. 10, 2000. Citado 4 vezes nas páginas 27, 32, 33 e 38.

EAGER, Derek L.; LAZOWSKA, Edward D.; ZAHORJAN, John. Adaptive load sharing in homogeneous distributed systems. **IEEE Transactions on Software Engineering**, SE-12, n. 5, 1986. Citado na página 27.

EUGSTER, Patrick Th et al. The many faces of publish/subscribe. **ACM computing surveys (CSUR)**, ACM New York, NY, USA, v. 35, n. 2, p. 114–131, 2003. Citado na página 36.

HAMDAN, Mosab et al. A comprehensive survey of load balancing techniques in software-defined network. **Journal of Network and Computer Applications**, v. 174, p. 102856, 2021. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804520303222>. Citado na página 18.

HAPROXY. **HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer**. 2025. Acessado em: 27 de março de 2025. Disponível em: <https://www.haproxy.org/>. Citado na página 29.

JANE, Kimberly. Scalability issues in database systems: Strategies for scaling databases horizontally and vertically. 03 2025. Citado na página 16.

KIM, Junyeop; WON, Youjip. Dynamic load balancing for efficient video streaming service. In: **2015 International Conference on Information Networking (ICOIN)**. [S.l.: s.n.], 2015. p. 216–221. Citado na página 30.

MA, Chen; CHI, Yuhong. Evaluation test and improvement of load balancing algorithms of nginx. **IEEE Access**, v. 10, p. 14311–14324, 2022. Citado na página 29.

MAHMOOD, Ibrahim et al. Web server performance improvement using dynamic load balancing techniques: A review. **Asian Journal of Research in Computer Science**, 06 2021. Citado na página 30.

MEI, Yiduo et al. Performance analysis of network i/o workloads in virtualized data centers. **IEEE Transactions on Services Computing**, v. 6, n. 1, 2013. Citado na página 12.

Microsoft Corporation. **Memory performance counters**. 2023. Referência oficial para os contadores de desempenho de memória no Windows. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/perfctr/mem-perfctr>. Citado na página 26.

Microsoft Corporation. **PhysicalDisk object**. 2023. Referência oficial para os contadores de desempenho do objeto PhysicalDisk, usado para monitoramento de I/O de disco. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/perfctr/physicaldisk-object>. Citado na página 26.

MIRCHANDANEY, R.; TOWSLEY, D.; STANKOVIC, J.A. Analysis of the effects of delays on load sharing. **IEEE Transactions on Computers**, v. 38, n. 11, 1989. Citado na página 27.

MITZENMACHER, Michael. How useful is old information? **Parallel and Distributed Systems, IEEE Transactions on**, v. 11, 02 2000. Citado na página 27.

NYGREN, Erik; SITARAMAN, Ramesh K.; SUN, Jennifer. The akamai network: a platform for high-performance internet applications. **SIGOPS Oper. Syst. Rev.**, Association for Computing Machinery, New York, NY, USA, v. 44, n. 3, p. 2–19, ago. 2010. ISSN 0163-5980. Disponível em: <https://doi.org/10.1145/1842733.1842736>. Citado na página 24.

PANDIAN, A. Pasumpon; FERNANDO, Xavier; HAOXIANG, Wang (Ed.). **Computer Networks, Big Data and IoT: Proceedings of ICCBI 2021**. [S.l.]: Springer Nature Singapore, 2022. v. 117. (Lecture Notes on Data Engineering and Communications Technologies, v. 117). ISBN 978-981-19-0897-2. Citado na página 17.

PASSOS, Edenilson Jônatas dos; FIORESE, Adriano. **Balanceamento de Tráfego Entre Servidores de Vídeo MPEG-DASH Em Redes Definidas Por Software**. 2019. <https://doi.org/10.5753/wgrs.2019.7684>. Accessed 15 May 2024. Citado na página 12.

PRESTON, Sam. **Finding the Best Servers to Answer Queries—Edge DNS and Anycast**. Akamai Technologies, 2021. Akamai Blog. Disponível em: <https://www.akamai.com/blog/edge/finding-the-best-servers-to-answer-queries-edge-dns-and-anycast>. Citado na página 22.

SHIVARATRI, Niranjan; KRUEGER, Phillip; SINGHAL, Manish. Load distributing in locally distributed systems. **Computer**, v. 25, 01 1993. Citado na página 27.

SODAGAR, Iraj. The mpeg-dash standard for multimedia streaming over the internet. **IEEE MultiMedia**, v. 18, n. 4, 2011. Citado 3 vezes nas páginas 19, 20 e 21.

STOCKHAMMER, Thomas. Dynamic adaptive streaming over http: Standards and design principles. In: . [S.l.: s.n.], 2011. Citado 2 vezes nas páginas 19 e 20.

TAILOR SUSHANT CHOUDHARY, Vinay Jain Hemang. **Rise of NewSQL**. 2014. Accessed: 2025-03-27. Disponível em: https://dlwqtxts1xzle7.cloudfront.net/84529847/pid-212015010-libre.pdf?1650446168=&response-content-disposition=inline%3B+filename%3DRise_of_NewSQL.pdf&Expires=1743081319&Signature=PrYDe1OEWBWnlHmNa8ucCPryddRUjK5rqTYuRcvGMkLt5SWDKJNVrRE585GUmF~7VpfvOR8CwRQ_&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA. Citado na página 12.

The Linux Kernel Developers. **I/O statistics fields for block devices**. 2011. Descreve os campos fornecidos pelo arquivo /proc/diskstats. Disponível em: <https://www.kernel.org/doc/Documentation/iostats.txt>. Citado na página 26.

The Linux Kernel Developers. **The /proc Filesystem**. 2020. Documentação do kernel sobre a estrutura e os arquivos do sistema de arquivos /proc, incluindo /proc/meminfo. Disponível em: <https://www.kernel.org/doc/Documentation/filesystems/proc.txt>. Citado na página 26.

TSAO, Shiao-Li et al. Data allocation and dynamic load balancing for distributed video storage server. **Journal of Visual Communication and Image Representation**, v. 10, n. 2, p. 197–218, 1999. ISSN 1047-3203. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1047320399904200>. Citado na página 30.

VAKALI, A.; PALLIS, G. Content delivery networks: status and trends. **IEEE Internet Computing**, v. 7, n. 6, 2003. Citado 2 vezes nas páginas 23 e 25.

WANG, Zheng; HUANG, Jun; ROSE, Scott. Evolution and challenges of dns-based cdns. **Digital Communications and Networks**, v. 4, n. 4, 2018. ISSN 2352-8648. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2352864817300731>. Citado na página 23.

WU, Song et al. A large-scale study of i/o workload's impact on disk failure. **IEEE Access**, v. 6, 2018. Citado na página 12.

GLOSSÁRIO

Anycast: Técnica de rede onde a mesma faixa de endereços IP é utilizada em múltiplos servidores em locais diferentes. As requisições são roteadas para o servidor topologicamente mais próximo do cliente, melhorando o desempenho e a resiliência.

Broker: Em sistemas distribuídos, um intermediário que gerencia a comunicação entre diferentes componentes de *software*. Ele recebe mensagens de um publicador e as direciona para os assinantes interessados.

Buffer: Uma área de memória temporária usada para armazenar dados enquanto eles estão sendo movidos de um lugar para outro, compensando diferenças de velocidade entre o produtor e o consumidor dos dados.

Cache: Componente de *hardware* ou *software* que armazena dados para que futuras requisições por esses mesmos dados possam ser atendidas de forma mais rápida; os dados armazenados em um *cache* podem ser o resultado de uma computação anterior ou uma cópia de dados armazenados em outro lugar.

Cloud: Modelo de computação que permite o acesso sob demanda, via rede, a um conjunto compartilhado de recursos computacionais configuráveis (ex: redes, servidores, armazenamento, aplicações e serviços).

contêiner: Uma unidade padrão de *software* que empacota o código e todas as suas dependências para que a aplicação seja executada de forma rápida e confiável de um ambiente computacional para outro.

Cookies: Pequenos arquivos de texto que os sites enviam para o navegador de um usuário para armazenar informações sobre sua sessão, preferências ou estado de autenticação, permitindo uma experiência de navegação personalizada.

data center: Instalação física centralizada que abriga a infraestrutura de TI de uma organização, incluindo servidores, sistemas de armazenamento de dados, equipamentos de rede e outros componentes necessários para operar e gerenciar aplicações e dados.

Docker: Plataforma de *software* de código aberto usada para desenvolver, empacotar e executar aplicações em *contêineres*, permitindo a separação de suas aplicações da infraestrutura para que você possa entregar *software* rapidamente.

Ethernet: Família de tecnologias de rede de computadores com fio para redes locais (LANs), definindo o cabeamento e os sinais elétricos para a camada física, e um formato de pacote e protocolo para a camada de enlace de dados do modelo OSI.

Failover: Processo de comutação automática para um sistema ou componente de backup em caso de falha do sistema primário, garantindo a continuidade do serviço com o mínimo de interrupção.

Firewall: Dispositivo de segurança de rede que monitora o tráfego de rede de entrada e saída e decide permitir ou bloquear tráfegos específicos com base em um conjunto definido de regras de segurança.

Gateway: Nó de rede que conecta duas redes diferentes que usam protocolos de comunicação

distintos, atuando como um ponto de entrada e saída para o tráfego de rede.

Host: Qualquer dispositivo computacional conectado a uma rede que oferece recursos, serviços e aplicações aos usuários ou a outros nós na rede.

Hypervisor: *Software, firmware* ou *hardware* que cria e executa máquinas virtuais (VMs). Ele permite que um único computador *host* suporte múltiplas VMs, compartilhando virtualmente seus recursos, como processador e memória.

Middleware: *Software* que atua como uma ponte entre um sistema operacional e as aplicações nele executadas, oferecendo serviços comuns e funcionalidades de conectividade que não são fornecidas nativamente pelo sistema operacional.

Mininet: Emulador de rede que cria uma rede de *hosts* virtuais, *switches*, controladores e links em um único kernel Linux, permitindo o desenvolvimento, teste e prototipagem de aplicações de Redes Definidas por *Software* (SDN).

Namespaces: Recurso do kernel Linux que particiona recursos do kernel de forma que um conjunto de processos veja um conjunto de recursos, enquanto outro conjunto de processos vê um conjunto diferente, sendo a base tecnológica dos *contêineres*.

NetBox: Aplicação *web* de código aberto projetada para gerenciamento de infraestrutura de rede (IPAM - IP Address Management) e de infraestrutura de *data center* (DCIM - Data Center Infrastructure Management).

OpenFlow: Protocolo de comunicação que permite que o plano de controle de uma rede definida por *software* (SDN) interaja e gerencie os dispositivos do plano de dados (como *switches* e roteadores).

Peering: Interconexão voluntária de redes de Internet administrativamente separadas com o propósito de trocar tráfego entre os usuários de cada rede.

Player (aplicação Player MPEG-DASH): Aplicação cliente responsável por requisitar os segmentos de mídia de um servidor HTTP e decodificá-los para reprodução, adaptando a qualidade do vídeo com base nas condições da rede.

Proxy: Servidor que atua como intermediário para requisições de clientes buscando recursos de outros servidores. Ele pode ser usado para filtrar requisições, registrar logs, ou compartilhar conexões.

Python: Linguagem de programação de alto nível, interpretada, de *script*, imperativa, orientada a objetos, funcional, de tipagem dinâmica e forte, conhecida por sua sintaxe clara e legibilidade.

Remote: Refere-se a um sistema, dispositivo ou recurso que está localizado em um local diferente do usuário ou do sistema principal, sendo acessado através de uma rede.

Round-Robin: Algoritmo de escalonamento de processos ou de balanceamento de carga que distribui as tarefas ou o tráfego de rede de forma sequencial e cíclica entre os recursos disponíveis.

Rust: Linguagem de programação multiparadigma compilada, focada em segurança, especialmente a segurança de concorrência e de memória, mantendo um alto desempenho.

Script: Sequência de comandos armazenada em um arquivo de texto simples, executada por um interpretador, para automatizar tarefas ou configurar um sistema.

Shell: Programa de interface de usuário, geralmente baseado em linha de comando, que permite aos usuários controlar o computador através da inserção de comandos textuais.

Socket: Ponto final de um fluxo de comunicação bidirecional entre processos em uma rede. É definido por um endereço IP e um número de porta.

Stalls: Em *streaming* de vídeo, uma interrupção na reprodução causada quando o *buffer* do *player* fica vazio, forçando o vídeo a pausar enquanto mais dados são baixados.

Startup: O processo de inicialização de um sistema computacional ou de uma aplicação, envolvendo o carregamento do sistema operacional, serviços e programas necessários para sua operação.

Streaming: Tecnologia de transmissão contínua de dados, geralmente áudio e vídeo, de um servidor para um cliente, permitindo que a reprodução comece antes que todo o arquivo tenha sido baixado.

Switches: Dispositivos de rede que operam na camada de enlace de dados (camada 2 do modelo OSI) para conectar múltiplos dispositivos em uma mesma rede local (LAN), encaminhando pacotes de dados apenas para o destinatário específico.

WebSockets: Protocolo de comunicação que proporciona canais de comunicação *full-duplex* sobre uma única conexão TCP, permitindo a interação em tempo real entre um navegador e um servidor *web*.

Windows: Família de sistemas operacionais gráficos desenvolvidos, comercializados e vendidos pela Microsoft, amplamente utilizados em computadores pessoais.